



Introduction into Cryptography Fundamentals



Catalin Boja, Ph.D.
catalin.boja@ie.ase.ro

October 2024

Catalin Boja

Your trainer

- Full Professor at Bucharest University of Economic Studies / Faculty of Cybernetics
- PhD in Informatics / Software optimization
- Board member and professor at IT&C Security Master program (<https://ism.ase.ro/>)
- More than 20 years of experience in programming (C/C++/Java/C#/Python and others)
- Developed projects on different platforms (J2ME, Android, ASP.NET, .NET Core, Java Spring, Angular)
- Training others coding skills and cybersecurity stuff since 2004



Objectives

Gaining theoretical and practical knowledge needed to understand and use in a correct manner, cryptographic algorithms, and to reason about computer security

Assures you the full knowledge regarding meaning and usage of Computer Security concepts



Course

- **Activities:** Exam 70% + Course & Laboratory 30%
- **Evaluation:** Written exam (quiz) + short quizzes/challenges during the labs
- Minimum 5 in the exam



Prerequisites

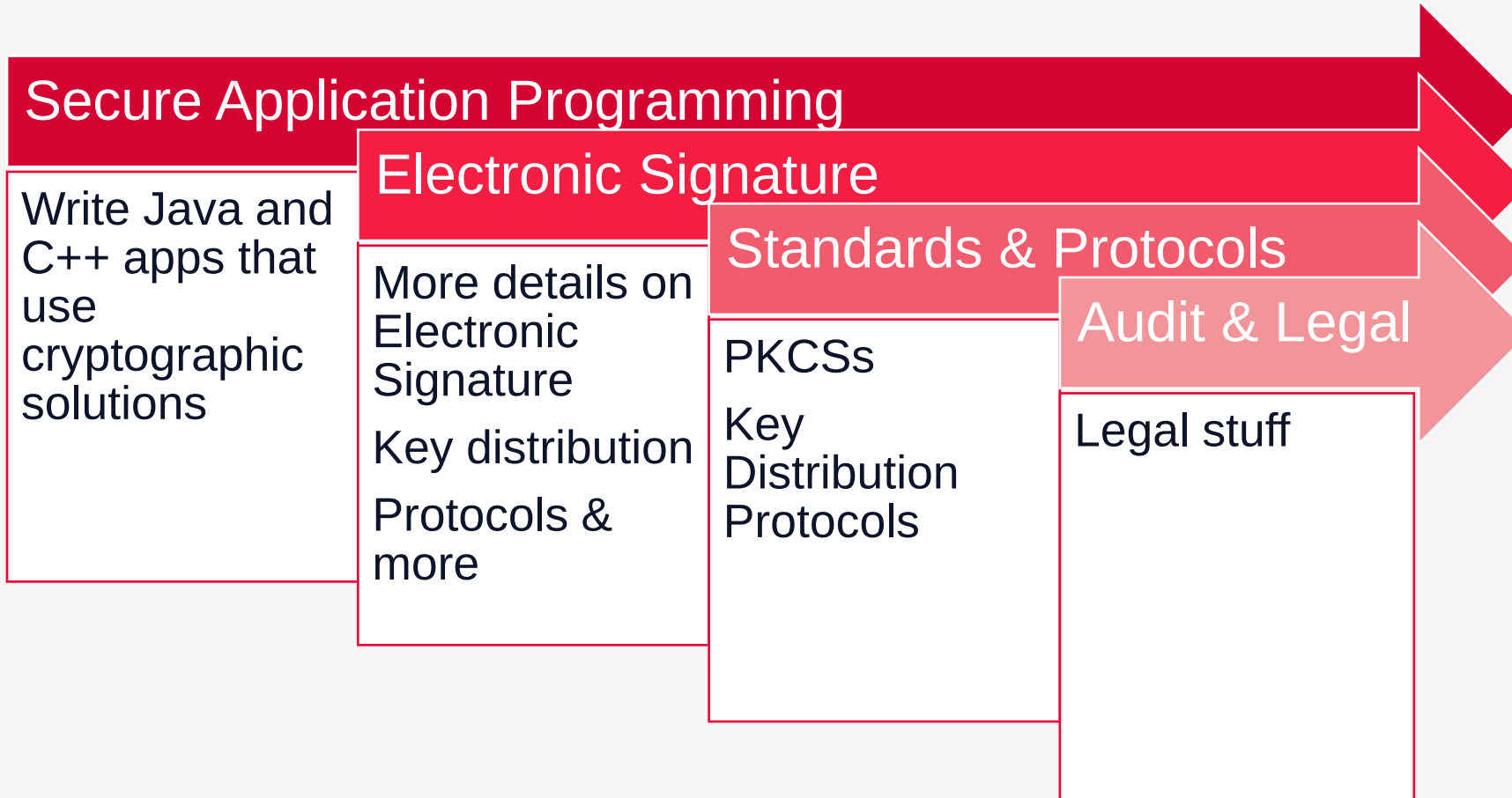
Things to remember and refresh

- Basic IT/Computer/Computer Science knowledge
- Base 2 (Binary) and Base 16 (Hexadecimal) operations.....besides Base 10
- Basic mathematical background



Follow Up

Course relation with the next ones



Follow Up

Certifications

Covers Cryptography topics for all Cybersecurity related certifications

- Certified Information Systems Security Professional (CISSP)
 - Broad, management-oriented certification for experienced professionals
- Certified Ethical Hacker (CEH)
 - Focuses on ethical hacking and penetration testing
- Certified Information Security Manager (CISM)
 - Emphasizes management and strategy in information security
- Certified Information Systems Auditor (CISA)
 - Best for those specializing in auditing and compliance
- CompTIA Security+
 - A foundational cert for those new to cybersecurity
- Offensive Security Certified Professional (OSCP)
 - Recognized for hands-on penetration testing skills



Agenda

Some major topics

DAY 1

- Concepts
- Mathematical Background
- Prime numbers
- Random Number Generators

DAY 2

- Hash (Message Digest) functions and Data Integrity
- Message Authentication Codes (MAC) and Data Integrity

DAY 3

- Data Confidentiality and Encryption
- Transposition ciphers
- Substitution ciphers
- OTP (One Time Pad) ciphers
- Complex ciphers

DAY 4

- Encryption methods
- Authentication and Asymmetric Encryption
- Secure Storage
- Post Quantum Cryptography



Tools and References

This is just the beginning

- Tools
 - CrypTool (<https://www.cryptool.org>)
 - Python scripts (<https://pypi.org/project/cryptography>)
 - Kali Linux (<https://www.kali.org>)
- References
 - Schneier, B., "Applied Cryptography Second Edition", John Wiley & Sons, Inc., New York, 1996
 - Cybrary, <https://www.cybrary.it>
 - Coursera, <https://www.coursera.org>
 - edX, <https://www.edx.org>
 - www.wikipedia.com / www.google.com/ openAI





In cybersecurity, common mistakes can lead to significant vulnerabilities and potential breaches.



Weak Passwords

- Using easily guessable passwords or default passwords



Insufficient Encryption

- Not encrypting sensitive data at rest or in transit



Improper Use of Crypto

- Not using cryptographic algorithms for their designed objective
- Not understanding that each crypto algorithm provides solution for a limited range of problems

Consequences

Jeep Cherokee Hack (2015)

- Who: Charlie Miller and Chris Valasek
- How: Exploited a vulnerability in the Uconnect infotainment system.
- What: Remote control over steering, brakes, and transmission via the internet.
- Consequence: Led to a recall of 1.4 million vehicles by Fiat Chrysler to fix the security flaw.
- <https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/>
- <https://www.kaspersky.com/blog/blackhat-jeep-cherokee-hack-explained/9493/>

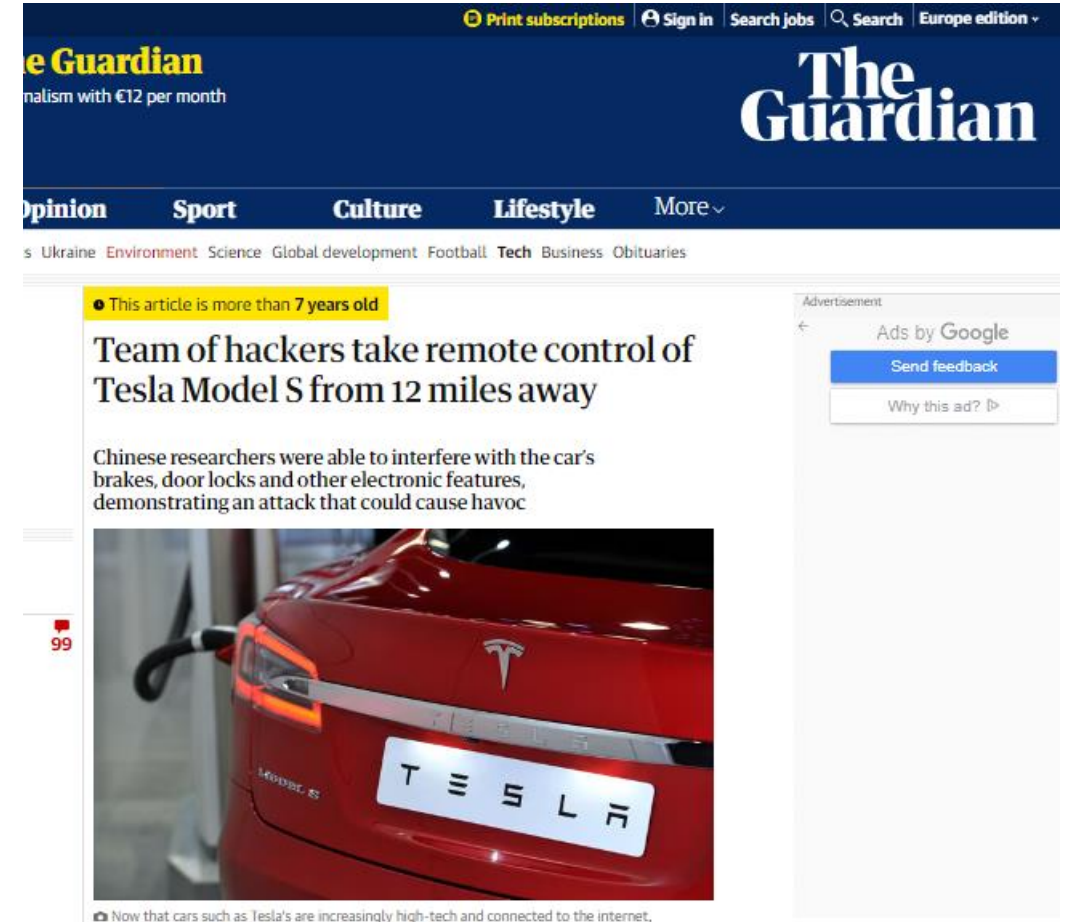


Source: WIRED, [Hackers Remotely Kill a Jeep on the Highway—With Me in It](https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/)

Consequences

Tesla Model S Hack (2016)

- Who: Keen Security Lab
- How: Exploited the car's CAN bus through the web browser.
- What: Gained control over the braking system from a distance of 12 miles.
- Consequence: Tesla issued an over-the-air update to fix the vulnerability.
- <https://www.theguardian.com/technology/2016/sep/20/tesla-model-s-chinese-hack-remote-control-brakes>
- <https://www.theverge.com/2016/9/19/12985120/tesla-model-s-hack-vulnerability-keen-labs>



Source: The Guardian, [Team of hackers take remote control of Tesla Model S from 12 miles away](https://www.theguardian.com/technology/2016/sep/20/tesla-model-s-chinese-hack-remote-control-brakes)

Consequences

Adobe (2013)

- Nearly 150 million people have been affected by a loss of customer data by Adobe, over 20 times more than the company admitted [in its initial statement last week](#).
- As well as allowing the data to be stolen in the first place, Adobe made two other serious errors when storing the data. Firstly, it encrypted all the passwords with the same key; secondly, the encryption used a method (ECB mode) which renders the encrypted data insecure.
- Every identical password also looks identical when encrypted. So if the database shows 1.9 million people whose password, when encrypted, reads "EQ7fIpT7i/Q", then researchers know that they all have the same password.

theguardian

News Sport Comment Culture Business Money Life & style

News Technology Adobe

Did your Adobe password leak? Now you and 150m others can check

Leak is 20 times worse than the company initially revealed, and could put huge numbers of peoples' online lives at risk

Alex Hern

Follow @alexhern Follow @guardiantech
theguardian.com, Thursday 7 November 2013 12.27 GMT

Jump to comments (25)



Adobe's HQ. The company leaked over 100m users' details. Photograph: PAUL SAKUMA/ASSOCIATED PRESS

Consequences

Facebook (2018)

- The Facebook 2018 hack was one of the most significant cybersecurity breaches the company had experienced, affecting around 50 million accounts.
- In September 2018, Facebook announced that attackers exploited a vulnerability in the platform's code that allowed them to steal "access tokens," which are essentially digital keys that keep users logged into Facebook without having to re-enter their password.
- The vulnerability was in Facebook's "View As" feature, which allows users to see how their profile appears to others. The issue was linked to a combination of three bugs that arose in Facebook's code after an update to the video uploading feature in July 2017. When exploited together, these bugs allowed attackers to gain access tokens, effectively hijacking accounts.
- In July 2019, Facebook was fined \$5 billion by the U.S. Federal Trade Commission (FTC) for privacy violations, partly influenced by events like this breach



Consequences

This year so far (2024)

- **Trello** – In January 2024, project management tool Trello was breached, exposing data of over 15 million users. The leak involved email addresses, usernames, and other account details, highlighting security weaknesses in its public
- **Bank of America** – A ransomware attack on Infosys McCamish Systems in late 2023 was revealed in early 2024. It compromised the personal information of over 57,000 Bank of America customers, including names, social security numbers, and other sensitive data
- **VARTA** – German battery manufacturer VARTA suffered a cyberattack in February 2024, leading to the shutdown of five production plants. The attack impacted IT systems and disrupted production, though the exact details of the data compromised remain unclear
- **750 Million Indian Telecom Users** – A massive data breach in January 2024 affected 750 million telecom users in India. The breach exposed names, mobile numbers, addresses, and Aadhaar numbers, with the data being sold on the dark web
- <https://www.techradar.com/pro/top-data-breaches-and-cyber-attacks-in-2024>



Top data breaches and cyber attacks in 2024

Features

By Christian Cawley published May 22, 2024

Already 20 major incidents have occurred so far

When you purchase through links on our site, we may earn an affiliate commission. [Here's how it works.](#)



1. Concepts



Concepts

- Keywords and vocabulary
- Cryptography
- Key ingredients of a Cryptographic system
- Types of security
- Vulnerabilities
- Types of attacks
- Objectives



Concepts

Some definitions and differences

- **Cryptography** – **secret** writing science; the science of information security
- **Cryptanalysis** – science of “breaking” ciphertexts without knowing cipher key
- **Cryptology** – mathematic field that studies the mathematical fundamentals of cryptography
- **Steganography** - the art of **hiding** information;
 - the secret message is hidden in a public one (a image, sound file, text);
 - **is NOT Cryptography**
 - ...later covered by the Multimedia Security course (2nd year, 1st semester)



Cryptography

There is no Cybersecurity without it

- **Secret writing science**; the science of information security
- Provides concepts for:
 - Secret key establishment
 - Secure communication
 - Secure data
 - Digital signatures
 - Anonymous communication (Mix Net)
 - Anonymous digital cash (crypto-currency)
 - Electronic voting or auctions
 - Protocols (like “Zero knowledge”)
- Used to secure data in:
 - Networks: HTTPS, SSL/TLS, 802.11i WPA2 (Wi-Fi Protected Access), GSM, Bluetooth
 - Computers and mobile devices drives: TrueCrypt



Cryptography is NOT

Do's & Don'ts

- Is not a solution for all security problems: social engineering, reverse engineering, software bugs, design errors (see WEP - Wired Equivalent Privacy);
- Is not a solution when is not used or not implemented properly
- Is not an ad-hoc design or your personal invention (**DON'T TRUST PROPRIETARY SOLUTIONS**)



Keywords

Only some of them



Hash	SHA-1	MD5	AES	MAC	ECB
CBC	Symmetric Key	RSA	Public Key	Private Key	Encryption
	Decryption	Digital signature	DES	3DES	

RFC 4949 Internet Security Glossary, Version 2

- [illegible]



Cybersecurity Dictionary

Alice & Bob



Alice



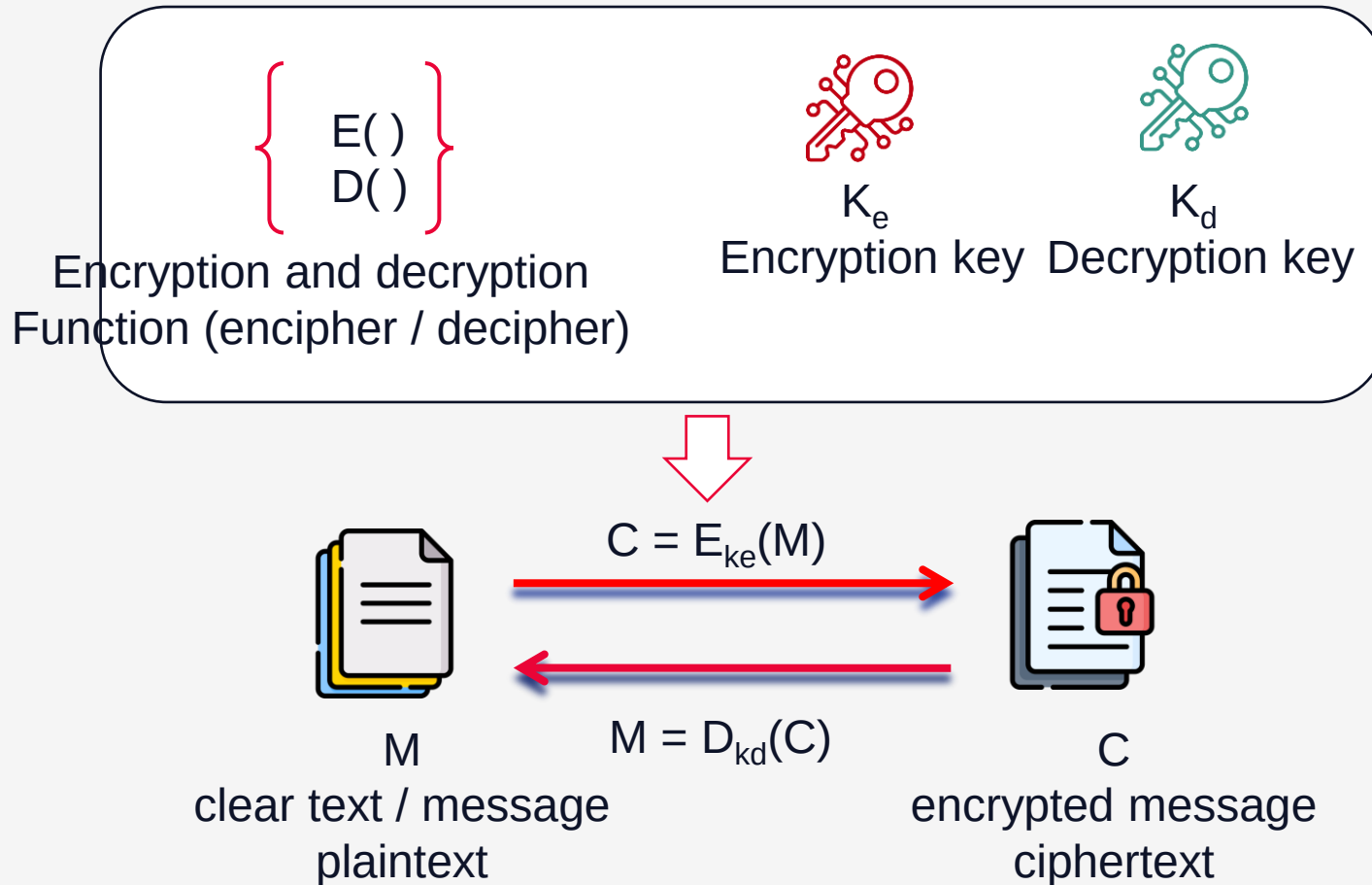
Bob

- The parties that are most often called upon to illustrate the operation of bipartite security protocols. These and other dramatis personae are listed by Schneier [[RFC 4949](#)]
- There is no relation with real persons. In all our examples Alice and Bob are fictional characters



Cryptographic System

Key ingredients



(M) plaintext – original message on clear

(C) ciphertext – encrypted message

cipher – algorithm for transforming plaintext to ciphertext

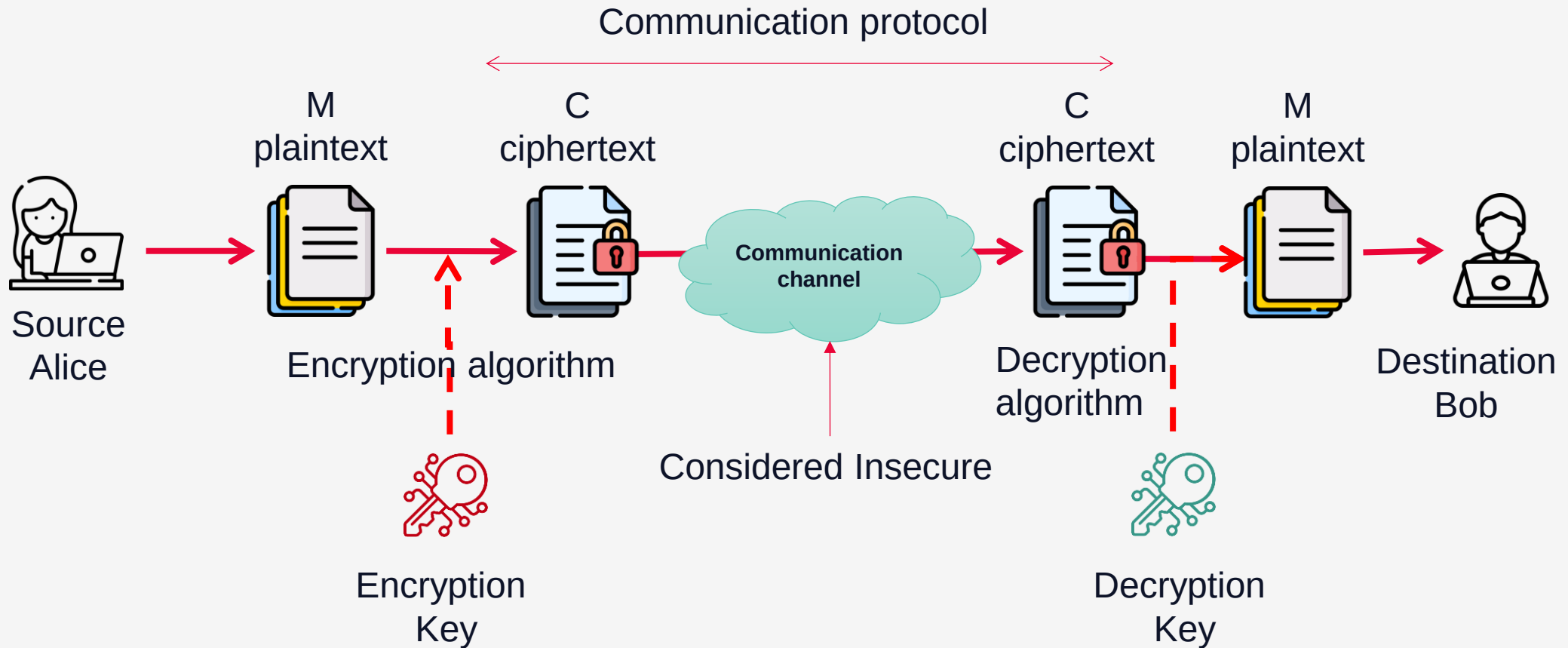
(K) key – information used to encrypt/decrypt

(E()) encipher (encrypt) – converting plaintext to ciphertext - encryption algorithm

(D()) decipher (decrypt) – converting ciphertext to plaintext - decryption algorithm

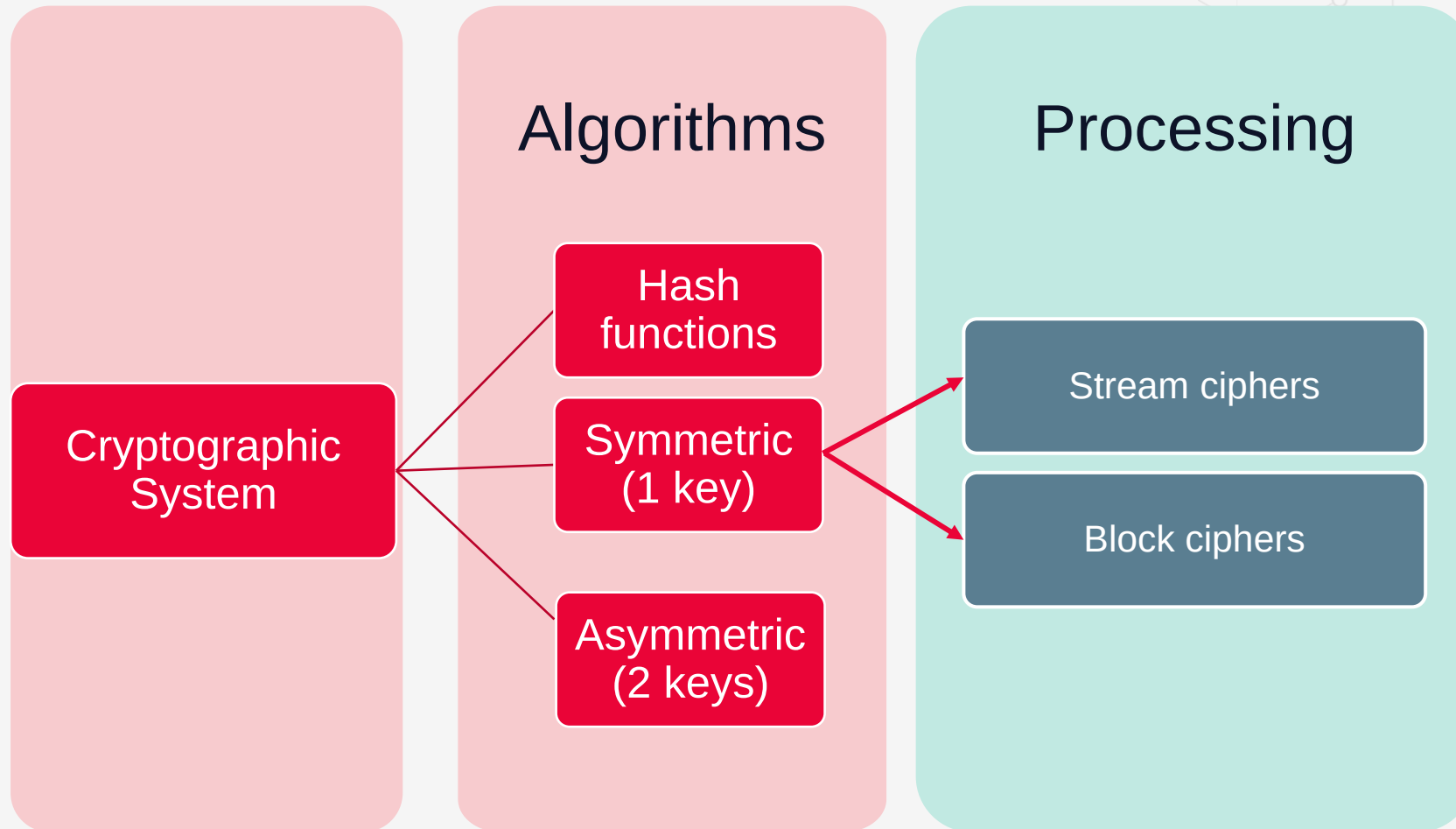
Cryptographic System

Key ingredients



Cryptographic System

Classification



Cryptographic System

Classification

OPERATION TYPE

- **substitution**
- **transposition**
- **complex/product**

NO OF KEYS

- **single-key/private** – symmetric systems
- **two-key/public** – asymmetric systems

PROCESSING UNIT

- **block cipher**: one that breaks a message up into chunks and combines a key with each chunk.
- **stream cipher**: one that applies a key to each bit, one at a time



Security

Types of Security

UNCONDITIONAL SECURITY

- the cipher cannot be broken no matter how much computer power or time is available (one-time-pad);
- unconditional security provides provable guarantees of security that hold under all conditions, including future advances in computing power and algorithms.
- achieving unconditional security often comes with practical limitations, such as key distribution challenges, the need for large key sizes, or restrictions on the types of operations that can be performed
- example: The one-time pad encryption scheme achieves perfect secrecy, where the encrypted message reveals no information about the plaintext, even with unlimited computational resources

COMPUTATIONAL SECURITY

- the cipher cannot be broken given limited computing resources (mostly time)
- relies on the assumption that certain computational problems are hard to solve within a feasible amount of time, typically due to the limitations of current computing power and algorithms
- includes a "security parameter" that defines the level of security based on the size of cryptographic keys or other parameters
- example: RSA encryption relies on the assumption that factoring large composite numbers into primes is computationally difficult



Vulnerabilities

Why it happens

- **Vulnerability**

- A flaw or weakness in a system's design, implementation, or operation and management that could be exploited to violate the system's security policy. [RFC 4949]

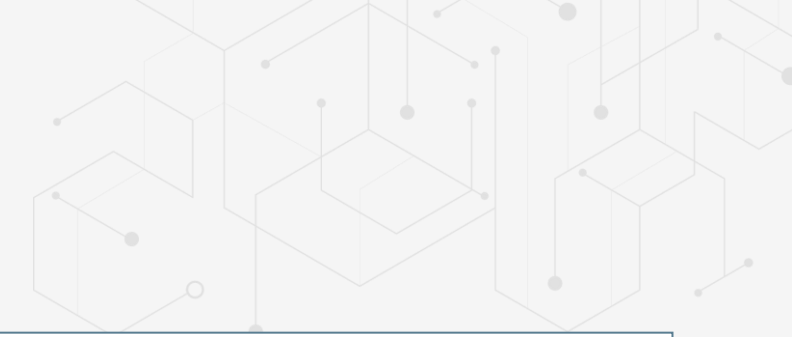
- **Most common vulnerabilities**

- Weak Passwords
- Lack of Updates and Patches
- Inadequate Access Controls
- Insufficient or No Encryption
- Poor Incident Response Plan
- Social Engineering Vulnerabilities
- Unsecured APIs and Endpoints
- Lack of Network Segmentation
- Ignoring Security Logs and Alerts....if they exists
- Misconfigured Cloud Services
- Inadequate Data Backup and Recovery
- Neglecting internal security



Vulnerabilities

Only a sample of the most common ones



WEAK PASSWORDS

- Using easily guessable passwords or default passwords.
- Increases the likelihood of brute-force attacks or unauthorized access.

LACK OF REGULAR UPDATES AND PATCHES

- Failing to regularly update software, operating systems, and applications
- Leaves systems vulnerable to known exploits and vulnerabilities

INADEQUATE ACCESS CONTROLS

- Granting excessive privileges to users and not following the principle of least privilege.
- Allows unauthorized access and potential data breaches.

INSUFFICIENT OR NO ENCRYPTION

- Not encrypting sensitive data at rest or in transit.
- Exposes data to interception and unauthorized access.

SOCIAL ENGINEERING

- Falling victim to phishing, spear-phishing, or other social engineering attacks.
- Compromises credentials and allows unauthorized access.

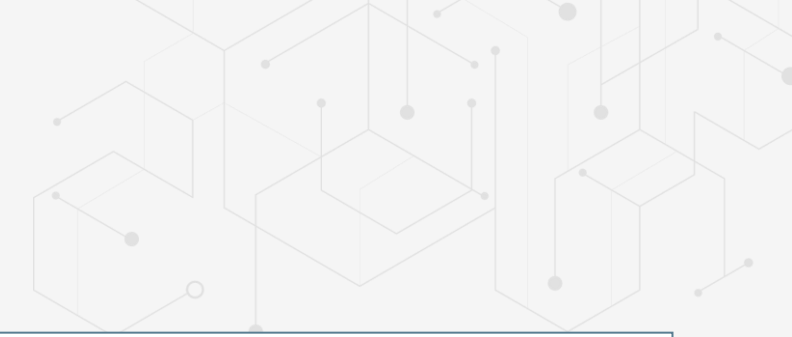
UNSECURED APIS AND ENDPOINTS

- Leaving APIs and endpoints exposed without proper security measures.
- Allows attackers to exploit vulnerabilities and gain access to systems.



Vulnerabilities

Only a sample of the most common ones



POOR INCIDENT RESPONSE PLAN

- Not having a well-defined and tested incident response plan
- Leads to delayed and ineffective responses to security incidents

LACK OF NETWORK SEGMENTATION

- Having a flat network structure without proper segmentation
- Allows attackers to move laterally within the network once they gain access

IGNORING SECURITY LOGS

- Not monitoring or analyzing security logs and alerts
- You miss early signs of breaches and attacks
- Not using a Security Information and Event Management (SIEM) system

MISCONFIGURED CLOUD SERVICES

- Incorrectly setting up cloud services, leading to exposed data and resources
- Makes sensitive data and services publicly accessible

NO DATA BACKUP AND RECOVERY

- Not having proper data backup and recovery mechanisms
- Leads to data loss and extended downtime in case of an attack

INTERNAL SECURITY

- Focusing too much on perimeter defenses and neglecting internal security
- Leaves internal systems and data vulnerable to insider threats and lateral attacks



Vulnerabilities

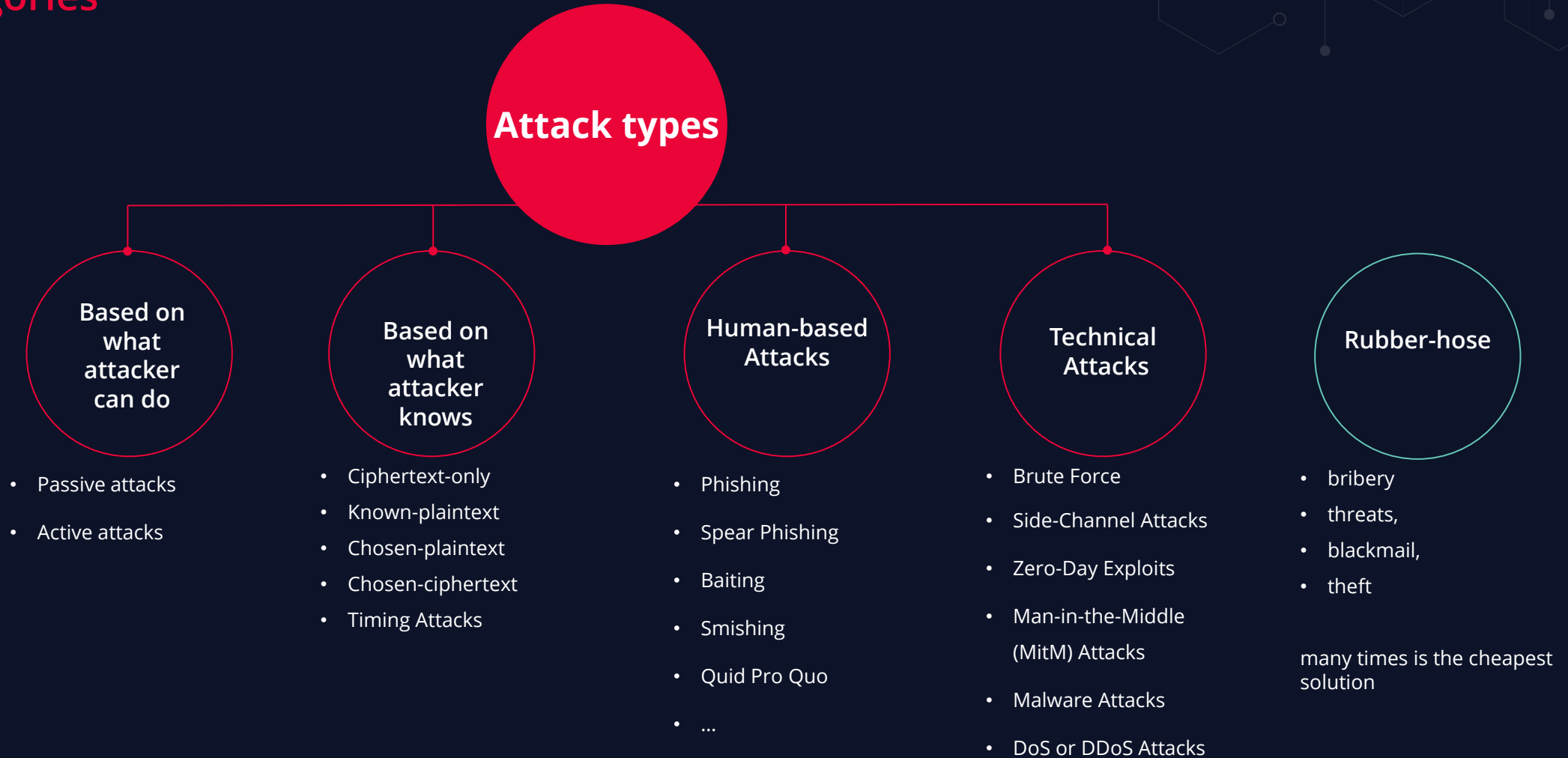
Mitigation Strategies

- **Weak Passwords:** Enforce strong password policies, including complexity, length, and regular changes. Use random generated ones
- **Lack of Regular Updates and Patches:** Implement automated patch management systems to ensure timely updates.
- **Insufficient Encryption:** Use strong encryption standards for data storage and transmission.
- **Social Engineering Vulnerabilities:** Conduct regular security awareness training for employees.
- **Unsecured APIs and Endpoints:** Implement API security best practices, including authentication, authorization, and input validation
- **Lack of Network Segmentation:** Segment networks into smaller, isolated sections to limit the spread of attacks
- **Inadequate Data Backup and Recovery:** Implement regular data backup procedures and test recovery processes



Attack Types

Categories



Attack Types

What the attacker can do

- **Passive attacks**

- attacker attempts to obtain information from the system or network without altering the data or system resources
- attacker aims to gather sensitive information covertly, without the knowledge of the system owner or legitimate users
- do not alter or disrupt system operations or data integrity
- focused on information gathering

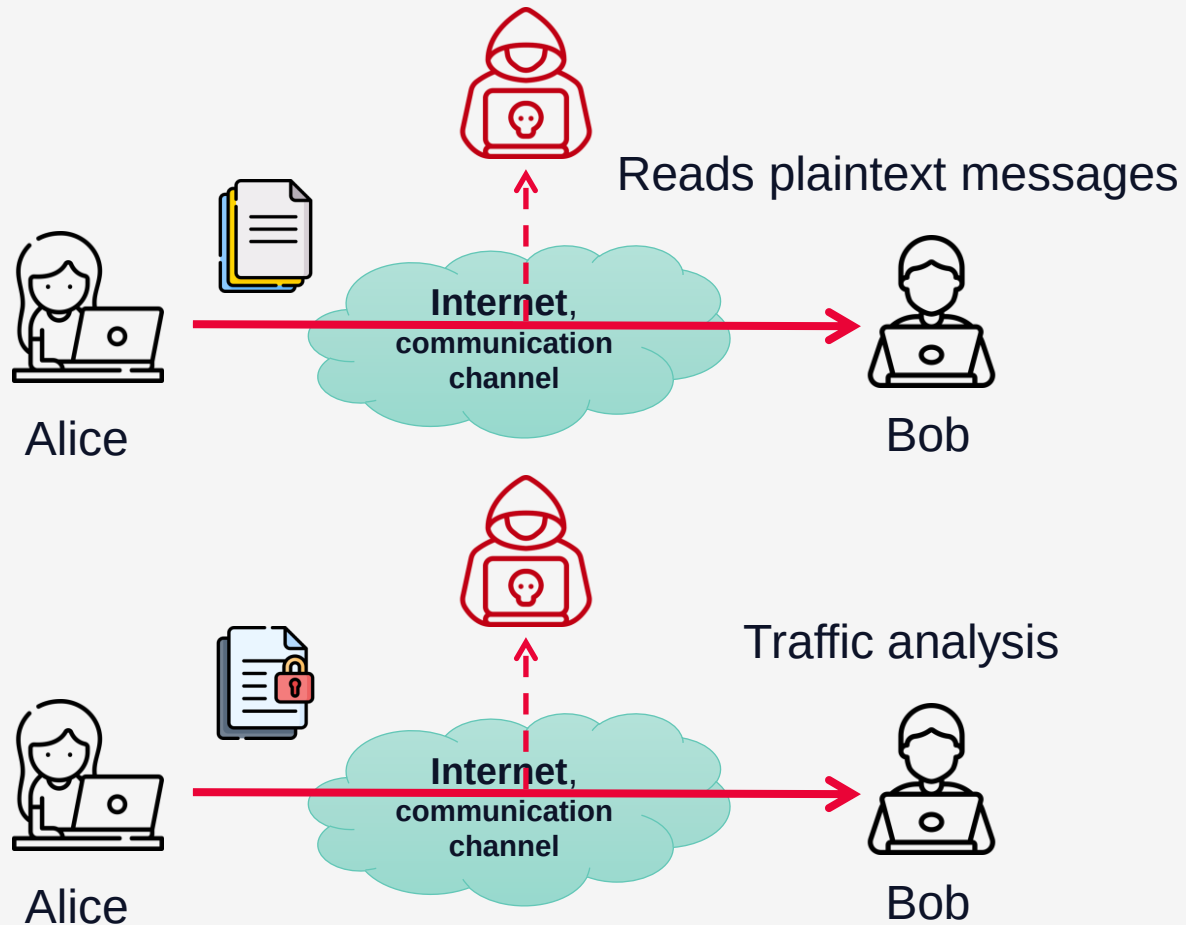
- **Active attacks**

- attacker attempts to alter or disrupt the operations of a system, network, or communication channel
- attacker seeks to modify, delete, or inject data, causing direct harm or unauthorized changes
- cause immediate damage or unauthorized changes, impacting system integrity and availability



Passive Attacks

Eavesdropping



Eavesdropping

- Attacker monitors or intercepts communications between parties.
- Attacker gathers information like passwords, credit card numbers, or other confidential data over unencrypted communication
- Attacker gathers ciphertext over encrypted communication to break it later by cryptanalysis or brute force
- Attacker analyzes patterns and characteristics of data flows or communications without accessing the content directly

Passive Attacks

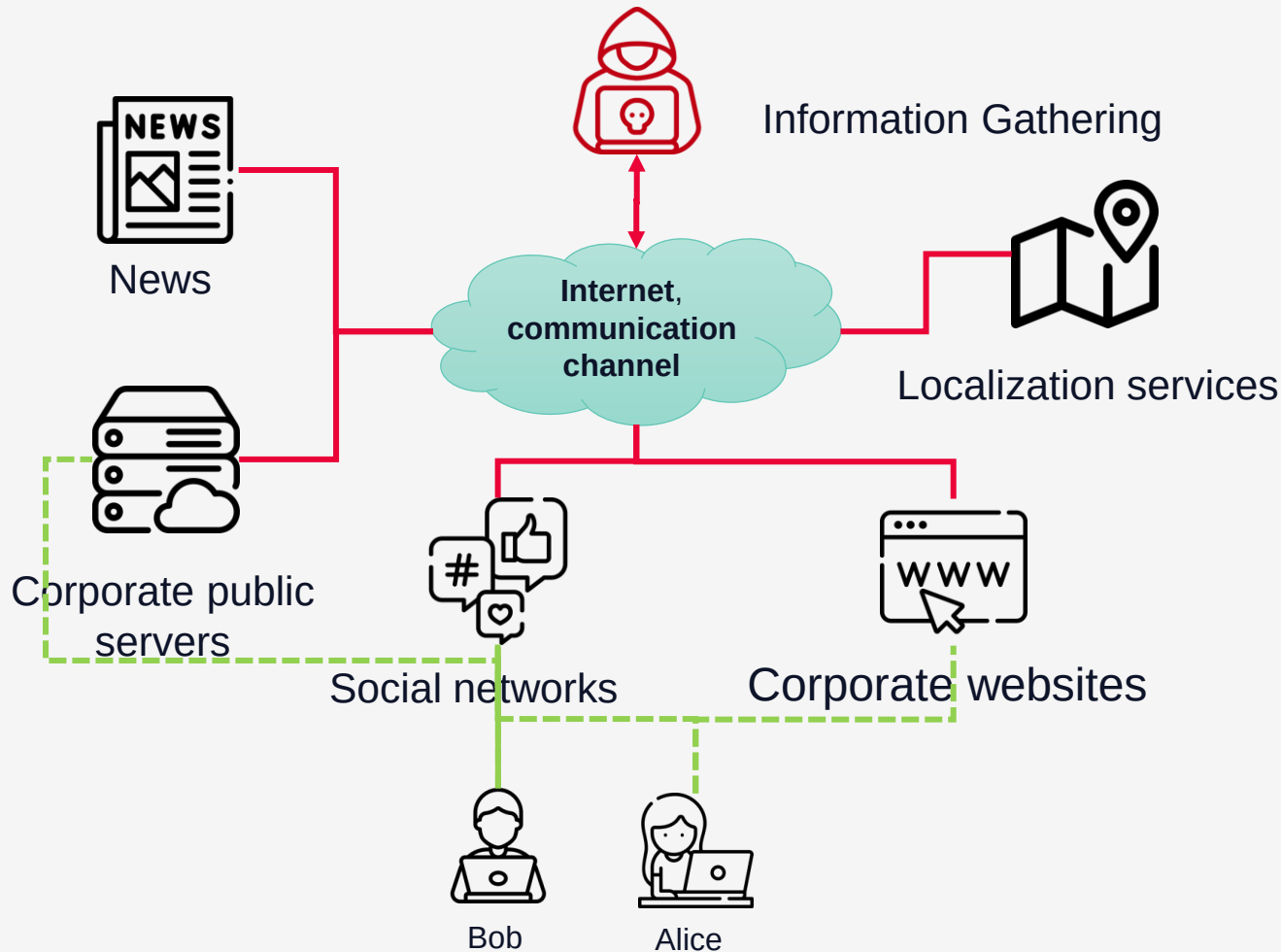
Eavesdropping

- Wi-Fi and Bluetooth networks are by default vulnerable to eavesdropping because any attacker can get transmitted packets if is in the network range
- For cable networks the attacker needs to have access to that network
- In Wi-Fi and Bluetooth networks is impossible to know if someone is eavesdropping
- In cable networks the attacker is (looks like) a valid network device/user
- Attacker has full access to plaintext messages transmitted over the communication channel



Passive Attacks

Reconnaissance / Information Gathering

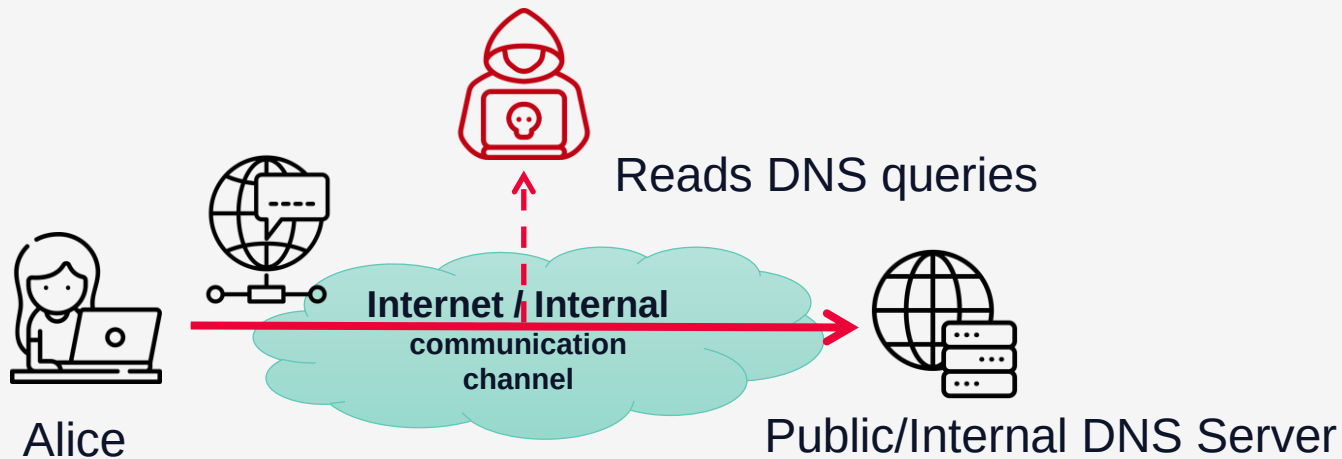


Reconnaissance

- **Attacker collects publicly available information about a target system or network**
- **Attacker uses the data to find potential vulnerabilities, system configurations, or personnel information that could aid in further attacks**
- **Example: Scanning a website to gather email addresses of employees for targeted phishing attacks.**
- **Example: Port scanning company domains and IPs to identify online servers and services.**

Passive Attacks

Passive DNS (Domain Name System) Monitoring



Eavesdropping

- Attacker observes DNS queries and responses to gather information about network infrastructure and communication patterns.
- Attacker map out organizational networks
- Attacker identify points of interest and future attack vectors
- Attacker monitors DNS queries to detect subdomains used by a company for internal services

Passive Attacks

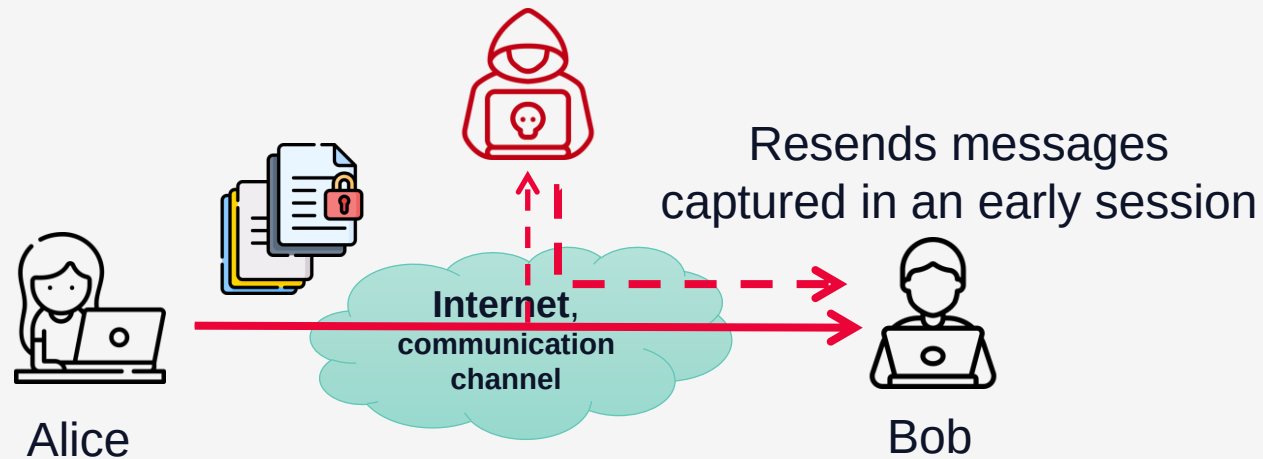
Mitigation Strategies

- **Encryption:** Use strong encryption protocols (like TLS/SSL) to protect sensitive data in transit.
- **Network Segmentation:** Implement network segmentation and access controls to limit exposure of sensitive information.
- **Traffic Monitoring:** Employ intrusion detection systems (IDS) and network monitoring tools to detect suspicious activity.
- **Security User Awareness:** Educate users about the risks of sharing sensitive information online or over unsecured channels.



Active Attacks

Replay Attack

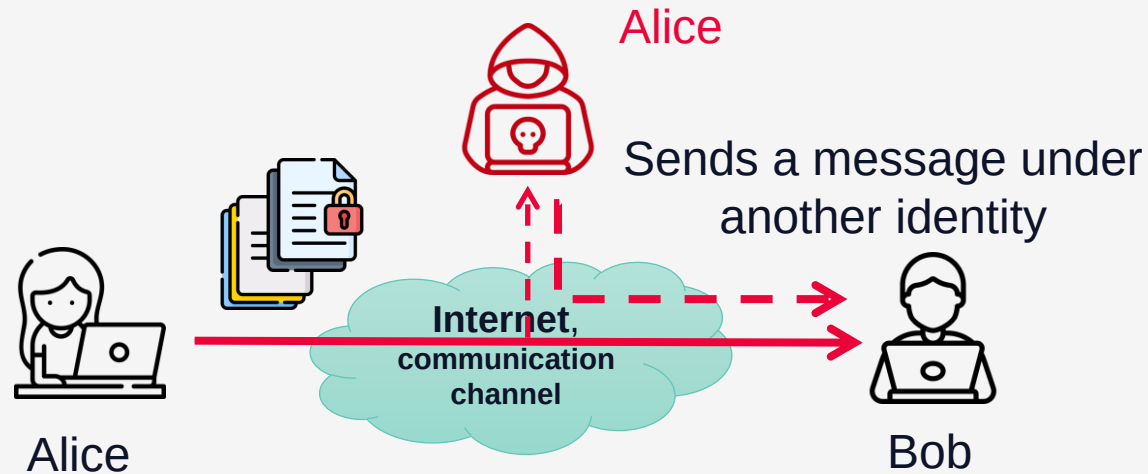


Replay

- Attacker intercepts and retransmits valid messages.
- Attacker tries to deceive the recipient into accepting a duplicate or delayed message
- Attacker tries to reuse valid authentication tokens to gain unauthorized access
- All replayed messages are valid, so it could force the destination to process them
- Attacker can't generate new valid messages but it can replay existing ones

Active Attacks

Impersonation/Masquerade Attack

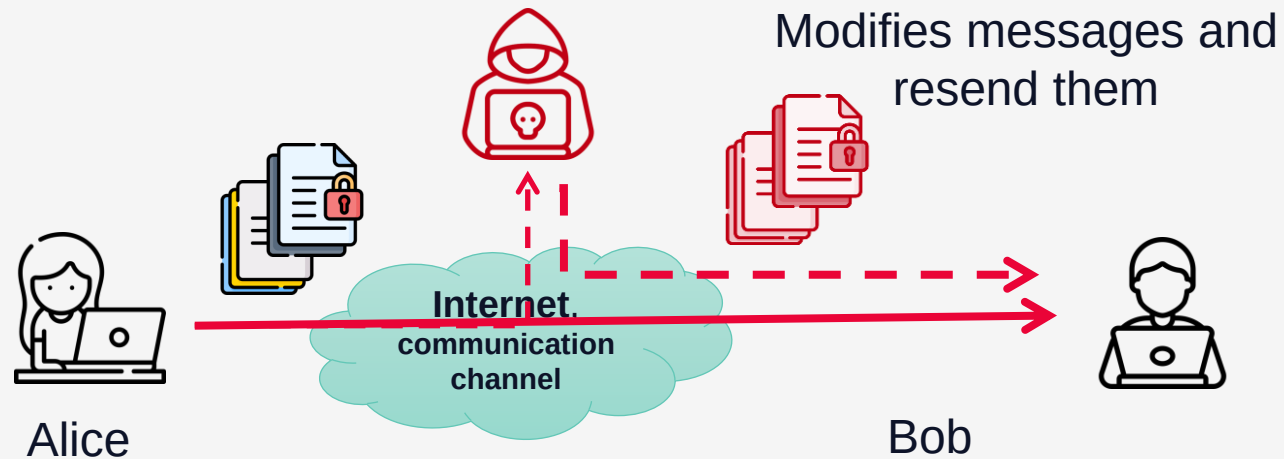


Impersonation

- Attacker intercepts authentication tokens, session cookies and messages with authentication headers.
- Attacker pretends to be an authorized user or system
- Attacker tries to gain unauthorized access to resources or data
- Attacker tries to reuse valid authentication tokens to log into a system as an authorized user
- For web applications is known as Session Hijacking

Active Attacks

Tampering Attack



Tampering

- Attacker tampers / alters data in transit or stored information.
- Attacker tries to change the content of the data, causing unauthorized effects
- Attacker can't break the encryption or the authentication mechanism, so it tries to hack it by changing existing messages

Cybersecurity Dictionary

Tamper

- **tamper**

- Make an unauthorized modification in a system that alters the system's functioning in a way that degrades the security services that the system was intended to provide.

- **tamper-evident**

- A characteristic of a system component that provides evidence that an attack has been attempted on that component or system.

- **tamper-resistant**

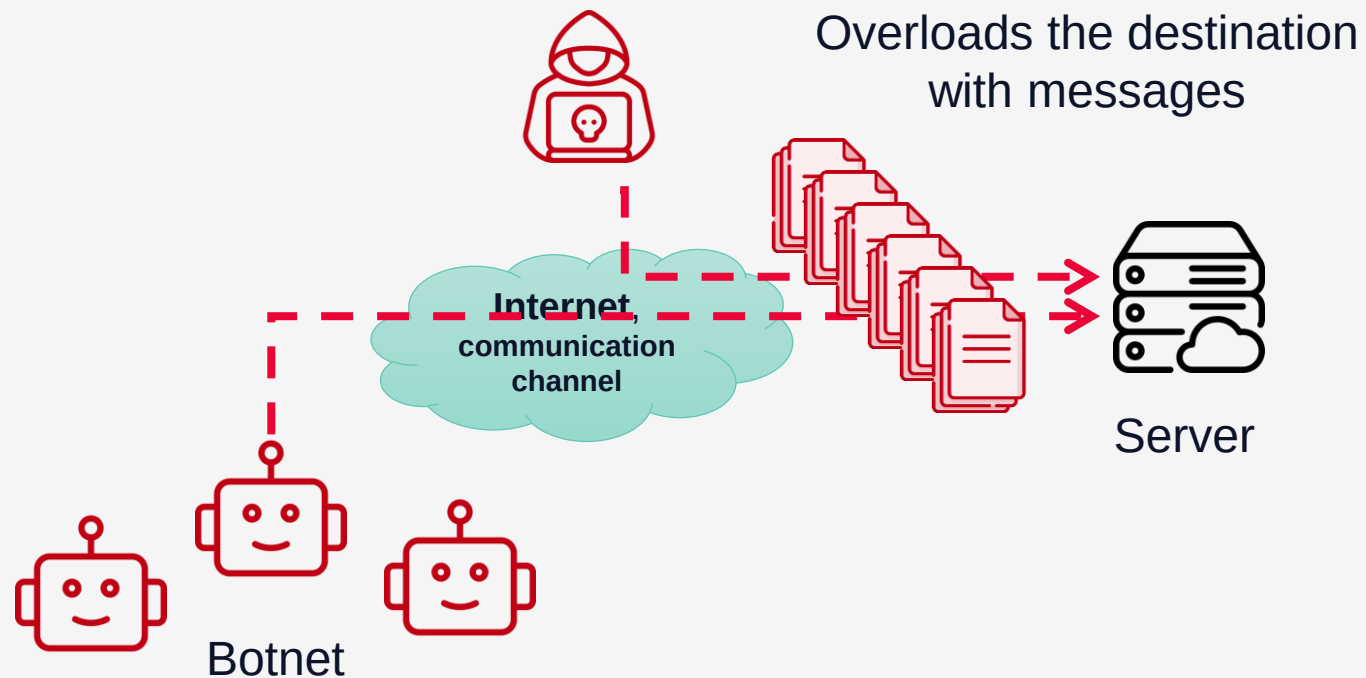
- A characteristic of a system component that provides passive protection against an attack.

[[RFC 4949](#)]



Active Attacks

(Distributed) Denial-of-Service (DDoS / DoS) Attack

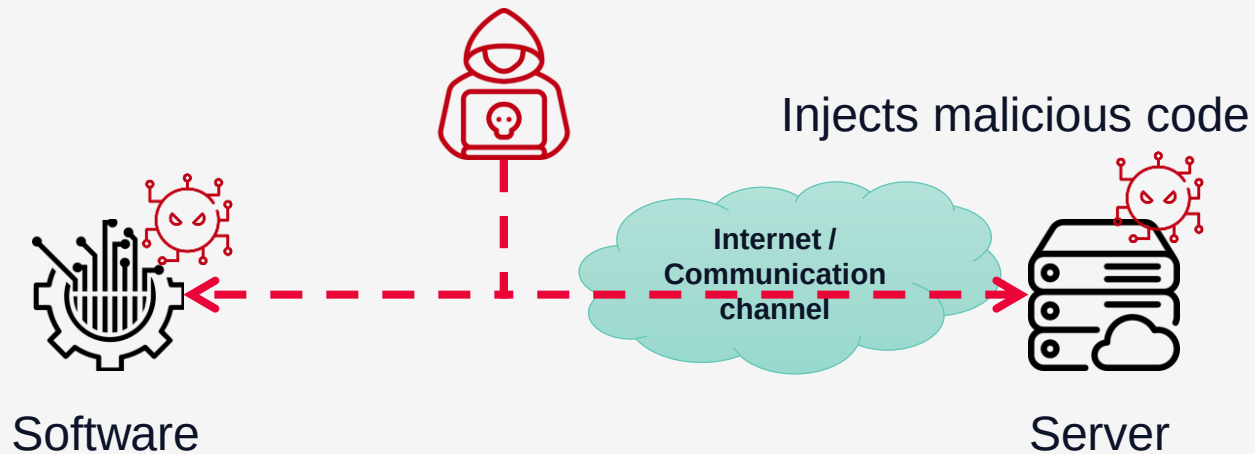


Denial-of-Service

- Attacker overloads a system or network to make it unavailable to legitimate users.
- Attacker tries to disrupt services and cause operational downtime
- Attacker can use valid messages which were recorded earlier
- DDoS is like a DoS attack but launched from multiple devices simultaneously
- DDoS attacks amplify the impact and make it harder to mitigate because it has many sources
- DDoS attacks involves botnets and generate massive amounts of traffic

Active Attacks

Injection Attack

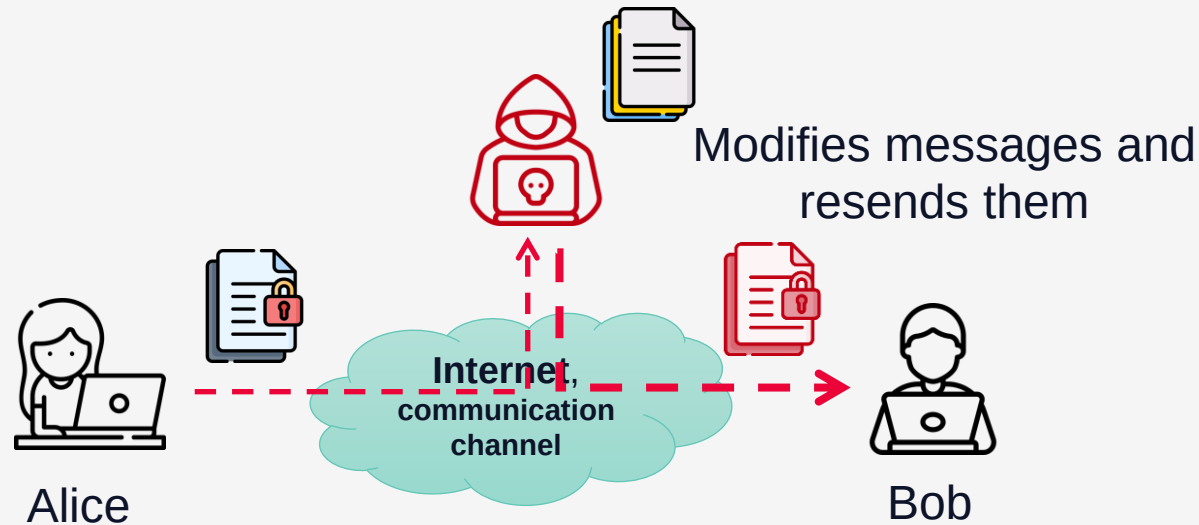


Injection

- Attacker tries to insert malicious code into a system or application
- Attacker tries to execute unauthorized commands or exploit vulnerabilities
- Attacker exploits vulnerabilities to perform SQL injection (mostly for Web) or buffer overflow attacks

Active Attacks

Man-in-the-Middle (MitM) Attack



Man-in-the-Middle

- **Attacker intercepts and potentially alters the communication between two parties.**
- **Attacker can eavesdrop, modify, or inject false information**
- **Attacker has full control over the communication between Alice and Bob**
- **At the attacker side, all messages are plaintext**
- **Alice and Bob can be unaware of the MitM attack**
- **can happen even over HTTPS/TLS channels if the destination certificate is not validated**

Active Attacks

Mitigation Strategies

- **Authentication and Authorization:** Use strong authentication methods and enforce strict access controls.
- **Encryption:** Encrypt sensitive data both in transit and at rest to prevent unauthorized modification.
- **Prevent replay attacks:** use nonces, timestamps or rolling codes
- **Intrusion Detection and Prevention Systems (IDPS):** Deploy IDPS to monitor, detect, and respond to suspicious activities.
- **Network Segmentation:** Segment networks to limit the spread of attacks and reduce attack surfaces.
- **Regular Updates and Patching:** Keep systems and applications updated to protect against known vulnerabilities.
- **Security Awareness Training:** Educate users about recognizing and preventing potential attacks, such as phishing and social engineering.



Cybersecurity Dictionary

- **nonce**

- A random or non-repeating value that is included in data exchanged by a protocol, usually for the purpose of guaranteeing liveness and thus detecting and protecting against replay attacks.

- **salt**

- A data value used to vary the results of a computation in a security mechanism, so that an exposed computational result from one instance of applying the mechanism cannot be reused by an attacker in another instance.

[[RFC 4949](#)]



Attack Types

What the attacker knows

- **You can't hide information over the network or in software**
 - this classification evaluates the level of information the attacker has
 - attacker will always get some information by just observing network traffic, read documentation, reverse engineering software or hardware, get inside documents, etc.
- **Types of attacks**
 - Ciphertext-only
 - Known-plaintext
 - Chosen-plaintext
 - Chosen-ciphertext
 - Side-Channel Attacks



Attack Types

What the attacker knows



CIPHERTEXT-ONLY

- Encryption algorithm
- Encrypted messages



KNOWN-PLAINTEXT

- Encryption algorithm
- Encrypted messages
- Plaintext and its corresponding ciphertext texts



CHOSEN-PLAINTEXT

- Encryption algorithm
- Encrypted messages
- Plaintext and its corresponding ciphertext texts
- Can choose the plaintext to be encrypted



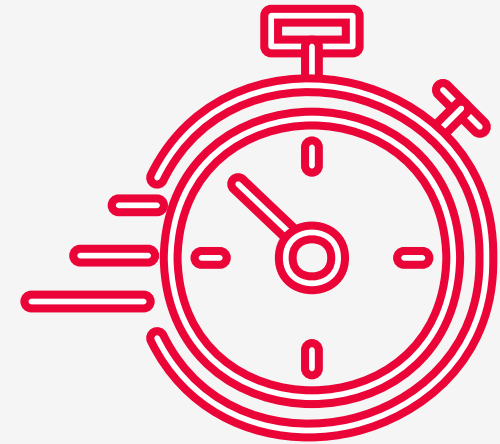
CHOSEN-CIPHERTEXT

- Encryption algorithm
- Can choose the ciphertext to be decrypted

Attack Types

Side-Channel Attacks

- Attacker exploits physical characteristics of a system (like electromagnetic emissions or power consumption) to gain information
- **Timing Attacks:**
 - Measures the time taken to execute cryptographic algorithms
 - Some keys require more time
- **Power Analysis:**
 - Analyzes power consumption patterns to extract cryptographic keys
 - Some keys require more computing power



Social Engineering Attack

Attacking the weakest link

- Is the psychological manipulation of people into performing actions or divulging confidential information. [Wikipedia]
- Can take many forms:
 - Phishing
 - Spear Phishing
 - Whaling
 - Baiting
 - Business Email Compromise (BEC)
 - Smishing
 - Quid Pro Quo
 - Pretexting
 - Honeytrap
 - Tailgating/Piggybacking



Social Engineering Attacks

Some samples

PHISHING

- uses email, phone, SMS, social media or other form of personal communication to trick users to click a malicious link, download infected files or reveal personal information
- spear phishing when the attack is targeted at specific individuals or organizations

WHALING

- phishing attack designed for a specific person (important one), typically a high-level executive.
- requires a significant amount of research on that individual

BAITING

- uses tempting ads or online promotions, such as free game, music or phone apps to trick users to reveal personal information or to install malware on their systems
- leaving a malware-infected physical device, such as a USB drive, in a place where someone will find it and use it

BUSINESS EMAIL COMPROMISE

- the attacker impersonates an executive who is authorized to deal with financial matters within the organization and convince subordinates to make wire transfers

SMISHING

- SMS-phishing, or smishing, is conducted specifically through SMS messages

QUID PRO QUO

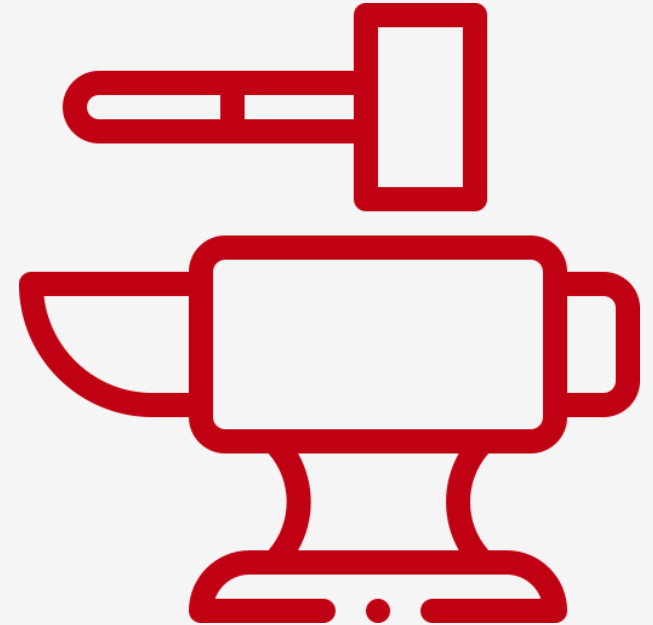
- involves the attacker requesting sensitive information from the victim in exchange for a desirable service
- the attacker may pose as an IT support technician



Brute Force Attack

Time is the issue

- **Brute force attacks involve trying all possible combinations of passwords or encryption keys until the correct one is found.**
 - These attacks rely on computational power and are often automated using software tools
 - Depending on passwords and keys sizes and available computational power it can take from several hours to thousands of years
 - Exploits weak passwords
- **Types of attacks**
 - Password Cracking
 - Dictionary Attack
 - Hybrid Attack
 - Credential Stuffing



Brute Force Attack

Types

PASSWORD CRACKING

- Systematically trying every possible password combination to gain access to an account

DICTIONARY ATTACK

- Using a list of common passwords or words to attempt to gain access

HYBRID ATTACK

- Combining dictionary attacks with brute force attacks by adding variations to dictionary words (e.g., adding numbers or symbols)

CREDENTIAL STUFFING

- Using lists of previously breached usernames and passwords to try to gain access to accounts on different systems



Brute Force Attack

Mitigation Strategies

- Implement Account Lockout Mechanisms
- Use Strong Password Policies
- Enable Multi-Factor Authentication (MFA)
- Rate-Limiting
- Use Captchas
- Monitor and Analyze Login Attempts
- Use Password Hashing with Salt
- Password Blacklisting
- IP Blacklisting or Geo-Blocking
- Progressive Delays



Zero Stuff

A lot of zero's

- **Zero-Day vulnerability.**

- is a software flaw or security hole that is unknown to the software vendor or the general public and has not yet been patched or addressed
- The attacker exploits it to gain unauthorized access to systems or steal data before the vendor has a chance to develop and release a fix

- **Zero-Knowledge protocol:**

- is a cryptographic method by which one party (the prover) can prove to another party (the verifier) that a given statement is true without revealing secrets - zero-knowledge proof
- a processing method that involves processing data without revealing it to other parties

- **Zero-Trust security framework:**

- assumes no entity, whether inside or outside the network, is automatically trusted; every access request must be verified before being granted, regardless of the source or destination



Cybersecurity Objectives

To secure messages and transactions in software distributed systems.

Cryptographic systems characteristics:

- Total or partial confidentiality
- Authentication
- Data integrity
- Nonrepudiation

Security Services:

- X.800, <http://www.itu.int/rec/T-REC-X.800-199103-1>
- RFC 4949, <https://tools.ietf.org/html/rfc4949>
- CIA triad



Cybersecurity Objectives

Some standards

X.800

- 1. Authentication:** Peer entity authentication and Data origin authentication
- 2. Access Control**
- 3. Data Confidentiality:** Connection, Connectionless and Selective field confidentiality
- 4. Data Integrity**
 - with Recovery
 - without Recovery
- 5. Nonrepudiation**
 - Origin
 - Destination

RFC 4949

- 1. Access control** service
- 2. Audit** service
- 3. Authentication** service
 - data origin authentication
 - peer entity authentication
- 4. Availability** service
- 5. Data confidentiality** service
- 6. System integrity** service
- 7. Non-repudiation** service

CIA TRIAD

- 1. Confidentiality**
- 2. Integrity**
- 3. Availability**



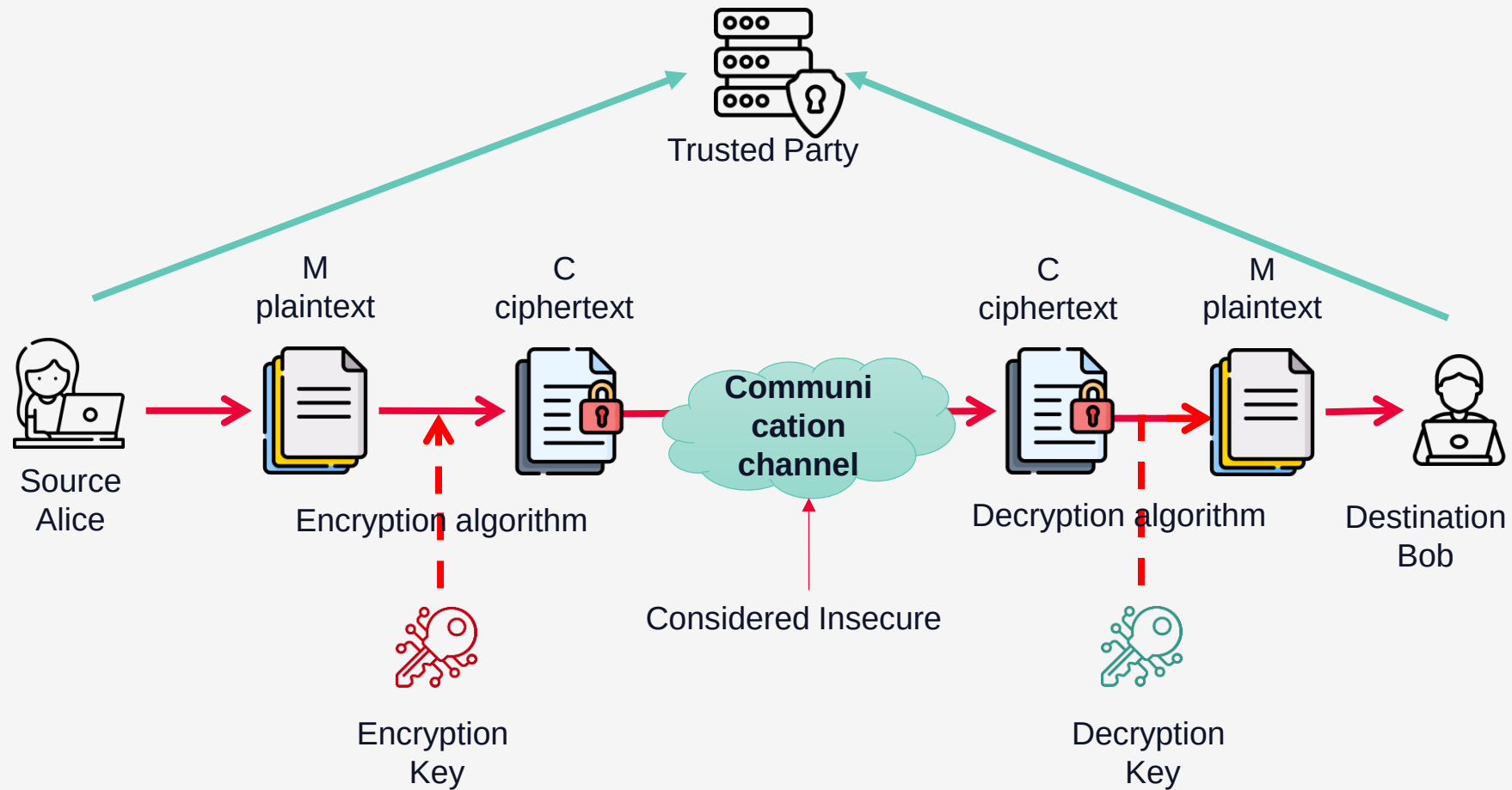
Cybersecurity Objectives

The most important ones



Objectives	Properties
Confidentiality	Hides the message content Implemented by symmetric algorithms that generate ciphertexts Does NOT assure the integrity and accuracy of the content
Integrity	Guarantees the integrity and accuracy of the content Implemented by <i>one-way hash functions</i> that generate message digest values
Authentication	Ensures the communication parties identities It presumes that the communication channel is not safe Implemented by <i>Message Authentication Codes (MAC)</i> that generate message tag values or by public key digital signatures.
Non-repudiation <i>Proof of Origin</i>	Guarantees the message source, the connection between the source and its sent message. Prevents situations in which the source denies it has sent the message Implemented by <i>public key digital</i> signatures that generates signature values

Security Model



Dolev-Yao Security Model

WHAT THE ATTACKER CAN DO

- Get any transmitted message throughout communication channel
- It is a network user (with rights)
- Opens communication channels with other users
- He can become the destination of a message
- He sends messages in the name of another user
- Has full control over the network

WHAT THE ATTACKER CAN'T DO

- He CAN'T guess a random number from a large enough set
- Without the secret key, he CAN'T get the plaintext and he CAN'T get a valid cipher (depends on the encryption algorithm)
- He CAN'T generate the private key related to a public key
- He DOES'T have physical access to the user machine



Kerckhoff's Principles

INITIAL PRINCIPLES

- publishes in 1883 two reports in 'le Journal des Sciences Militaires' about Military Cryptography and highlighted the desired characteristics of cryptography standards
 1. **The system must be practically, if not mathematically, indecipherable.**
 2. **It should not require secrecy, and it should not be a problem if it falls into enemy hands.**
 3. **It must be possible to communicate and remember the key without using written notes, and correspondents must be able to change or modify it at will.**
 4. It must be applicable to telegraph communications.
 5. ...

TODAY TRANSLATION

- The strength of the cryptosystem is given by the secrecy of the key
- The algorithm is made public
- After the algorithm has been analyzed and is proven it has no flaws, weak keys, backdoors, it is considered strong



Zero Trust

Core Principles

1. Verify Explicitly:

- Always authenticate and authorize based on all available data points, including user identity, device status, location, and other behavioral attributes.

2. Least Privilege Access:

- Limit user access rights to the minimum necessary to do their jobs, reducing the potential damage from compromised credentials or insider threats.

3. Assume Breach:

- Operate with the assumption that the network is already compromised. Focus on minimizing damage, protecting sensitive data, and maintaining operations even if a breach occurs.



Security risks

Not knowing the vulnerabilities of cryptographic algorithms

Not knowing how to correct implement them



2. Threat modelling and Risk assessment



Threat modeling

Anything is possible

- is a structured approach used to identify, quantify, and address the security risks associated with an application, system, or process
- It involves understanding the potential threats that could exploit vulnerabilities, determining their impact, and devising mitigations to protect the system



Threat modeling

Key Steps

1. **Identify Assets:** Determine what needs protection, such as data, applications, networks, and hardware.
2. **Create an Architecture Overview:** Develop a detailed representation of the system's architecture, including data flow diagrams, network diagrams, and component interactions.
3. **Identify Threats:** Use frameworks like STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) to categorize and identify potential threats
4. **Identify Vulnerabilities:** Assess the system for weaknesses that could be exploited by the identified threats. This includes reviewing design documents, performing code reviews, and conducting vulnerability scans.
5. **Assess Risk:** Evaluate the likelihood and potential impact of each threat exploiting a vulnerability.
6. **Develop Mitigations:** Define strategies and controls to mitigate identified risks. This can include technical measures (like encryption and firewalls), procedural changes (such as improved access controls), and policies (like regular security training)
7. **Validate and Review:** Continuously test and review the implemented mitigations to ensure they are effective. This includes regular security testing, audits, and updates to the threat model as the system evolves.



Threat modeling

Frameworks and Methodologies

- **STRIDE:** Focuses on six categories of threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege
- **DREAD:** A risk assessment model evaluating threats based on Damage, Reproducibility, Exploitability, Affected Users, and Discoverability
- **PASTA (Process for Attack Simulation and Threat Analysis):** A seven-step risk-centric methodology focusing on aligning business objectives with technical requirements to assess threats
- **OCTAVE (Operationally Critical Threat, Asset, and Vulnerability Evaluation):** A framework that focuses on organizational risk management, emphasizing the context in which information assets are used
- **Attack Trees:** A hierarchical model used to map out potential attack vectors and the paths an attacker might take to achieve a particular goal

Threat modeling

An example

In an e-commerce platform, threat modeling might involve:

- Identifying assets such as user data, payment information, and inventory systems.
- Creating an architecture diagram detailing web servers, databases, and payment gateways.
- Using STRIDE to identify threats like spoofing user identities, tampering with transaction data, and denial of service attacks.
- Assessing vulnerabilities in user authentication mechanisms, data transmission channels, and server configurations.
- Developing mitigations like implementing multi-factor authentication, encrypting data in transit, and deploying anti-DDoS solutions.
- Continuously validating security measures through penetration testing, code reviews, and security audits.



Risk assessment

Anything is possible

- A systematic process used to identify, evaluate, and prioritize risks to an organization's information systems and data.
- The goal is to understand the potential impact of different threats, the likelihood of their occurrence, and to determine appropriate measures to mitigate these risks



Risk assessment

Key Components

1. **Identify Assets:** Determine what valuable assets need protection, such as data, hardware, software, intellectual property, and network infrastructure.
2. **Identify Threats:** Identify potential threats that could exploit vulnerabilities. These can include external threats (e.g., hackers, malware) and internal threats (e.g., insider threats, accidental data leaks).
3. **Identify Vulnerabilities:** Assess the system for weaknesses that could be exploited by threats. This includes technical vulnerabilities (e.g., unpatched software), procedural vulnerabilities (e.g., lack of security policies), and physical vulnerabilities (e.g., unsecured access points).
4. **Determine Likelihood:** Estimate the likelihood of different threats exploiting identified vulnerabilities. This involves considering factors like threat actor capabilities, historical data, and current security posture.
5. **Evaluate Impact:** Assess the potential impact of successful attacks on the organization. This can include financial losses, reputational damage, legal implications, and operational disruptions.
6. **Calculate Risk:** Combine the likelihood and impact assessments to determine the overall risk level. This can be done using qualitative methods (e.g., high, medium, low) or quantitative methods (e.g., monetary value of potential losses).

Risk assessment

Key Components

7. **Prioritize Risks:** Rank the identified risks based on their severity and importance. This helps in focusing resources on mitigating the most critical risks first.
8. **Develop Mitigation Strategies:** Define actions to mitigate identified risks. This can include implementing security controls, enhancing policies, conducting training, and applying technical measures such as encryption and access controls.
9. **Implement Mitigation Measures:** Put the mitigation strategies into action to reduce the identified risks to an acceptable level.
10. **Monitor and Review:** Continuously monitor the effectiveness of implemented measures and review the risk assessment regularly to ensure it remains relevant and up-to-date with evolving threats and changes in the organization.

Risk assessment

Methodologies

1. Qualitative Risk Assessment:

- Uses descriptive terms to evaluate risk (e.g., high, medium, low).
- It is often based on expert judgment and is simpler but less precise.

2. Quantitative Risk Assessment:

- Uses numerical values and statistical methods to evaluate risk.
- It provides a more detailed and measurable analysis but can be more complex and data-intensive.

3. Hybrid Risk Assessment:

- Combines elements of both qualitative and quantitative approaches to leverage the strengths of each.



Risk assessment

Frameworks

- **NIST SP 800-30:** A guide for conducting risk assessments, providing a detailed methodology for identifying, estimating, and evaluating risk.
- **ISO/IEC 27005:** An international standard providing guidelines for information security risk management within an organization.
- **OCTAVE (Operationally Critical Threat, Asset, and Vulnerability Evaluation):** A risk assessment methodology focused on organizational risk and strategic security planning.
- **Threat Analysis and Risk Assessment (TARA) process from ISO/SAE 21434**



Risk assessment

ISO/SAE 21434 TARA

The ISO/SAE 21434 key risk assessment component is the **Threat Analysis and Risk Assessment (TARA)** process

1. Asset Identification

- Identifying what needs to be protected (e.g., ECUs, sensors, data).

2. Threat Identification

- Identifying potential threats that could exploit vulnerabilities in the system.

3. Risk Assessment

- Evaluating the likelihood and potential impact of identified threats.
- Prioritizing risks based on their severity and likelihood.

4. Mitigation Strategies

- Developing and implementing measures to reduce or eliminate identified risks.

Common Vulnerabilities and Exposures (CVE)

- a publicly disclosed catalog of information security vulnerabilities
- a standardized list of known vulnerabilities and exposures in software and hardware
- maintained by the nonprofit organization MITRE Corporation
- is more like a dictionary assigning a unique ID to each known vulnerabilities, a description, a CVSS score (highest will be 10) and an affected product/platform
- at this moment (July 2024) contains more than 240.000 known vulnerabilities
- A zero-day vulnerability is not present in the CVE database. If it is, is not a zero-day vulnerability anymore (it was, but it not anymore)
- [CVE - CVE \(mitre.org\)](#)
- [CVE Website](#)
- [OpenCVE](#)

CVE vulnerability



CVSS SCORE

CVE

CVE-2023-29389 example on OpenCVE



Sign inRegister

CVE-2023-29389

OpenCVE > Vulnerabilities (CVE) > CVE-2023-29389

Toyota RAV4 2021 vehicles automatically trust messages from other ECUs on a CAN bus, which allows physically proximate attackers to drive a vehicle by accessing the control CAN bus after pulling the bumper away and reaching the headlight connector, and then sending forged "Key is validated" messages via CAN Injection, as exploited in the wild in (for example) July 2022.

CVSS v36.8 MEDIUM

6.8/10

CVSS v3 : MEDIUM

V3 Legend

Attack VectorPHYSICAL

Attack ComplexityLOW

Privileges RequiredNONE

User InteractionNONE

Confidentiality ImpactHIGH

Integrity ImpactHIGH

Availability ImpactHIGH

ScopeUNCHANGED

Vector :

Exploitability : 0.9 / **Impact :** 5.9

References

Link	Resource
https://kentindell.github.io/2023/04/03/can-injection/	ExploitThird Party Advisory
https://news.ycombinator.com/item?id=35452963	Issue Tracking

Information

Published : 2023-04-05 16:15

Updated : 2023-12-10 15:01

[NVD link : CVE-2023-29389](#)

[Mitre link : CVE-2023-29389](#)

[CVE.ORG link : CVE-2023-29389](#)

<> **JSON object :** [View](#)

Products Affected

toyota

- rav4
- rav4_firmware

CWE

CWE-74

Improper Neutralization of Special Elements in



CVE

Services



IDENTIFICATION

- Each vulnerability has a unique CVE ID, making it easier to reference and track
- Each vulnerability has a standard description



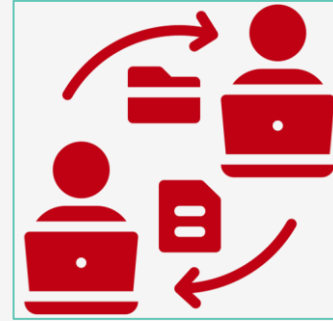
AWARENESS

- The CVE list is publicly available
- Organizations and individuals stay informed, in real time, about recent found vulnerabilities



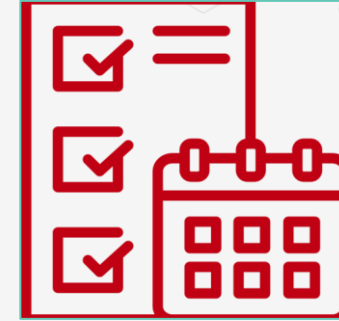
INFORMATION SHARING

- Information about known vulnerabilities is publicly disclosed by organizations or individuals



INTEROPERABILITY

- CVE IDs facilitate communication between different security tools and databases, improving interoperability and security coverage



MANAGEMENT

- Organizations can use CVE to prioritize and manage vulnerabilities based on their severity and potential impact



COMPLIANCE AND REPORTING

- Various regulatory frameworks and industry standards require organizations to manage and report vulnerabilities using the CVE list

Common Vulnerability Scoring System (CVSS)

Measuring vulnerability severity

- free and open standard that provides a qualitative measure of vulnerability severity
- CVSS is not a measure of risk
- [NVD - Vulnerability Metrics \(nist.gov\)](https://nvd.nist.gov/vuln-metrics/cvss)

Qualitative Severity Ratings

CVSS v2.0 Ratings		CVSS v3.x Ratings		CVSS v4.0 Ratings	
Severity	Severity Score Range	Severity	Severity Score Range	Severity	Severity Score Range
		None*	0.0	None*	0.0
Low	0.0-3.9	Low	0.1-3.9	Low	0.1-3.9
Medium	4.0-6.9	Medium	4.0-6.9	Medium	4.0-6.9
High	7.0-10.0	High	7.0-8.9	High	7.0-8.9
		Critical	9.0-10.0	Critical	9.0-10.0

[NVD - Vulnerability Metrics \(nist.gov\)](https://nvd.nist.gov/vuln-metrics/cvss)



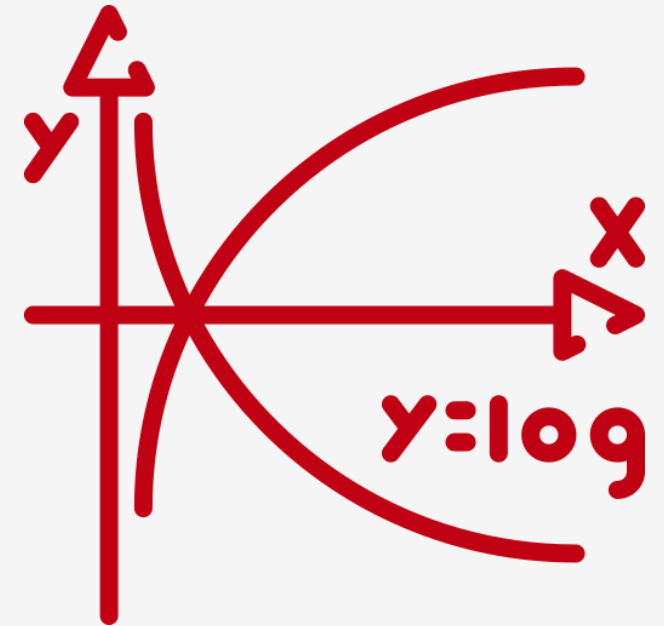
3. Mathematical Background



Mathematical Background

It's all math

- Complex mathematical theories are the foundations for most complex cryptographic functions and algorithms
- Some simple and basic functions are quite useful in assessing security
 - XOR logical function
 - $\log_2 x$
 - Entropy
 - Complexity
 - Big numbers
 - Random numbers



Mathematical Background

XOR logical function

- XOR function (exclusive or) – one of the most used function in cryptographic systems
- Available in programming languages like C, C++, Java and represented by the ^ operator
- Implements mod 2 addition
- Has an essential role in OTP ciphers (one-time pad, stream ciphers) and AES (Advanced Encryption Standard)
- The simplest encryption function



X	Y	$X \oplus Y$
0	0	0
0	1	1
1	0	1
1	1	0

Mathematical Background

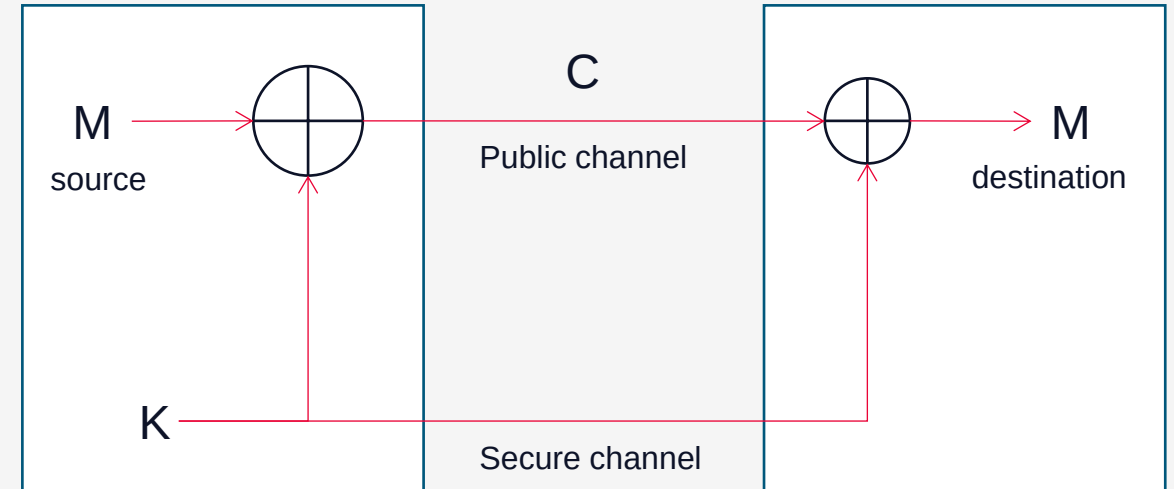
XOR logical function

Cryptographic system based on XOR:

- the fastest
- the simplest
- the most secure one

... but there is a condition

- the key is randomly generated, has the same size with the message M and you can reuse it again



Mathematical Background

Modular Arithmetic

- 'clock arithmetic'
- **uses a finite number of values;**
- **generates results in the same set**
- can do reduction at any point:
 - $a + b \bmod n = [a \bmod n + b \bmod n] \bmod n$
- can do modular arithmetic with any group of integers: $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$



Mathematical Background

Modular Arithmetic

- $(a+b) \bmod n = ((a \bmod n) + (b \bmod n)) \bmod n$
- $(a-b) \bmod n = ((a \bmod n) - (b \bmod n)) \bmod n$
- $(a*b) \bmod n = ((a \bmod n) * (b \bmod n)) \bmod n$
- $(a*(b+c)) \bmod n = (((a*b) \bmod n) + ((a*c) \bmod n)) \bmod n$
- for a k bits modulus the intermediate result of any $+, -, *$ has a maximum of $2k$ bits
- $a^8 \bmod n =$
 - simplest solution: $(a*a*a*a*a*a*a*a) \bmod n$
 - addition chaining: $((a^2 \bmod n)^2 \bmod n)^2 \bmod n$



Mathematical Background

$\log_2 x$

- $2^y = x$ or $y = \log_2 x$
- Used by cryptographic systems because of their accent on binary numbers
- **tells how many bits it takes to represent x in binary**
- no direct implementation in programming languages, but can be computed as

$$\log_2 x = \log_e x / \log_e 2$$

where $\log_e 2 = 0.69314\ 71805\ 59945\ 30941\ 72321$

EXAMPLE

Value 35 requires 6 bits to be represented as $\log_2 35 = 5.129$ and we round it up to the next integer, which is 6



Mathematical Background

Prime Numbers

- You know them from math: 1, 3, 5, 7, 11....
- [Prime Number Theorem](#) gives an approximation for the number of prime numbers less than a given number n .

$$\pi(n) \approx \frac{n}{\ln(n)}$$

where:

- $\pi(n)$ is the number of primes less than or equal to n
- $\ln(n)$ is the natural logarithm of n .
- There are approximately **415.6 trillion (4.156×10^{17})** prime numbers in the 64-bit range
- There are approximately **3.775×10^{151} prime numbers** in the 512-bit range
- There are approximately **2.28×10^{613} prime numbers** in the 2048-bit range



Mathematical Background

Prime Numbers

- Large random prime integers are important components of a cryptographic system
- *“The problem of distinguishing prime numbers from composite numbers and of resolving the latter into their prime factors is known to be one of the most important and useful in arithmetic.”* Carl Friedrich Gauss (1805)
 - Test primes
 - Factor a composite number in primes

Mathematical Background

Testing Prime Numbers

- Naive way.....no way
 - checks $n \% i$ with $i = 2..n-1$; with $i = 2..\sqrt{n}$
- Test that verifies if a number is probably prime - Simple Pseudo-prime Test;
 - are used to increase the algorithm efficiency; the probability to get a correct result is so high that risks are accepted
 - Solovay-Strassen
 - Fermat
 - Rabin-Miller



Mathematical Background

Testing Prime Numbers

Rabin-Miller test for a prime p :

- calculate b , where b is the number of times 2 divides $p - 1$
- calculate m , such that $p = 1 + 2^b \cdot m$.
- (1) Choose a random number, a , such that $a < p$.
- (2) Set $j = 0$ and set $z = a^m \bmod p$.
- (3) If $z = 1$, or if $z = p - 1$, then p passes the test and may be prime.
- (4) If $j > 0$ and $z = 1$, then p is not prime.
- (5) Set $j=j+1$. If $j < b$ and $z \neq p-1$, set $z=z^2 \bmod p$ and go back to step(4). If $z = p - 1$, then p passes the test and may be prime.
- (6) If $j = b$ and $z \neq p - 1$, then p is not prime.



Mathematical Background

Integer Factorization Problem

- For a positive integer n get the factorization $n = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k}$ where p_i are prime values and $e_i \geq 1$.
- Cryptographic algorithms based on this problem:
 - RSA public key encryption
 - RSA signature
 - Rabin public key encryption



Mathematical Background

Factoring Composite Numbers

- The best-known algorithm: Number Field Sieve (NFS) factorization of large integers (http://en.wikipedia.org/wiki/General_number_field_sieve)
- Current world record: RSA-768 (232 digits) – 2 years on multiple machines. Factoring a 1024 bit integer: estimated about 1000 times harder (Dan Boneh, 2012)
- As of 2024, the most significant integer factorization accomplishment is the factorization of **RSA-250**, a **250-digit number, into two 125-digit prime numbers**. This record was achieved in 2020, and the computation took the equivalent of **2700 core-years** of processing time on Intel Xeon Gold 6130 processors (https://en.wikipedia.org/wiki/Integer_factorization_records). If you use 2700 cores, the task would take 1 year
- a 2048-bit number (current key sizes for RSA) has approximately 617 digits



Mathematical Background

Entropy

- the entropy of X represents a mathematical measurement of the amount of information obtained by analyzing X, where X is a set of possible values.
- is the uncertainty regarding the result before analyzing X;
- it represent [Claude Shannon] the number of bits needed to give the shortest binary representation of the message
- **p** is the probability of each unique value from the set to be used
- For example, if you are using a coin to store information then
 - the coin has 2 sides, head or tail, so you could use either of them to store a value vs the other one
 - each side has the same probability to be used, 50%
 - if you do the math, the entropy will be 1, which is true, as you need 1 bit to store 2 possible values (true or false, 1 or 0)

$$\sum_{i=1}^n p_i \log_2 \left(\frac{1}{p_i} \right)$$



Mathematical Background

Entropy

What's the entropy of your 8 case-insensitive alpha (a-z) characters password ?



Mathematical Background

Entropy



Scenario	Available Characters	Required Password Length for 56-Bit Key	Required Password Length for 128-Bit Key
Numeric PIN	10 (0–9)	17	40
Case-insensitive alpha	26 (A–Z or a–z)	12	28
Case-sensitive alpha	52 (A–Z and a–z)	10	23
Case-sensitive alpha and numeric	62 (A–Z, a–z, and 0–9)	10	22
Case-sensitive alpha, numeric, and punctuation	93 (A–Z, a–z, 0–9, and punctuation)	9	20



Mathematical Background

Complexity

- Algorithm complexity is measured by:
 - Input length
 - Processing time
- Complexity classes
 - constant, $f(n) = 1$;
 - linear, $f(n) = n$;
 - logarithmic, $f(n) = \log_2 n$;
 - square, $f(n) = n^2$;
 - cubic, $f(n) = n^3$
 - polynomial, $f(n) = n^c$, cu $c > 1$;
 - exponential, $f(n) = 2^n$ or $f(n) = a^n$, cu $a > 1$.
 - factorial, $f(n) = n!$



Mathematical Background

Complexity

Complexity

	Complexity
Direct access search	$O(1)$
Sequential search	$O(n)$
Binary search	$O(\log_2 n)$
Search in hash tables	$O(GU_{\text{hash}})$
Search in binary balanced search trees (AVL, Red & Black)	$O(\log_2 n)$
Search in B trees	$1 + \log_N((n+1)/2)$, where N is the B tree order
Sequential search in files	$O(n)$
Direct access search in files	$O(1)$
Search in indexed files	$O(\log_2 n)$ for an index of binary balanced search trees type
Search in reverse files	$O(n)$



Mathematical Background

Complexity



Value n	$f(n) = 1$	$f(n) = n$	$f(n) = \log_2 n$	$f(n) = n^2$	$f(n) = 2^n$
10	1	10	3.32	100	1024
100	1	100	6.64	10000	$1,26 * 10^{30}$
1000	1	1000	9.97	1000000	-
10000	1	10000	13.29	100000000	-

Mathematical Background

Complexity

- **The complexity class P (Polynomial Time) is the set of all decision problems that are solvable in polynomial time.**
- **The complexity class NP (Nondeterministic Polynomial Time) is the set of all decision problems for which a YES answer can be verified in polynomial time given some extra information, called a certificate.**
- *The P vs. NP question asks whether every problem in NP can be solved in polynomial time by a deterministic Turing machine, i.e., whether $P = NP$. If $P = NP$, it would mean that every problem for which a solution can be verified quickly (in polynomial time) can also be solved quickly (in polynomial time).* [Source](#)
- One of the unsolved math theories
http://en.wikipedia.org/wiki/Millennium_Prize_Problems
- Over 3000 NP identified problems
http://en.wikipedia.org/wiki/List_of_NP-complete_problems

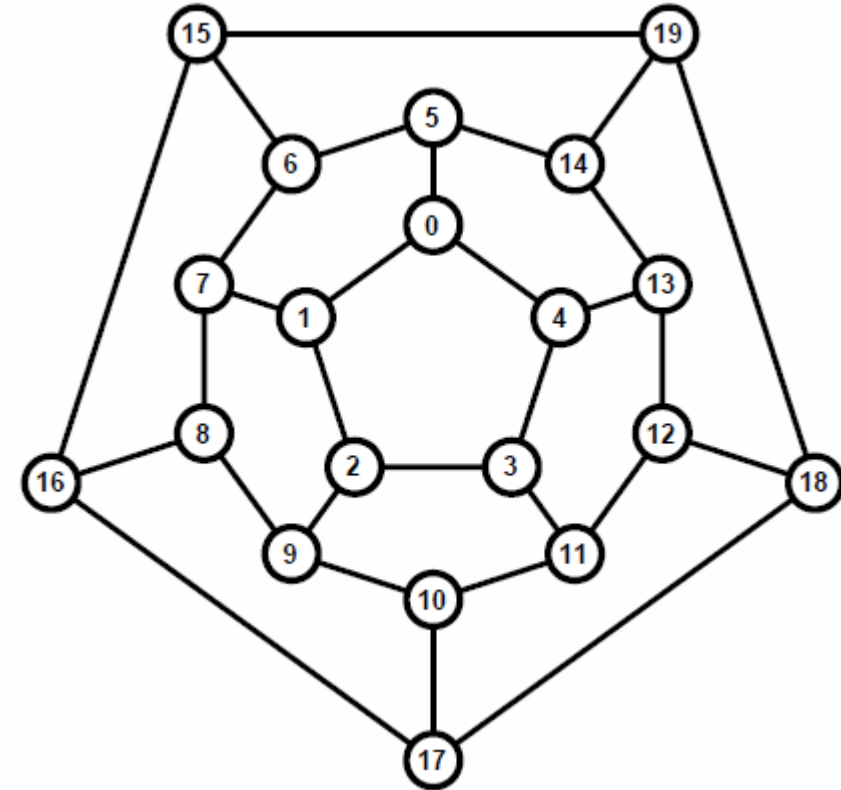
P vs NP



Mathematical Background

Complexity

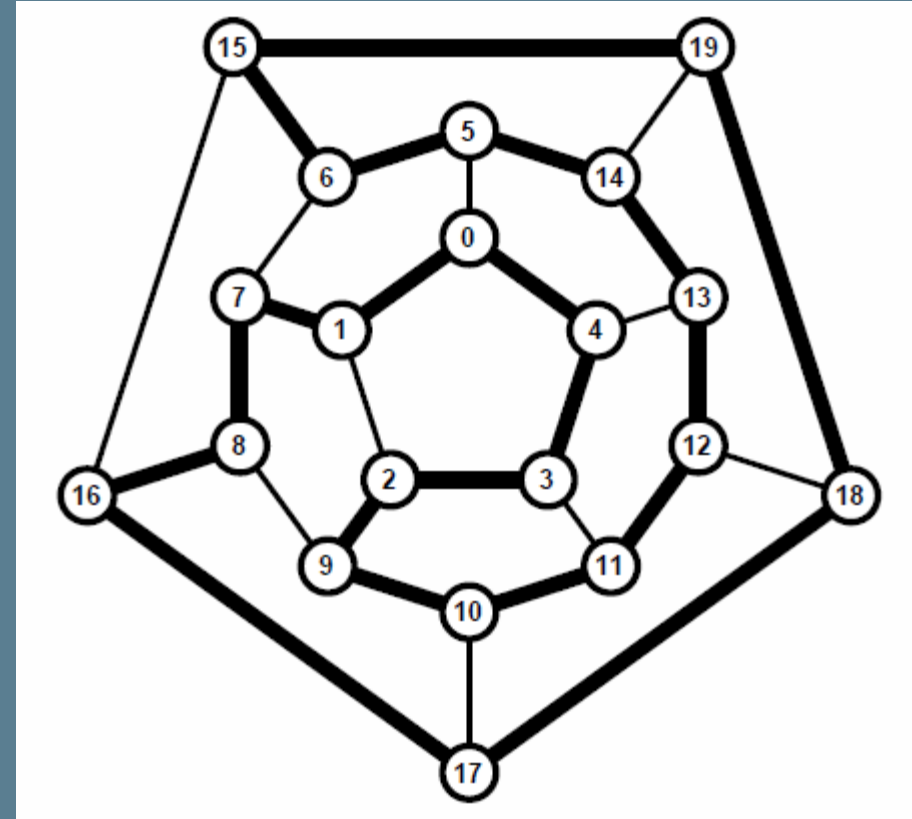
- Find a single path or all paths that will go through each node only once
- Can you solve this fast ?
- A **NP** problem



Mathematical Background

Complexity

- Is this a correct solution ?
- How fast have you checked it ?
- A **P** solution



Mathematical Background

Complex Theories

- From a mathematic viewpoint, the strength of a cryptographic algorithm = problem complexity
- A problem is considered simple if it can be solved (or a large part of solutions) in a polynomial time
- Are defined based on mathematic problems with unknown real complexity
- In well defined conditions (input data carefully selected) the solution is almost impossible to be determined
- Complex theories
 - The integer factorization problem
 - The RSA problem – *RSA inversion*
 - The knapsack problem – subset sum problem
 - The quadratic residuosity problem
 - Computing square roots in $\mathbb{Z}_n$
 - The discrete logarithm problem
 - The generalized discrete logarithm problem
 - The Diffie-Hellman problem
 - The generalized Diffie-Hellman problem



Mathematical Background

Big numbers

numbers with over 10 digits – mostly 100

Examples of big numbers:

Age of Universe: 2^{34}

Numbers of atoms in the planet: $2^{170} \leftrightarrow 10^{51}$

Problems for big numbers ($n = 1024/2048$ bit numbers) in software:

How you store them ?

How you process them with arithmetic operations?

MANAGING BIG NUMBERS

- Stored as fixed length blocks (with additional padding)
- Addition and subtraction (complexity $O(n)$)
- Multiplication
 - the basic approach – complexity $O(n^2)$
 - Karatsuba (1960) – complexity $O(n^{1.585})$
- Division with remainder – complexity $O(n^2)$



Random Numbers

Any value is possible

- number selected from a known set of numbers in such a way that each number in the set has the same probability of occurrence.
- a number obtained by chance.
- one of a sequence of numbers considered appropriate for satisfying certain statistical tests or believed to be free from conditions that might bias the result of a calculation.

[Federal Standard 1037C]



Random Numbers

- many uses of random numbers in cryptography:
 - nonces in authentication protocols to prevent replay;
 - session keys;
 - public key generation;
 - keystream for a one-time pad
 - salt values for “changing” encryption keys
 - initialization arrays for different encryption modes (ex. ECB)
- its critical that these values be:
 - **statistically random, uniform distribution, independent**
 - unpredictability of future values from previous values



Random Numbers

- Are generated by “physical sources that generate random events”, events that can’t be predicted
- Characteristics
 - **Unpredictability:** The next number in a sequence cannot be reasonably predicted based on the previous numbers.
 - **Uniform Distribution:** Over a large set, the numbers should be uniformly distributed across the expected range. This means each number has an equal probability of occurring.
 - **Independence:** Each number in the sequence should be independent of the others. The occurrence of one number should not affect the probability of another number occurring.

6	3	8	5	4	9	7	0	2	2	9	0	3	7	4	4	4	9
8	8	7	0	7	5	0	3	8	1	0	3	7	1	5	4	3	2
0	5	2	8	9	4	4	9	7	1	7	3	7	9	9	6	5	4
2	5	5	6	6	4	4	2	9	5	5	3	6	5	6	6	3	6
4	8	8	7	1	2	3	9	4	3	4	4	8	1	3	5	1	6
2	3	8	5	4	7	8	3	6	0	4	1	7	0	1	6	6	8
8	6	3	9	1	9	1	7	9	5	1	9	5	4	1	0	9	4
5	4	3	5	4	1	2	7	3	4	6	8	9	7	8	2	9	2
6	9	8	3	0	9	4	4	5	6	2	0	8	0	8	9	1	9
6	0	4	4	1	5	4	2	5	9	4	0	1	7	1	8	6	3
0	1	0	5	7	7	9	9	2	6	4	5	8	3	7	5	8	8
1	2	7	7	2	2	1	6	1	6	0	9	6	2	6	6	8	0
1	3	5	9	9	8	1	0	8	8	8	2	4	1	8	8	9	0
3	3	8	5	5	3	6	6	3	1	3	1	1	3	4	4	3	5
1	8	5	2	5	3	5	5	9	3	7	0	1	6	6	7	0	2
2	8	5	6	0	2	4	1	1	8	0	7	1	1	9	9	1	7
7	1	0	7	7	6	4	3	9	0	6	8	8	4	1	9	0	0
1	3	1	2	9	0	4	8	7	5	3	2	6	0	7	3	3	3

Random Numbers

Types of Random Numbers

TRUE RANDOM NUMBERS

Generated from physical processes, such as electronic noise, radioactive decay, or other unpredictable physical phenomena. These are genuinely random and exhibit all the required characteristics

PSEUDORANDOM NUMBERS

Generated by algorithms. While they appear random and can pass statistical tests for randomness, they are deterministic and reproducible if the initial seed value is known.



Random Number Generators

DILBERT By SCOTT ADAMS



Source <https://www.random.org/analysis/>

Random Number Generators

Types

Random Number Generator RNG - produce a sequence of zero and one bits that may be combined into sub-sequences or blocks of random numbers:

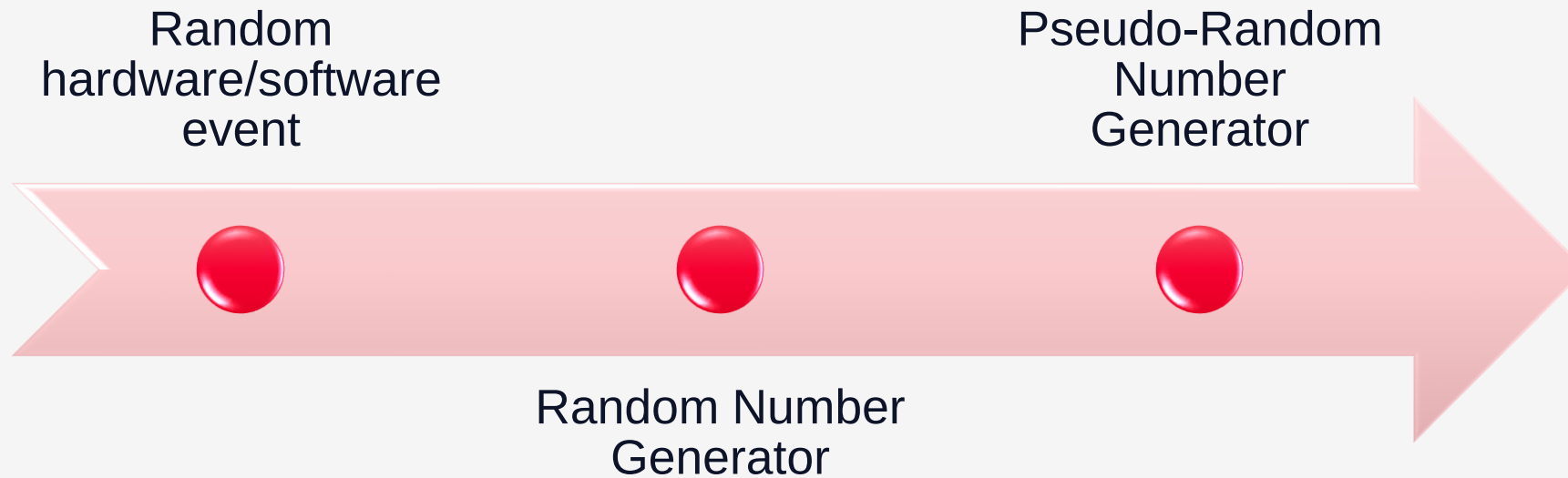
- **Pseudo Random Number Generator (PRNG):** Deterministic, based on an algorithm
- **True Random Number Generator (TRNG):** Nondeterministic

Another classification of NIST:

- **Random Bit Generator RBG** - a device or algorithm that outputs a sequence of binary bits that appears to be statistically independent and unbiased. An RBG is either a **Deterministic RNG (DRBG)** or a **Non-deterministic RBG (NDRBG)** [NIST Special Publication 800-90]



Random Number Generators



Random Number Generators

How to measure it

- The measure of randomness = *entropy*
- For a sequence of 16 bytes that are completely random (and *unbiased*) -> 128 bits of entropy -> the *security strength* of the value is 128 bits -> the amount of work required to break the security is 2^{128} operations
- what for 2 bytes ?



True Random Numbers Generators (TRNG)

- Are using “physical sources that generate random events”, events that can’t be predicted
- Sources
 - Electronic noise of semiconductor devices
 - The least significant bits of an audio channel
 - Intervals between interrupts of hardware devices
 - Logging pressed keys in an interval or recording cursor position
- Processing
 - The event is “distilled” by a cryptographic hash function to increase the dependence between bits
- Examples
 - Cloudflare Lava Lamp wall
 - <https://www.random.org/> based on atmospheric noise

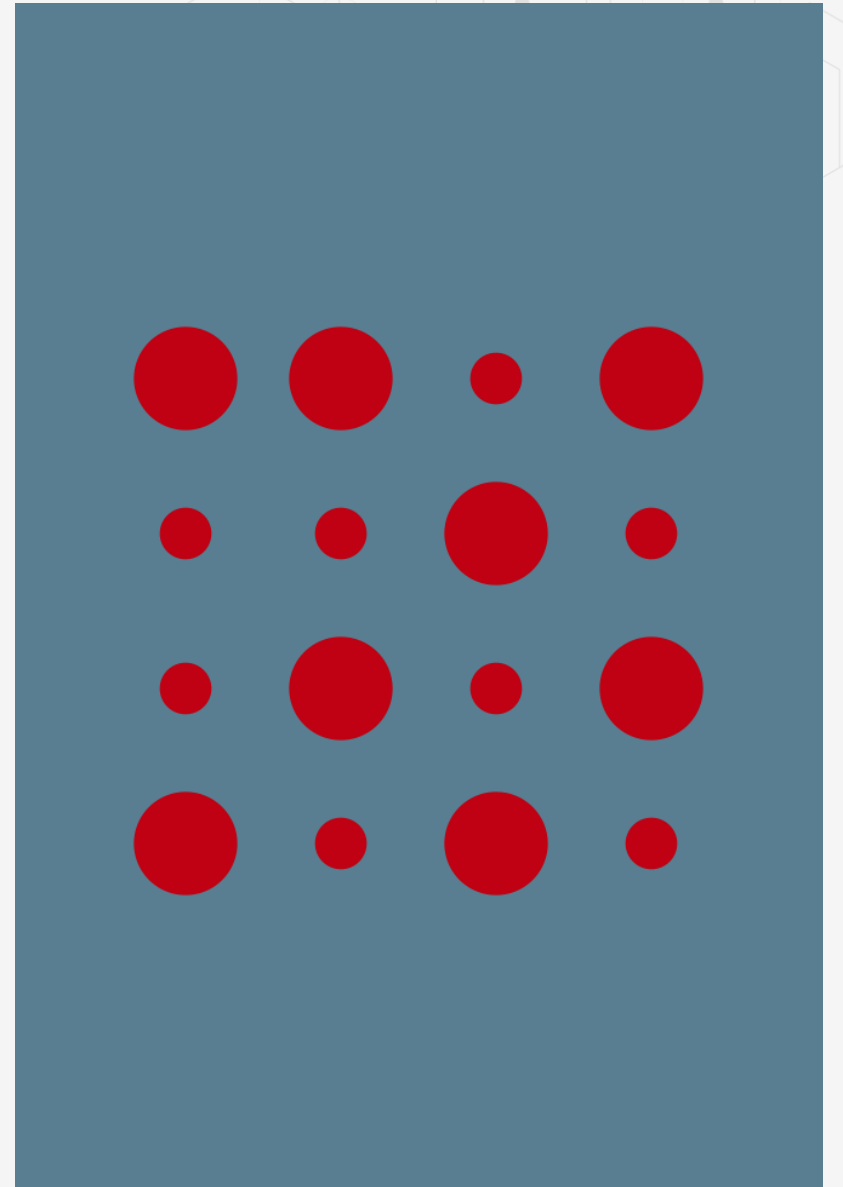


Pseudo Random Numbers Generators (PRNG)

A RNG that produces a sequence of values based on a seed and a current state. Given the same seed, it will always output the same sequence of values

Types of PRNG:

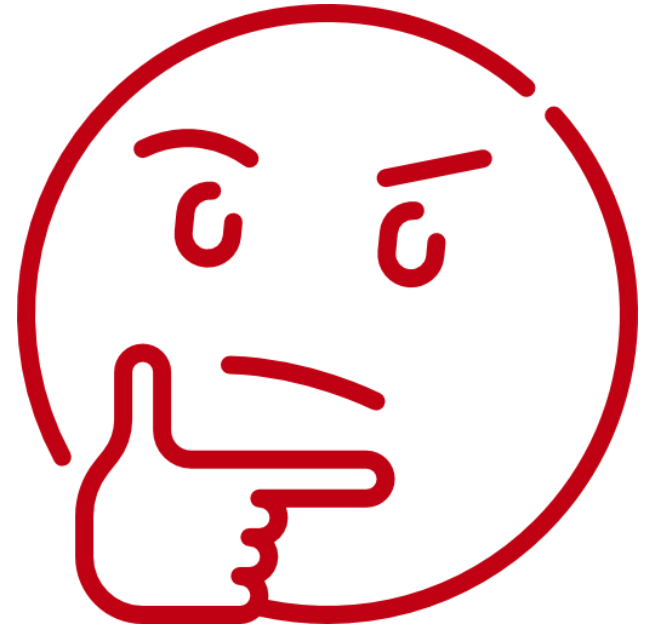
- **Statistically random** - will pass various statistical tests of randomness [FIPS 800-22]
- **Cryptographically secure** - knowing generated random data, an attacker will not be able to predict the rest
- **Security strength of n bits** - the amount of work (of operations) required to break the system is 2^n



Pseudo Random Numbers Generators (PRNG)

One should not use a random method to generate random numbers. [Donald Knuth]

Anyone who uses software to produce random numbers is in a “state of sin”. [John von Neumann]



Random Numbers Generators

Evaluation

- RNG must generate sequence of values that are uniform distributed and independent (difficult to analyze)
- Key tests:
 - **bit count** [an even distribution is expected]
 - **word count**: counts the number of k-bit words (01010101.... – fails)
 - **gap space count**: the size of the gaps between the zero / one bits
 - **autocorrelation**: tries to determine if a subset of bits is related to another subset from the same string

1111 and 1110 -> correlated

1111 and 0000 -> correlated

1100 and 1010 -> perfectly uncorrelated

$$R(j) = \sum_n x_n \text{ XOR } x_{n-j}$$

-> n/2 for uncorrelated streams

-> 0 or n for correlated streams



Random Numbers Generators

NIST Guide to the Statistical Tests

- Frequency Test: Monobit
- Frequency Test: Block
- Runs Test
- Test for the Longest Runs of Ones in a Block
- Binary Matrix Rank Test
- Discrete Fourier Transform (Spectral Test)
- Non-Overlapping Template Matching Test
- Overlapping Template Matching Test
- Maurer's Universal Statistical Test
- Linear Complexity Test
- Serial Test
- Approximate Entropy Test
- Cumulative Sums Test
- Random Excursions Test
- Random Excursions Variant Test

[NIST Guide to the Statistical Tests](#)



Pseudo Random Numbers Generators (PRNG)

- **Can become the weakest link of the cryptographic system**
- Example: [*How we Learned to Cheat in Online Poker: A Study in Software Security*](#), by Brad Arkin et. al.

Characteristics:

- Simple and fast
- Must generate variable length numbers that does not repeat (maximizing the period is better because it is impossible to make it going to infinity)
- Must generate independent values
- Must generate uniform distributed numbers



Pseudo Random Numbers Generators (PRNG)

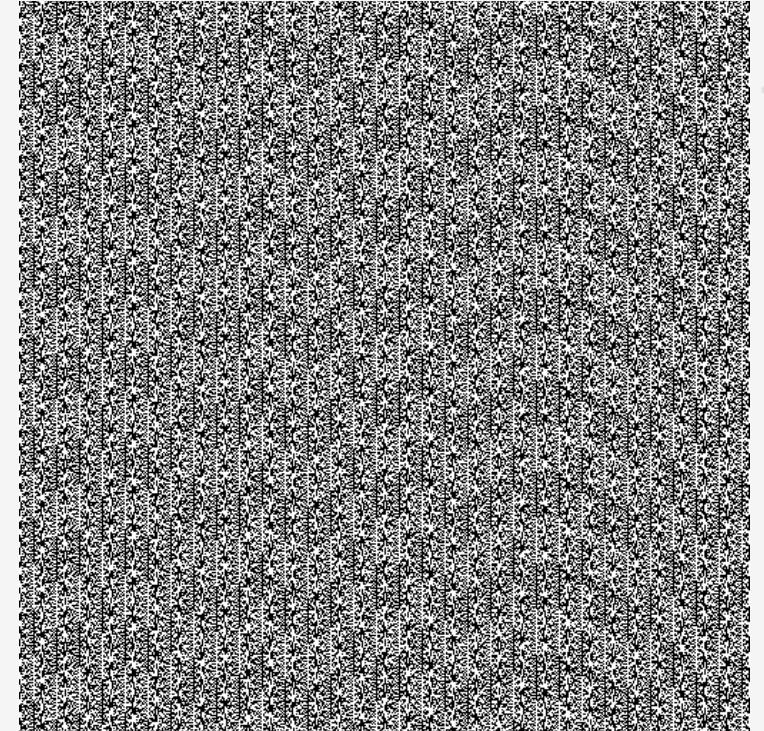
- rand function in C++ (MS VC C++ 7)

```
int __cdecl rand (void) {  
    return(((holdrand = holdrand * 214013L + 2531011L) >> 16)  
        & 0x7fff);  
}
```

- rand function in C (Kernighan & Ritchie C Standard)

```
unsigned long int next = 1;  
int rand(void) {  
    next = next * 1103515245 + 12345;  
    return (unsigned int)(next/65536) % 32768;  
}
```

- In cryptography you **DO NOT USE** rand functions from C,C++, Java programming languages because they are predictable



PHP rand() on Microsoft Windows

<https://www.random.org/analysis/>

Pseudo Random Numbers Generators (PRNG)

Some good examples

- **Yarrow** – most known PRNG
 - Defined by Bruce Schneier, John Kelsey and Niels Ferguson in Counterpane Labs and provided free as open-source solution
 - An improved design from Ferguson and Schneier, Fortuna
 - Yarrow was used in FreeBSD, iOS and macOS for their /dev/random devices, but is now superseded by Fortuna.
 - <http://www.schneier.com/yarrow.html>
- **ANSI X9.17**
 - One of the safest PRNG that uses encryption (triple DES - EDE)
 - It starts with 2 initial pseudo-random values: 64 bit value of current date and time, random generated 64 bit seed value
 - Uses 3 encryption modules that use triple-DES 56 bit key
 - It generates 2 * 64 bit values: a pseudo-random number and a seed value
- **Blum Blum Shub Generator** – BBS
 - One of the most used generators
 - Highly secure – it uses the factorization problem
 - Generates pseudo-random values of any length
 - cryptographically secure pseudorandom bit generator (CSPRBG) – it pass the *next-bit* test



Pseudo Random Numbers Generators (PRNG)

Linear congruential generator

- Uses sequential sets of pseudo-random numbers $\{U_n\} = U_0, U_1, \dots$ cu $0 \leq U_n \leq 1$
- Methods to generate $\{U_n\}$:
 - linear congruence method
 - adding congruence method
 - multiplicative congruence method
 - Linear Feedback Shift Registers Generator (LFSR)
 - meter generator method

Characteristics:

- Simple and fast
- Must generate variable length numbers that does not repeat (maximizing the period is better because it is impossible to make it going to infinity)
- Must generate independent values
- Must generate uniform distributed numbers

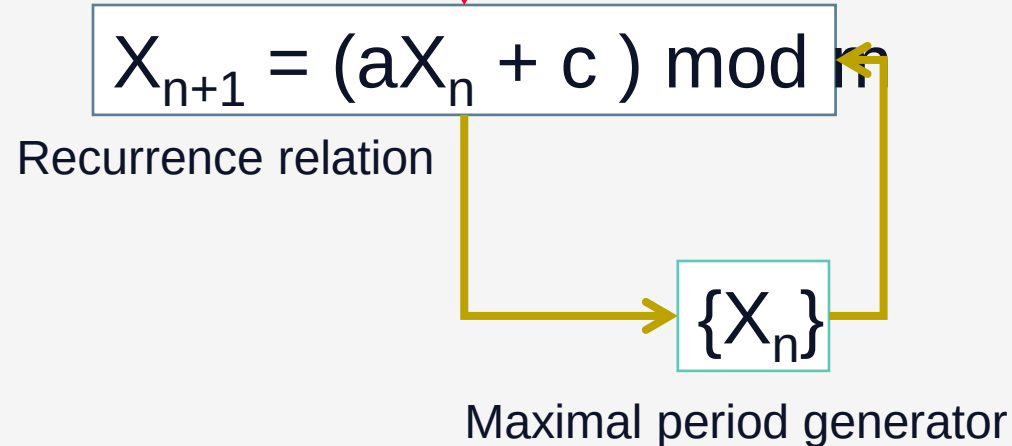


Pseudo Random Numbers Generators (PRNG)

Linear Congruential Generator

m – module, with $m > 0$
a – multiplier, with $0 \leq a < m$
c – increment, with $0 \leq c < m$
 X_0 – initial value, with $0 \leq X_0 < m$

Magic numbers



Pseudo Random Numbers Generators (PRNG)

Linear Congruential Generator

m	a
2^{31}	65539
$2^{31}-1$	16807
$2^{31}-249$	40692
$2^{31}-1$	48271
$2^{31}-1$	62089911
2^{32}	69069
2^{48}	31167285
2^{64}	6364136223846793005

- Linear congruence method



Pseudo Random Numbers Generators (PRNG)

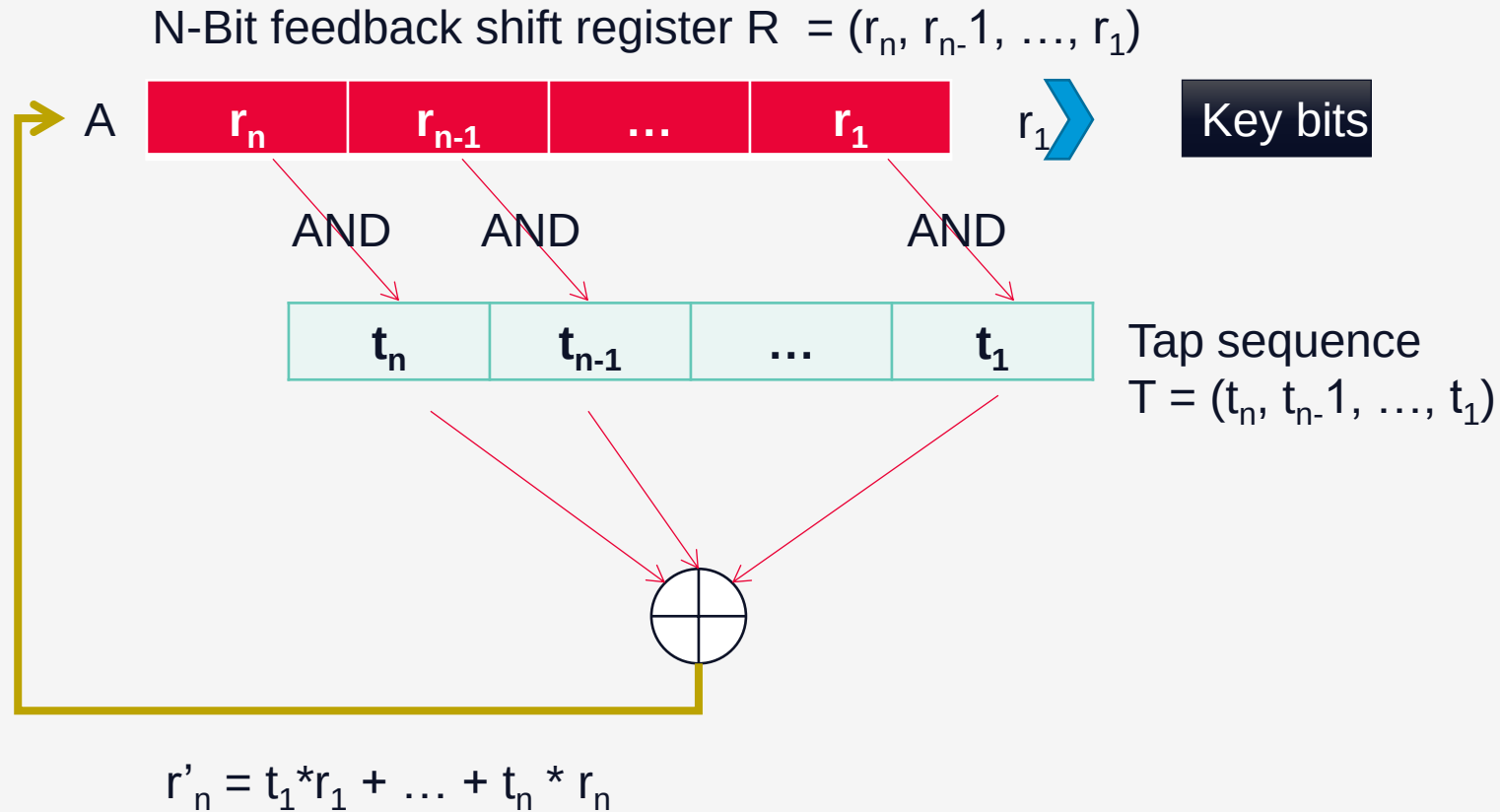
Linear Congruential Generator

- NOT used anymore in cryptography
- broken in 1977 by Jim Reeds
- combining linear congruential generators gives better results (ex. $2^{31} - 85$ with $2^{31} - 249$)

Overflow at	a	c	m
2^{31}	8121	28411	134456
2^{31}	4561	51349	243000
2^{31}	7141	54773	259200
2^{32}	9301	49297	233280
2^{32}	4096	150889	714025
2^{33}	2416	374441	1771875
2^{34}	17221	107839	510300

Pseudo Random Numbers Generators (PRNG)

Linear feedback shift register generator - LFSR



Pseudo Random Numbers Generators (PRNG)

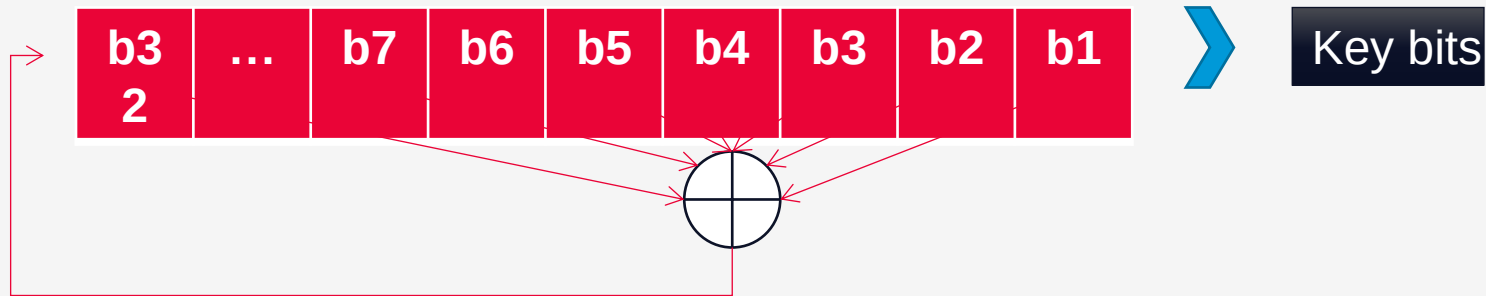
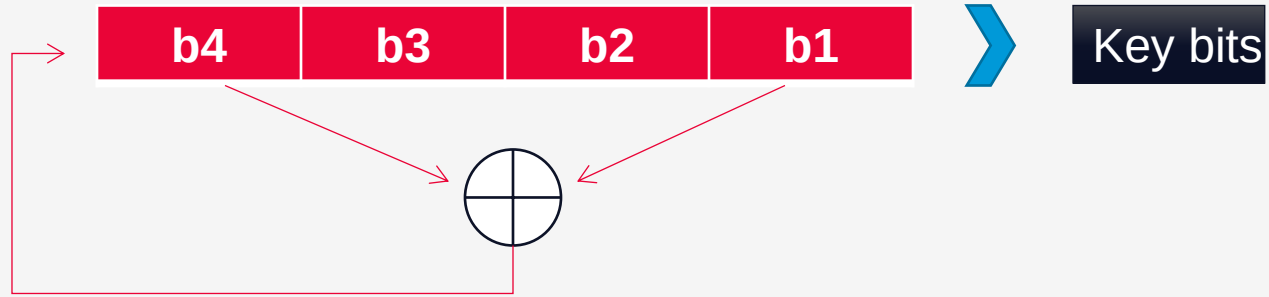
Linear feedback shift register generator - LFSR

- simple feedback sequence
- an n-bit LFSR can have $2^n - 1$ internal states (depends on the tap sequence – maximal period LFSR) – the polynomial formed by tap sequence plus constant 1 must be a primitive polynomial mod 2 (ex. $x^{10} + x^3 + 1$)
- stream ciphers have been built based on LFSR (ex. A5 for GSM) because they can be easily implemented in hardware
- competent pseudo-random-sequence generators
- *Berlekamp-Massey* algorithm can determine the feedback function from only $2 \cdot n$ output bits



Pseudo Random Numbers Generators (PRNG)

Linear feedback shift register generator - LFSR

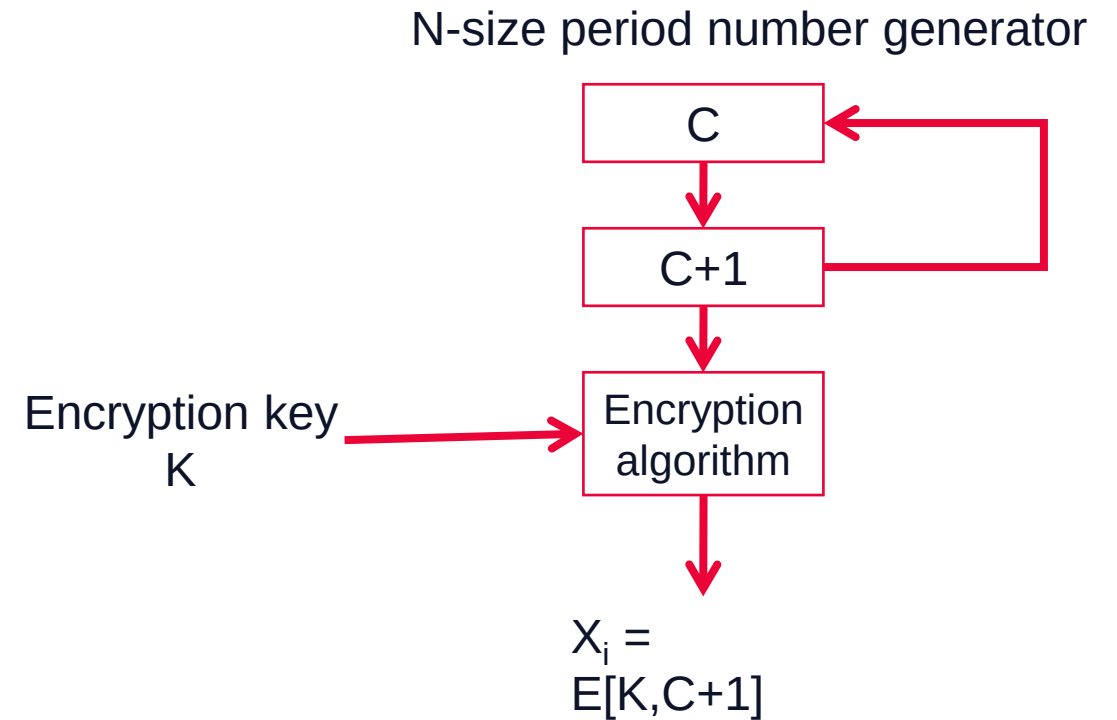


Pseudo Random Numbers Generators (PRNG)

Examples

- Some PRNGs use cryptographic methods – repeated encryption of an input

To generate 56 bit DES keys, the number generator has a $N = 2^{56}$ period



Pseudo Random Numbers Generators (PRNG)

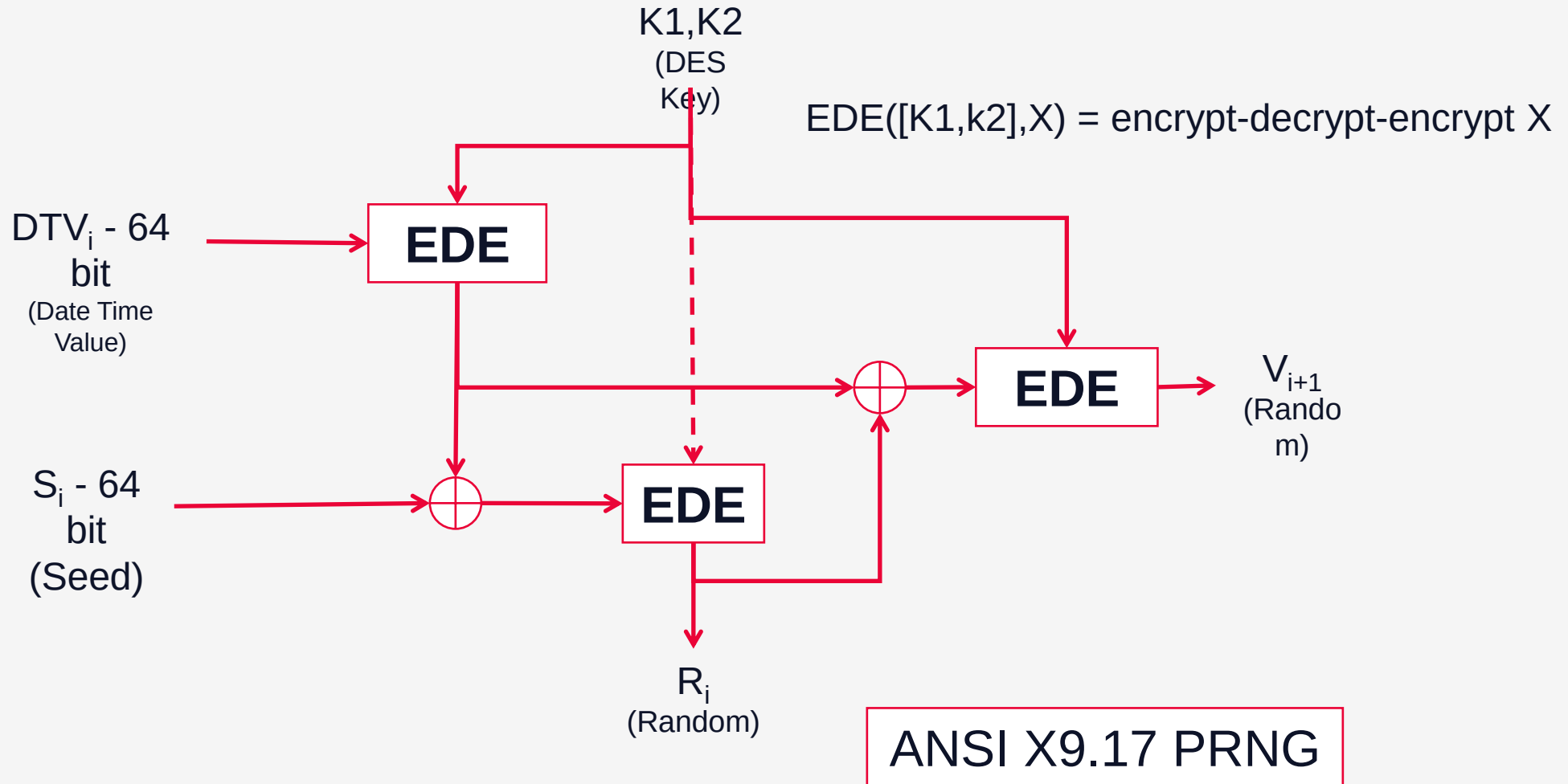
ANSI X9.17

- One of the safest PRNG that uses encryption (triple DES - EDE)
- It starts with 2 initial pseudo-random values: 64 bit value of current date and time, random generated 64 bit seed value
- Uses 3 encryption modules that use triple-DES 56 bit key
- It generates 2 * 64 bit values: a pseudo-random number and a seed value



Pseudo Random Numbers Generators (PRNG)

ANSI X9.17



Pseudo Random Numbers Generators (PRNG)

Blum Blum Shub Generator (BBS)

- One of the most used generators
- Highly secure – it uses the factorization problem
- Generates pseudo-random values of any length
 - 2 prime and large numbers are generated, p and q such that $p \equiv q \equiv 3 \pmod{4}$
 - It is computed $n = p * q$
 - It is selected a random seed number s that is relatively prime to n ($\gcd(s,n) = 1$)
 - Each bit is determined by

$$X_0 = s^2 \pmod{n}$$

for $i = 1$ to n

$$X_i = (X_{i-1})^2 \pmod{n}$$

$$B_i = X_i \pmod{2}$$

-- random bit



Pseudo Random Numbers Generators (PRNG)

Blum Blum Shub Generator (BBS)

- At each iteration, the least significant bit is selected to generate the random value

$n = 192649$
 $p = 383$
 $q = 503$
 $s = 101355$

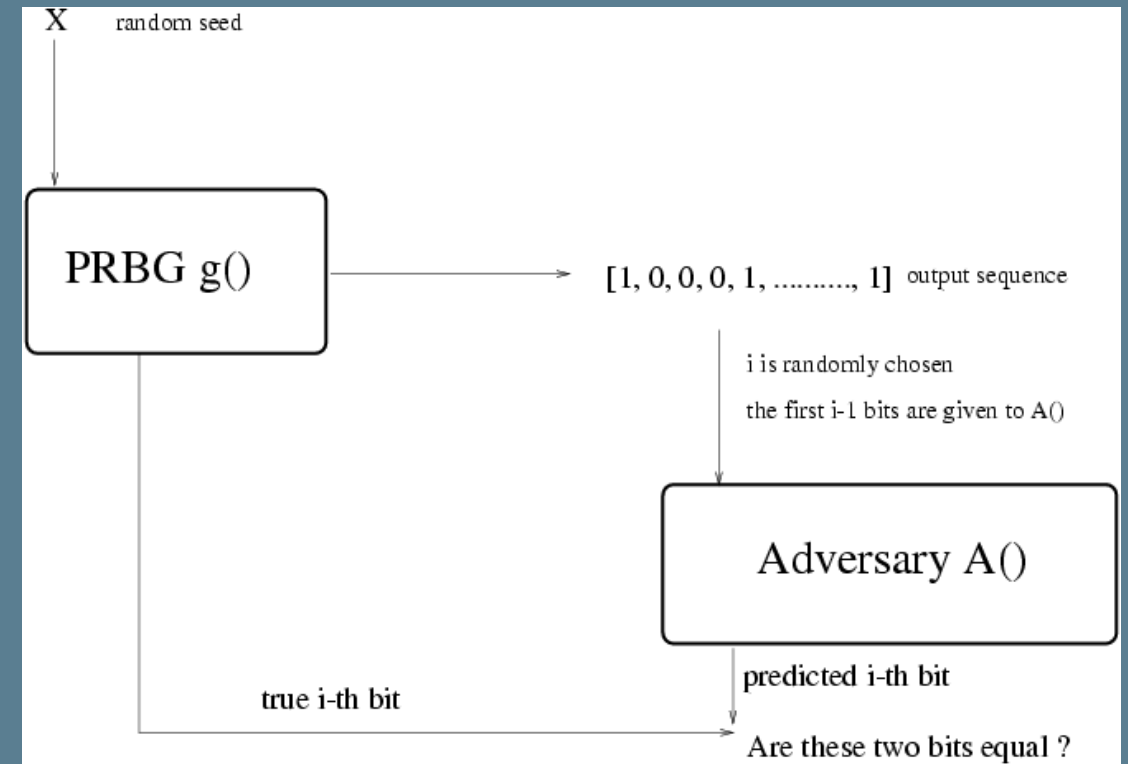
i	X_i	B_i
0	20749	
1	143135	1
2	177671	1
3	97048	0
4	89992	0
5	174051	1
6	80649	1
7	45663	1
8	69442	0
9	186894	0
10	177046	0

i	X_i	B_i
11	137922	0
12	123175	1
13	8630	0
14	114386	0
15	14863	1
16	133015	1
17	106065	1
18	45870	0
19	137171	1
20	48060	0

Pseudo Random Numbers Generators (PRNG)

Blum Blum Shub Generator (BBS)

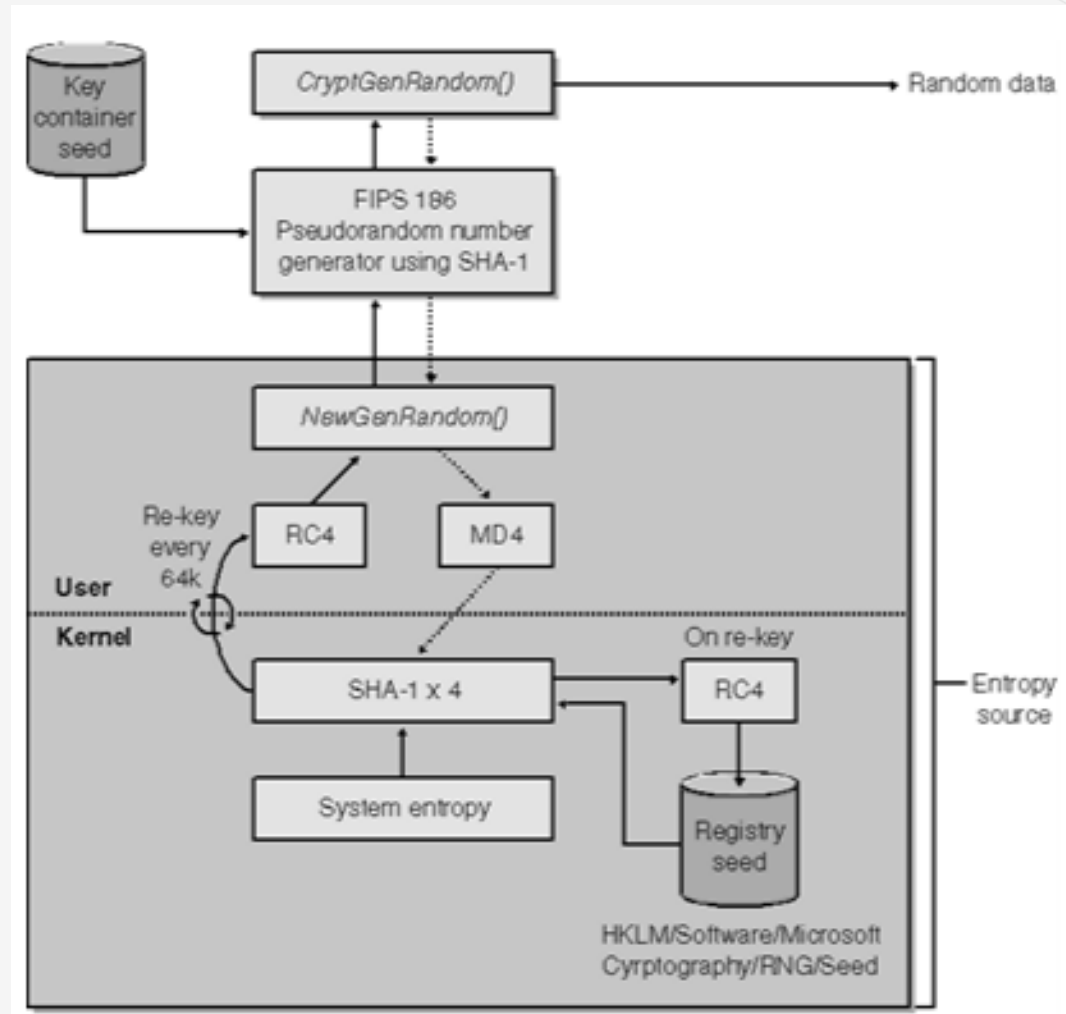
- Cryptographically Secure Pseudorandom Bit Generator (**CSPRBG**) – it pass the *next-bit* test
- Statistical experiment for the next-bit test



https://www.researchgate.net/publication/238120837_Cryptographic_Secure_PseudoRandom_Bits_Generation_The_Blum-Blum-Shub_Generator

Pseudo Random Numbers Generators (PRNG)

Microsoft PRNG – an older version



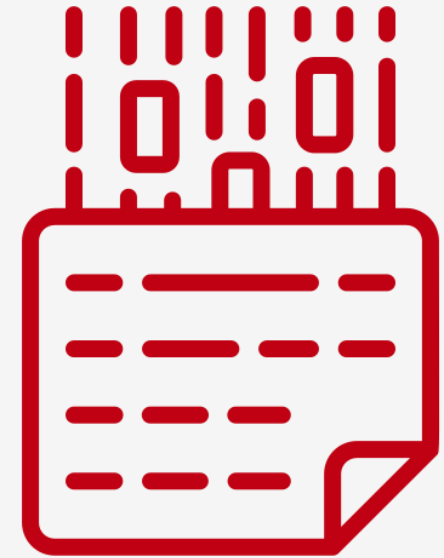
[Microsoft]

Encoding

Not encryption

Encoding is NOT Encryption

- Encoding is the process of putting a sequence of characters into a specialized format for efficient transmission or storage
- Encoding schemes: ASCII, Unicode, URL encoding, Base64



Base64 Encoding

- In computer programming, Base64 is a group of binary-to-text encoding schemes that represent binary data (more specifically, a sequence of 8-bit bytes) in sequences of 24 bits that can be represented by four 6-bit Base64 digits
- has 64 characters that are common to most encodings and that are also printable
- commonly used to store binary values as text
- <https://en.wikipedia.org/wiki/Base64>

Value	Encoding
VGhpcyBpcyBhIHRlc3Q=	Base64
54 68 69 73 20 69 73 20 61 20 74 65 73 74	Hexadecimal
01010100 01101000 01101001 01110011 00100000 01101001 01110011 00100000 01100001 00100000 01110100 01100101 01110011 01110100	Binary
This is a test	ASCII

Quantum Computing

Current phase

- In 2022 IBM revealed its quantum computing system with a processor that contained 433 quantum bits
- The target is to have a 100.000 qubit machine within 10 years
- Have not achieved anything useful that standard supercomputers cannot do
- Today issues of quantum computing: not enough qubits and disruptions by tiny perturbations in their environment ~ noise
- Google is targeting a million qubits by the end of the decade, but most of them will be used for error correction and only 10,000 will be available for computations.



Quantum Computing

Current phase

- IBM's qubits are made from rings of superconducting metal, which are operated at millikelvin temperatures, a tiny fraction of a degree above absolute zero
- IBM estimates that with current technology quantum computers can only scale up to 5,000 qubits
- Now, each one of IBM's superconducting qubits requires around 65 watts to operate. "If I want to do 100,000, that's a lot of energy: I'm going to need something the size of a building, and a nuclear power plant and a billion dollars, to make one machine," (IBM's VP of quantum, Jay Gambetta)
<https://www.technologyreview.com/2023/05/25/1073606/ibm-wants-to-build-a-100000-qubit-quantum-computer>



Quantum Computing

Post Quantum Cryptography PQC

- Quantum computers are an inevitable threat to digital security
- It's estimated that within the decade a quantum computer will be powerful enough to break cryptography as we know it today
- public key systems will be affected as symmetric key algorithms, like AES, are secure enough with bigger keys
- the goal is to move today's public key cryptographic systems from current RSA and ECC algorithms to new quantum safe algorithms
- In 2016, the National Institute of Standards and Technology (NIST) started the competition to select quantum-resistant algorithms - 69 candidate algorithms
- Summer of 2022, they announced the first set of winners,
 - one public key encryption algorithm (CRYSTALS-Kyber)
 - three digital signature algorithms (CRYSTALS-DILITHIUM, FALCON, and SPHINCS+)
- In 2024 there are expected final recommendations and formal standards for post-quantum cryptography
 - <https://csrc.nist.gov/Projects/post-quantum-cryptography>
 - https://en.wikipedia.org/wiki/Post-quantum_cryptography



Protocols

- a series of steps, involving 2 or more parties, designed to accomplish a task
- **types of protocols:**
 - **arbitrated** – with a trusted third party
 - **adjudicated** – 2 lower-level subprotocols (one nonarbitrated and one arbitrated)
 - **self-enforcing**
- **Used for:**
 - Secret key establishment
 - Elections
 - Auctions
 - Secure multi-party computation



Protocols

Coin-flipping Protocol

Conditions defined by Alice and Bob:

- There is a “**magic**” function f
 - it is easy to determine $f(x)$, but is impossible to determine x , knowing $f(x)$
 - it is impossible to find 2 values, x and y with $x \neq y$ such that $f(x) = f(y)$
- It is defined the correlation head = x if even, tail = x if odd
- The story
 - Alice chose a large random number x and tells Bob the value of $f(x)$
 - Bob says if x is odd (tail) or even (head) - he flips the coin and tells the result
 - Alice tells Bob the value of x
 - Bob computes $f(x)$ and checks if he has won or lost.



3. Hash functions



Hash functions

Definition

- a cryptographic one-way function $H(M)$ used to compute a fixed value h (hash) *unique* for the variable-length message
- used to provide **data integrity** to messages
- has a role in **authenticating the message content** (data authentication)
- used in digital signature procedure
- like a checksum value but more secure
- **DOES NOT hide the content**



Sponge Function

Not Sponge Bob

A class of algorithms with finite internal state that take an input bit stream of any length and produce an output bit stream of any desired length



Hash functions

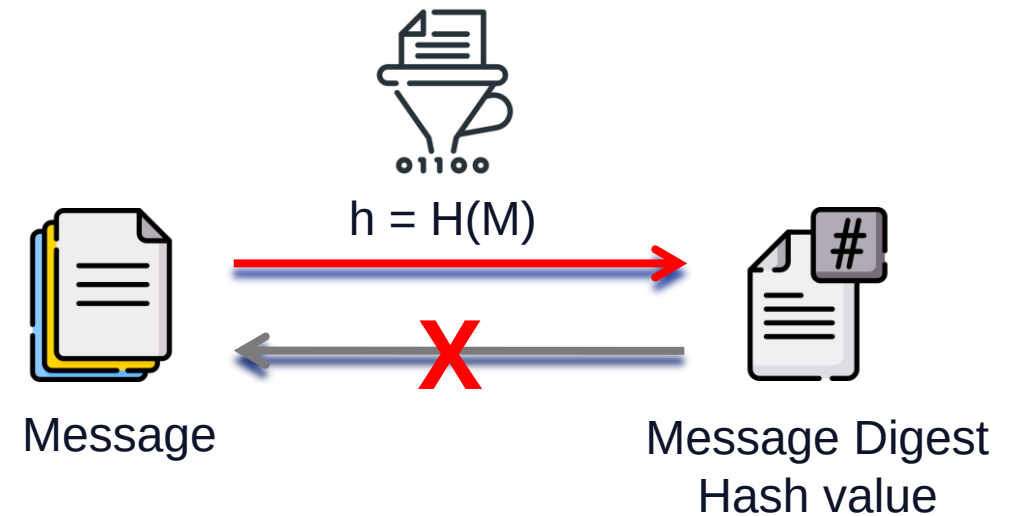
Characteristics

- for M (variable-length input), it is easy to compute h (fixed length)
- for h , is IMPOSSIBLE to determine M
- for M , it is very difficult to find M' such as $H(M) = H(M')$
- **collision-free** = the difficulty in finding M'
- modifying a single bit of M , the h value is totally different
- Properties
 - **Deterministic** – same input leads to same output
 - **Fast computation**
 - **Pre-Image Resistance**
 - **Collision Resistant**
 - **Puzzle Friendly**

MD5("a") = "0C C1 75 B9 C0 F1 B6 A8 31 C3 99 E2 69 77 26 61"

MD5("b") = "92 EB 5F FE E6 AE 2F EC 3A D7 1C 77 75 31 57 8F"

MD5(1 GB file) = "1D B9 DF A8 F2 7E 36 37 7C 07 5A C0 B8 4C 08 F3"



Hash functions

Algorithms



Function	Hash length (bits)
SHA-1 (Secure Hash Algorithm or SHS – Secure Hash Standard)	160
SHA-256	256
SHA-3	224, 256, 384, 512
RIPEMD-160	160
MD5 (Message Digest Algorithm)	128
Tiger	128
MD2, MD4	128

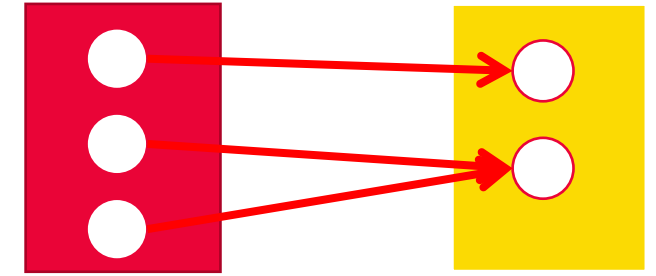


Hash functions

Properties

Collision-free:

- A collision for the hash function $h : D \rightarrow \{0, 1\}^n$ is the pair $x_1, x_2 \in D$ such that $h(x_1) = h(x_2)$ but with $x_1 \neq x_2$.
- Because $|D| > 2^n$
- The input size can vary from 1 byte to any value that you can store, hence you have an almost infinite input set, but the output is always fix (ex 160 bits for MD5)
- There is no correlation between the input size and hash value
- For known limited inputs (like passwords and keys) the attacker can brute force the initial value. Has more chances to find it rather than find a collision with any size



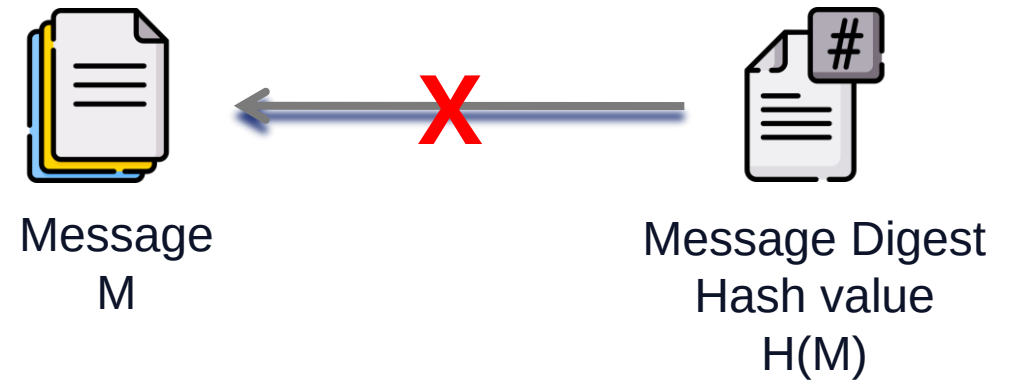
Collisions exist, but are difficult to find them

Hash functions

Properties

Pre-Image Resistance

- given $H(M)$ it is infeasible to determine M , where M is the input and $H(M)$ is the output hash
- Is NOT impossible to determine A
 - by brute-force you can try all possible inputs
 - If the input range is small enough you can do it

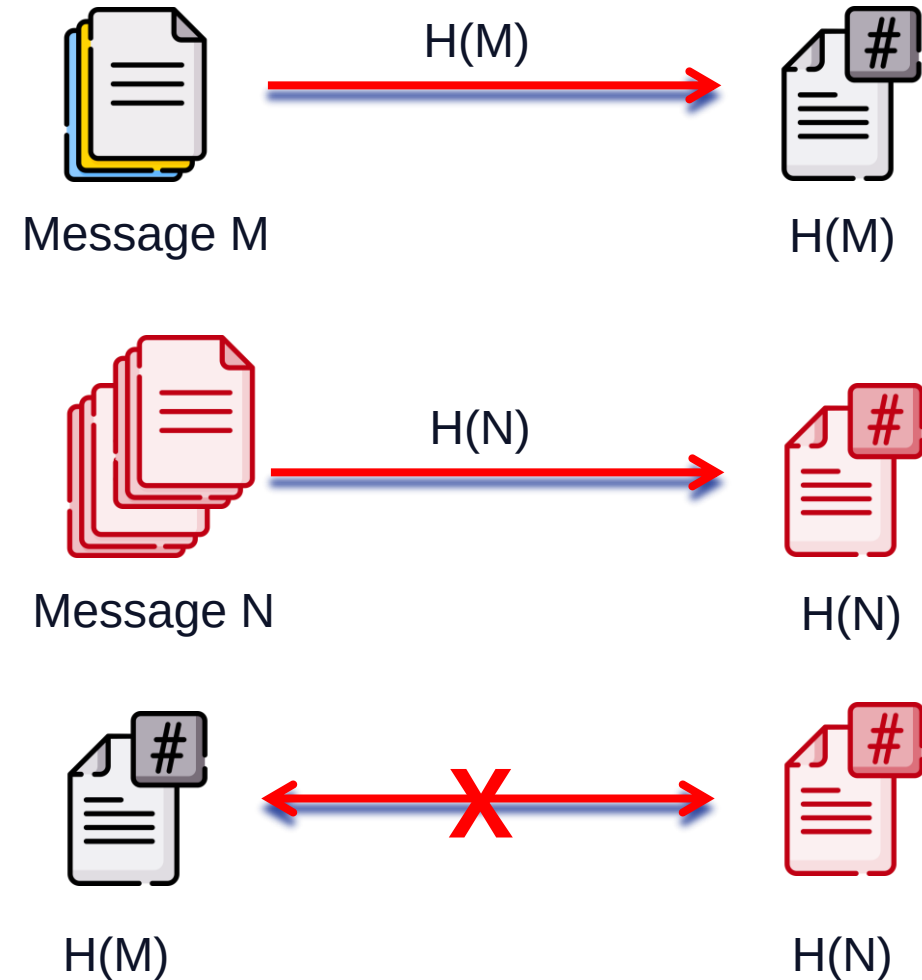


Hash functions

Properties

Collision Resistant:

- for two different inputs A and B where $H(A)$ and $H(B)$ are their respective hashes, it is infeasible for $H(A)$ to be equal to $H(B)$
- Is NOT impossible
 - MD 5: collision resistance was broken after $\sim 2^{21}$ hashes.
 - SHA 1: collision resistance broke after $\sim 2^{61}$ hashes.

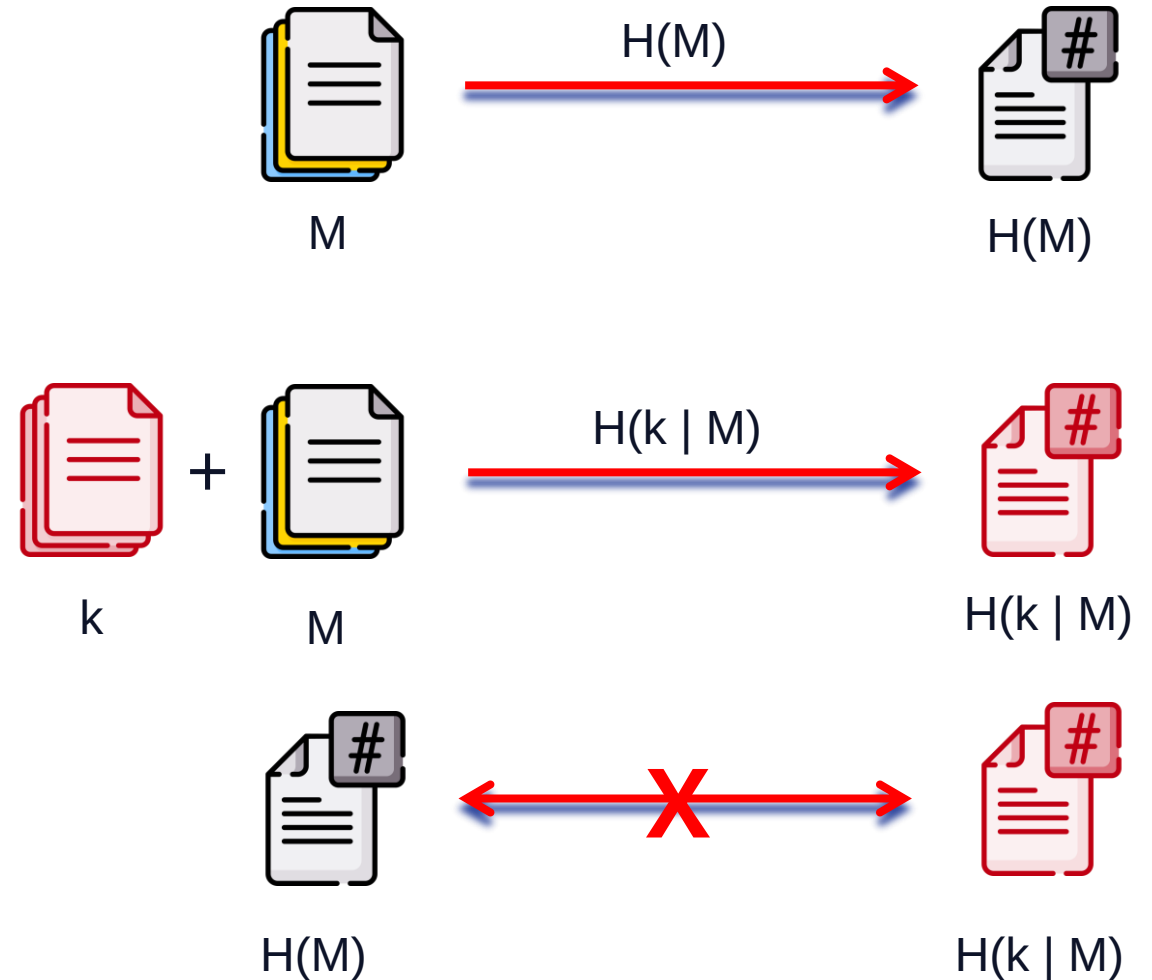


Hash functions

Properties

Puzzle Friendly

- For every output “Y”, if k is chosen from a distribution with high min-entropy it is infeasible to find an input M such that $H(k | M) = Y$
- high min-entropy - the distribution is hugely distributed so choosing a random value has a negligible probability (ex $[1..2^{256}]$)
- low min-entropy distribution – $[1..5]$



Hash functions

Properties

Deterministic

- for same input it will generate same output

Fast computation

- able to process large volumes of data
- low min-entropy distribution – [1..5]



Birthday paradox

What are the chances ?

- What is the probability to meet a random person on the street that has the same **birthday** (day not year) as you ?
 - if all days of the year have the same probability, the chances of another person sharing your birthday is $1/365$ which is 0.27%. Is low
- How many random people should you have in a room so the probability will be around 50% ?



Birthday paradox

What are the chances ?

- For N different possibilities of an event to happen, then you need square root of N random items for them to have a 50% chance of a collision
- For 365 different possibilities of birthdays, you need $\text{Sqrt}(365) = \sim 23$, randomly chosen people, for 50% chance of two people sharing same birthday (same day, not day and year)
- there are needed 253 persons in a room to have > 50% probability that one of them shares your birthday
- Allows “birthday attack” on a hash function to find collisions
- In conclusion, a collision can be found in almost $\text{Sqrt}(2n) \approx 2^{n/2}$ tries.
- the “*birthday attack*” allows attackers to find faster (with a high probability of ~ 50%) two random messages, M and M', such that $H(M) = H(M')$
- as a result, hashes have half of their digest size in strength: MD5 needs 2^{64} effort to find collisions



Hash functions

MD5

- Proposed by Ronald Rivest from MIT and developed by RSA Data Security company
- generates a 128 bit hash
- defined based on MD4
- has 5 important stages
- Attacks on MD5
 - In 2005 researches have announced that can find collisions for the hash function; now, the collisions can be generated in couple of hours
 - EuroCrypt 2005 - "How to break MD5 and other hash functions", Xiaoyun Wang et. Al.
 - two different Win32 executable with different functionality but equal MD5 hash values, <http://www.win.tue.nl/hashclash/SoftIntCodeSign/>
 - Generates 2 random messages with only 3 different bits that have same hash, <http://www.cs.colorado.edu/~jrblack/>



Hash functions

SHA-1 and SHA-2

- It was designed by the United States National Security Agency and is a U.S. Federal Information Processing Standard. The algorithm has been cryptographically broken but is still widely used
- SHA-1 Collision Search Graz - <http://www.iaik.tugraz.at>
- Based on this version, the second generation has been developed: SHA-256, SHA-512
- Attacks on SHA-1
 - Since 2005, SHA-1 has not been considered secure against well-funded opponents
 - Marked as deprecated by NIST in 2011
 - Not allowed for digital signatures since 2013,
 - Should be completely removed by 2030
 - In February 2017, [CWI Amsterdam](#) and Google announced they had performed a collision attack against SHA-1, publishing two dissimilar PDF files which produced the same SHA-1 hash
- SHA-256, SHA-512 still safe compared with the issues of SHA-1
- <https://en.wikipedia.org/wiki/SHA-1>



Hash functions

Comparison

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Max message size (bits)	Rounds	Security (bits)	Example Performance (MiB/s) ^[26]
MD5 (as reference)		128	128 (4×32)	512	$2^{64} - 1$	64	<64 (collisions found)	335
SHA-0		160	160 (5×32)	512	$2^{64} - 1$	80	<80 (collisions found)	-
SHA-1		160	160 (5×32)	512	$2^{64} - 1$	80	<80 (theoretical attack ^[27] in 2^{61})	192
SHA-2	SHA-224 SHA-256	224 256	256 (8×32)	512	$2^{64} - 1$	64	112 128	139
	SHA-384 SHA-512 SHA-512/224 SHA-512/256	384 512 224 256	512 (8×64)	1024	$2^{128} - 1$	80	192 256 112 128	154

<http://en.wikipedia.org/wiki/SHA-3>



Hash functions

SHA-3

- a subset of the cryptographic primitive family **Keccak**
- designed by Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, Benoît Viguier
- On October 2, 2012, Keccak was selected as the winner of the [NIST hash function competition](http://nist.gov/papers/2012/2012-10-02-keccak-wins-nist-hash-function-competition) (started in 2007)
 - http://ehash.iaik.tugraz.at/wiki/The_SHA-3_Zoo
 - https://en.wikipedia.org/wiki/NIST_hash_function_competition
- is not meant to replace [SHA-2](#), as no significant attack on SHA-2 has been demonstrated
- may make it useful for so-called “embedded” or smart devices
- SHA-3 uses the [sponge construction](#)



Hash functions

SHA-3

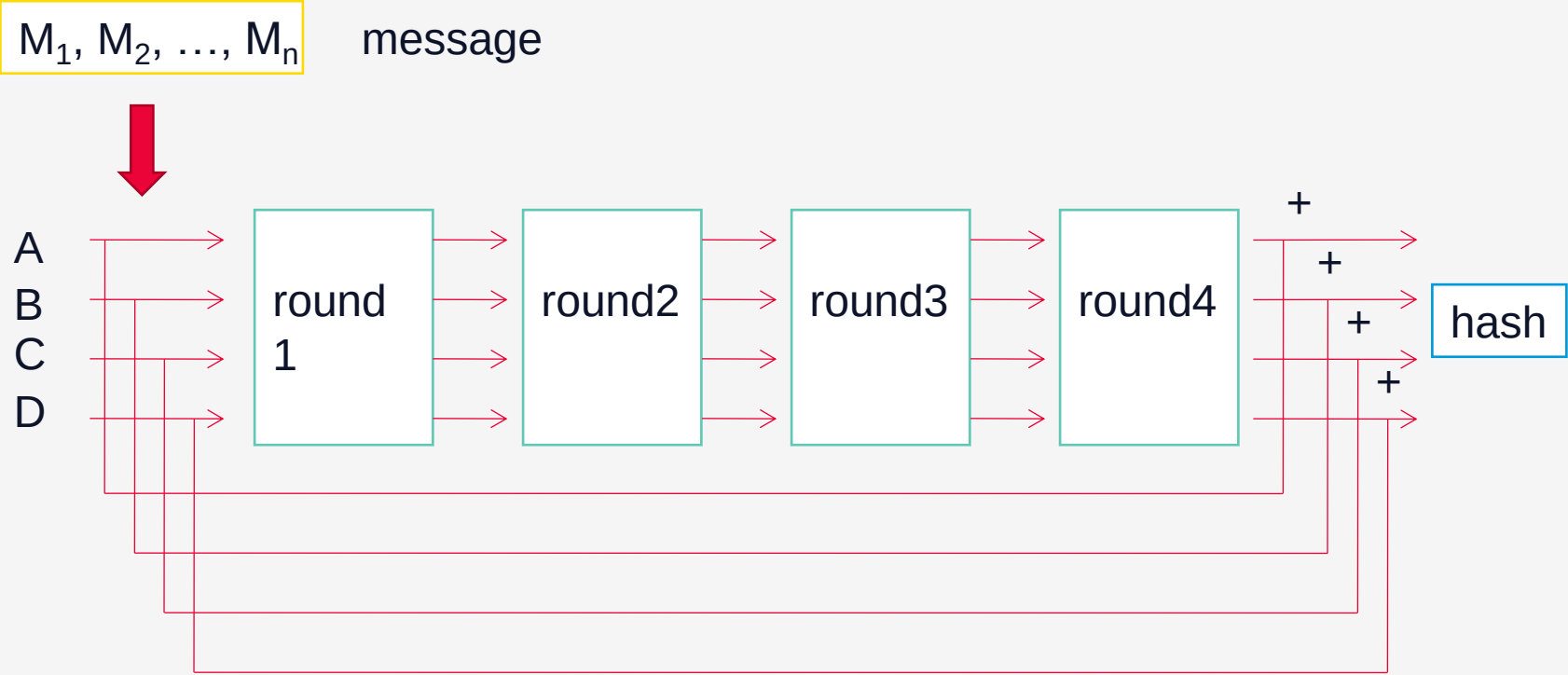
Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Max message size (bits)	Rounds	Operations	Security (bits)	Example Performance (MiB/s) ^[26] ₁
SHA-3	SHA3-224								
	SHA3-256	224		1152				112	
	SHA3-384	256		1088				128	
	SHA3-512	384		832				192	
	SHAKE128	512	1600	576	∞	24	and, xor, not, rot	256	
	SHAKE192	d (arbitrary)	$(5 \times 5 \times 64)$	1344				$\min(d/2, 128)$	
	SHAKE256	d (arbitrary)		1088				$\min(d/2, 256)$	

<http://en.wikipedia.org/wiki/SHA-3>



Hash functions

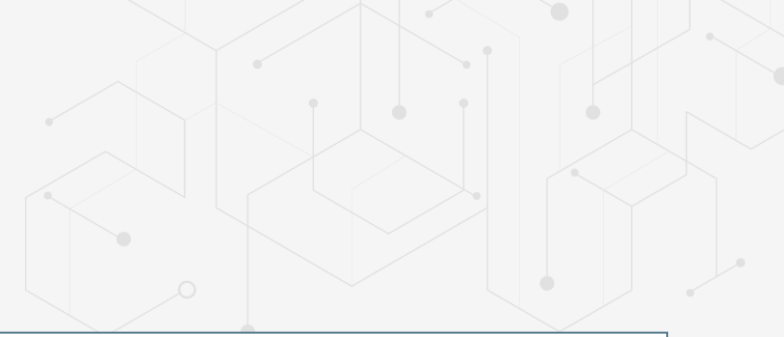
MD5 implementation.....for history



General scheme of MD5 algorithm

Hash functions

MD5 implementation



STAGE 1

- The message M is extended to a length (measured in bits), L , that is congruent with 448 mod 512 ($L - 448 = K * 512$, with k – integer value)
- the padding is made with “1” followed by many “0” bits

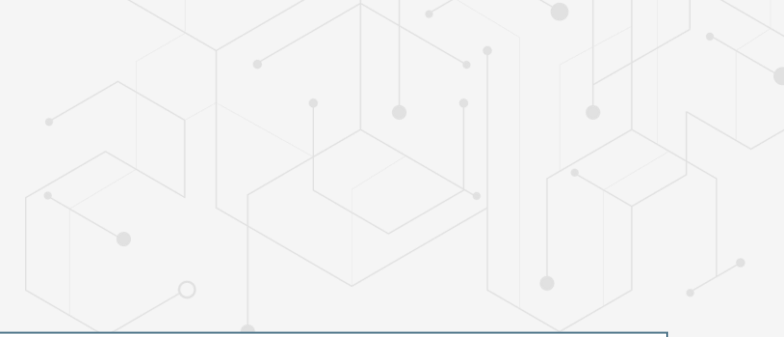
STAGE 2

- There are added 64 bits that represent the length of the initial message
- the message is split in n blocks of 512 bits, $M_1, M_2, \dots, M_n$
- each block M_i has 16 words of 32 bits
- text dimension is now $= n * 16 * 32$;



Hash functions

MD5 implementation



STAGE 3

- to generate the hash it is used a register MD that is 128 long (4 words of 32 bits each – A,B,C,D)
- the initial value MD0 is obtained by concatenating the constants: $h1 = 0x67452301$, $h2 = 0xefcdab89$, $h3 = 0x98badcfe$, $h4 = 0x10325476$.

STAGE 4

- Each block M_j (16 words* 32 bits) is processed in 4 rounds with the functions FF, GG, HH, II
- $MD_j = MD_{j-1} + II(M_j, HH(M_j, GG(M_j, FF(M_j, MD_{j-1}))))$
- each round has 16 steps



Hash functions

MD5 implementation

in each step of the round (it has 16):

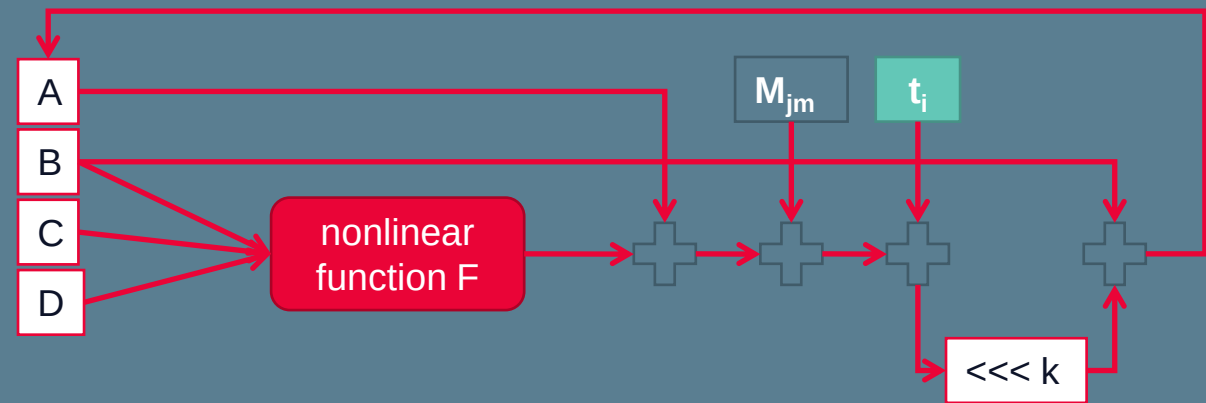
$$A = B + ((A + F(B, C, D) + M_{jm} + t_i) \lll k)$$

t_i – constant value, step dependent, equal with the first 32 bits of the value $\text{abs}(\sin(j+1))$, $0 \leq j \leq 63$

M_{jm} – the m th 32 bit word from the M_j block

F – nonlinear function that is modified in each round

$\lll k$ – shifts to the left the word with k positions



One round in MD5

Hash functions

DO's and DON'TS

USED FOR

- Allow storing passwords and hiding their plaintext value
- Use by PRNG to generate random numbers (hide the initial seed value)
- Used to validate files integrity or verify files versions (ex Git is using sha-256)
- Used to detect modifications – Intrusion Detection Software (IDS)
- Extend/Reduce secret values to a predefined size
- Used by Version Control Systems (like Git) to check for changes
- Used by blockchains as a *Proof of Work*

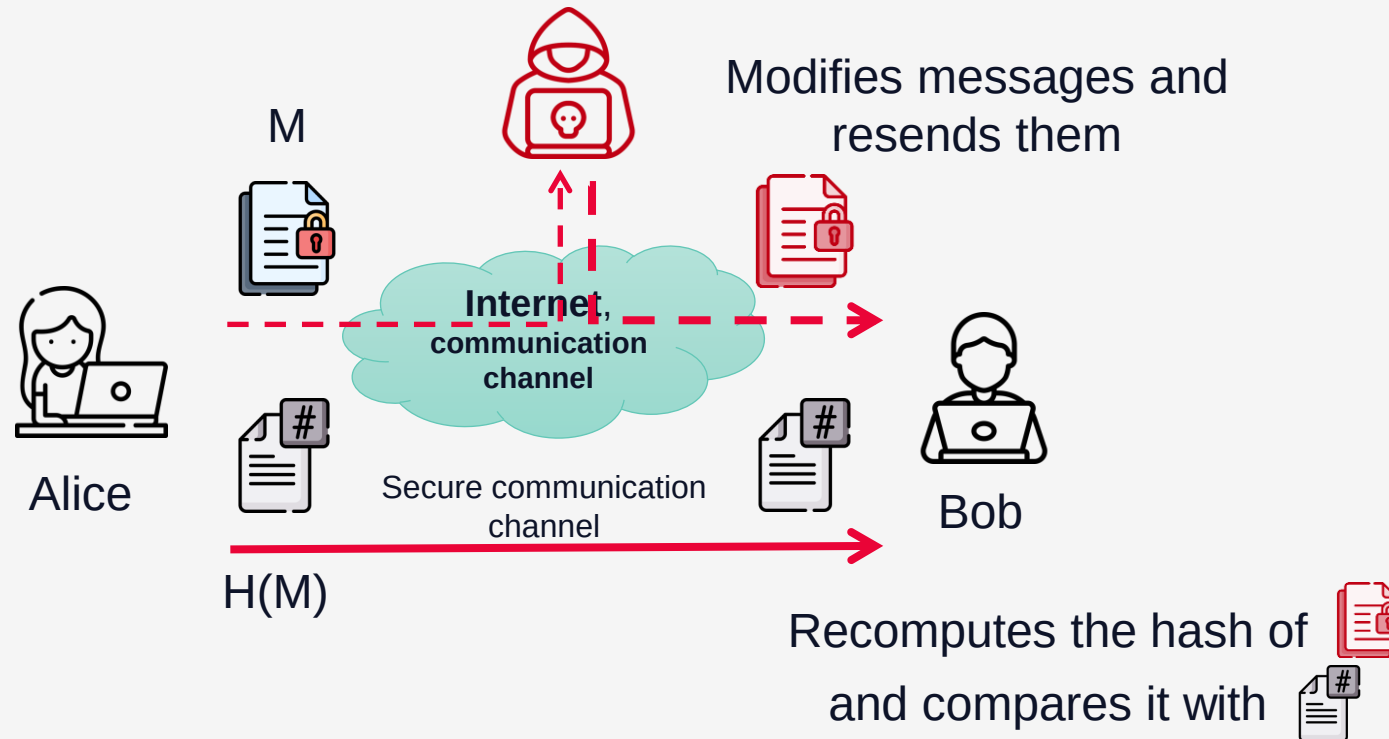


NOT USED FOR

- Store passwords without applying a salt value
- Replace Message Authentication Code
- Generate cryptographic algorithms
- Double the size of the message digest by concatenating two message digests of slightly different messages
- Concatenate two message digests from different hashes

Hash functions

Use case for Message Integrity

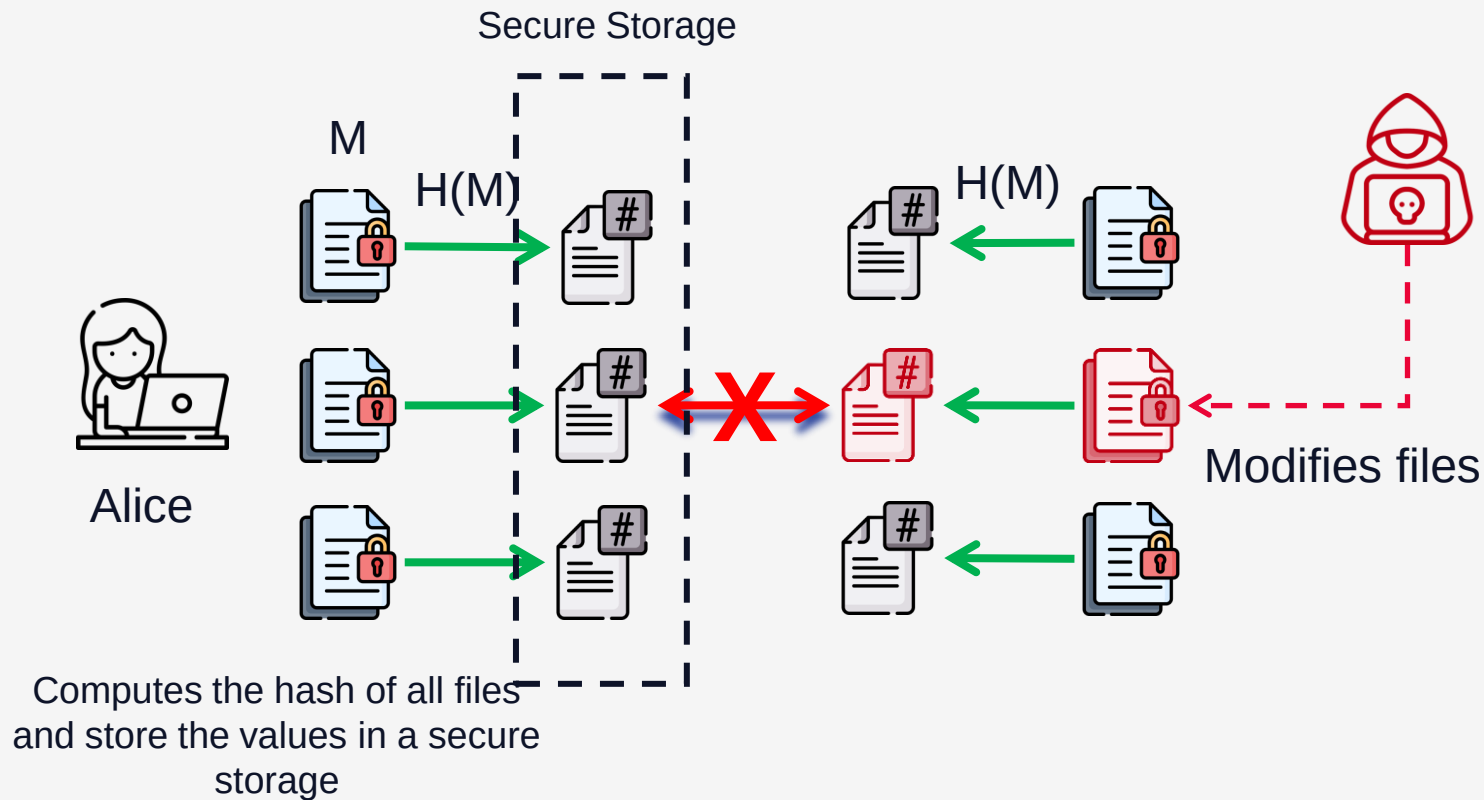


Man-in-the-Middle Mitigation

- Attacker intercepts and alters the communication between two parties.
- Alice sends on a different channel (not controlled by the attacker) the hash value of the message
- Bob can verify the integrity of the received message by recomputing the hash value and compare it with the one received from Alice
- If the hash value is different, it means the message is different
- **If the attacker controls the hash transmission channel**, then it can recompute the hash value for the altered message and Bob will receive the new value (will not know the message was altered)

Hash functions

Use case for Intrusion Detection Systems

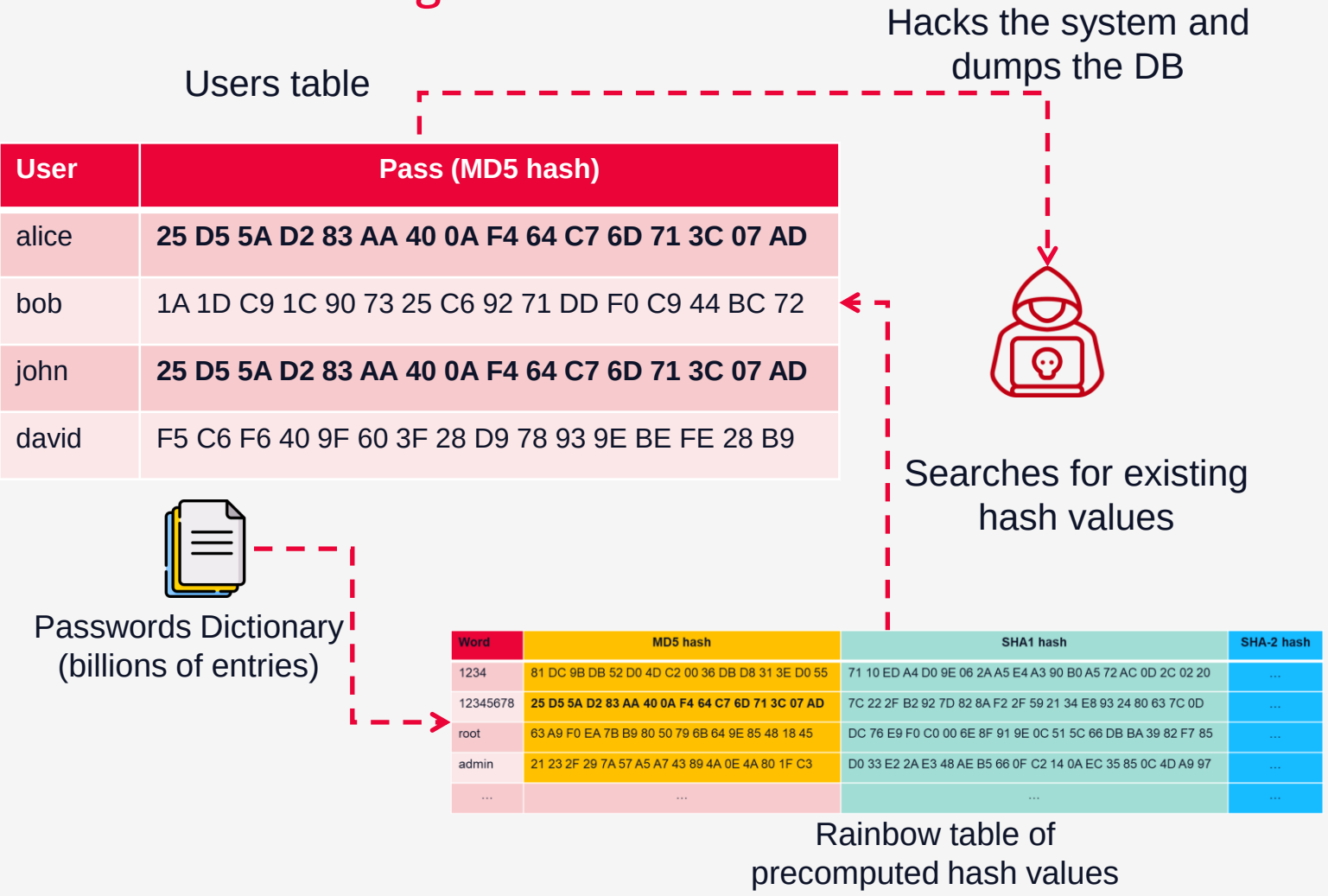


Intrusion Detection

- Alice computes the hash value for all the files and store the values in a secure place
- Attacker accesses the system and modify files.
- Alice recomputes the has values of all files and check them against the initial values. If the hash is different, it means someone altered it
- **If the attacker can access the secure storage, it can change the hash value of the altered files and hide its attack**
- Git works in similar way

Hash functions

Use case for Storing Passwords



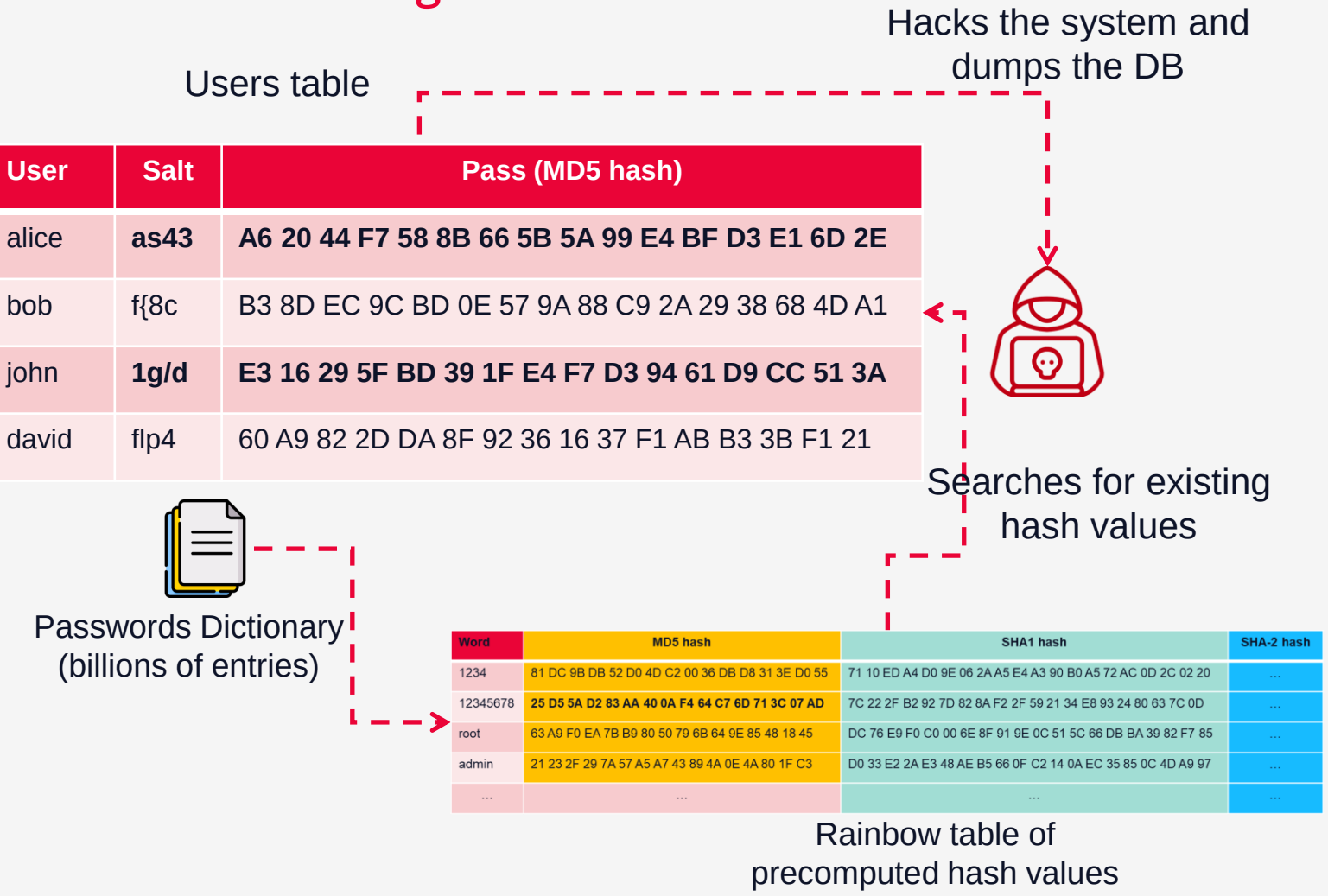
Hacking Common Passwords

- Passwords are stored as hash values in the Users table
- Attacker hacks the system and dumps the users DB.
- Attacker has a huge dictionary of common passwords
- In advance of the attack, generates a Rainbow table of all common passwords for all known hash functions (MD5, SHA1, etc.)
- Attacker checks if any of the Rainbow table hash values are present in the users DB
- If it finds matches, then determines the password instantly
- Is faster if it sorts the user table by the hash values (same hash value it means same password)



Hash functions

Use case for Storing Passwords with Salt



Protecting Common Passwords

- For each user, a random salt is generated
- Passwords are stored as hash(Password + Salt) values in the Users table
- Attacker hacks the system and dumps the users DB.
- Attacker has a huge dictionary of common passwords
- Has the Rainbow table of all common passwords for all known hash functions (MD5, SHA1, etc.)
- The existing rainbow table is useless as it does not consider the salt
- Attacker needs to regenerate the Rainbow table for each salt – you gain time



Hash functions

Use case for Storing Passwords with salt

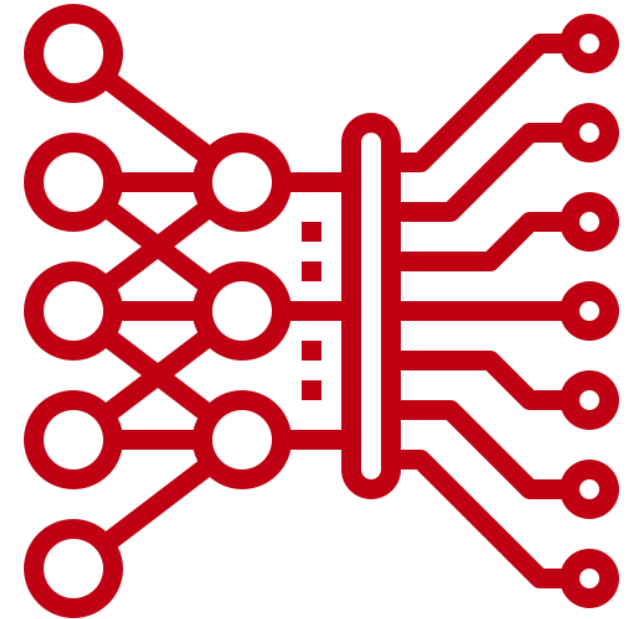
- The salt value is a predefined value / **randomly generated value** for each stored password.
- Using the same salt for all entries will not hide all the identical passwords (they will still have the same hash). A lot of users are still using common passwords like '12345678'
- The salt value **IS NOT A SECRET**. No need to protect it
- The role of the salt value is to delay the attack by breaking the Rainbow table (existing Rainbow tables are useless as they didn't consider the salt values)
- The attacker must regenerate the Rainbow table and for billions of common passwords will take some time
- Having different salt values for each stored password, the attacker must regenerate the Rainbow table for each account. For the entire database this will take a lot as he can't reuse previous Rainbow tables
- Users with common passwords will be eventually hacked but the defender gains a lot of time (enough time to notify everyone to change their passwords)
- The Rainbow table attack can be made delayed even more (useless eventually) by introducing a computational effort required to compute the hash value. See **Password Based Key Derivation Functions (PBKDF)**. Example: 1 additional millisecond for a 10 billion dictionary requires 115 days more to regenerate the hash values



Hash functions

Password-Based Key Derivation Function (PBKDF)

- a cryptographic function that is designed to generate a cryptographic key from a user's password. PBKDF is widely used to securely store passwords and derive keys for encryption
- **Input:** It takes a password, a salt (a random value), and a number of iterations as inputs.
- **Iterations:** The function applies a hash function (like SHA-256) multiple times (e.g., thousands or millions of iterations) to make the process of generating the key computationally expensive.
- **Salt:** The salt ensures that even identical passwords result in different hash outputs, preventing attacks like rainbow table attacks.
- **Output:** Produces a derived key, with any required size, which can be used for cryptographic purposes like encryption or HMAC generation.
- <https://en.wikipedia.org/wiki/PBKDF2>



Hash functions

Forensics

- For **digital forensics** tasks such as detecting file changes, modifications, or other alterations in a system are important as allows professionals to find fast “normal” and “suspicious” files. They are based on hashes, by comparing known values with the ones computed from the target system.
- Known tools
 - Autopsy and The Sleuth Kit (TSK), <https://www.sleuthkit.org/index.php>
 - HashDeep and md5deep, <https://www.kali.org/tools/hashdeep/> , <https://github.com/jessek/hashdeep>
 - Chkrootkit, <https://www.chkrootkit.org/>
 - Tools from Linux (e.g., diff, sha256sum)
- Known hashes databases
 - National Software Reference Library (NSRL), <https://www.nist.gov/itl/ssd/software-quality-group/national-software-reference-library-nsrl>
 - Autopsy NSRL, <https://sourceforge.net/projects/autopsy/files/NSRL/>



Hash functions

Crypto

Hash functions play a crucial role in various aspects of **cryptography** and are fundamental to the security and integrity of many blockchain and cryptocurrency systems.

- **Data Integrity and Verification:** used to verify that data (e.g., transactions) has not been altered
- **Mining in Proof of Work (PoW) Consensus:** miners compete to solve a cryptographic puzzle by finding a hash value that meets specific criteria (e.g., a certain number of leading zeros)
- **Address Generation:** A user's **public key** is hashed to create a shorter, more manageable address
- **Digital Signatures:** are used to authenticate transactions
- **Merkle Trees and Block Validation:** secure large set of transactions
- **Key Derivation and Password Hashing:** help derive private keys from a user's password, making it more difficult for attackers to brute-force the wallet key
- **Proof of Stake (PoS) and Delegated Proof of Stake (DPoS):** hash functions help determine randomly selected validators to create new blocks or validate transactions



Hash functions

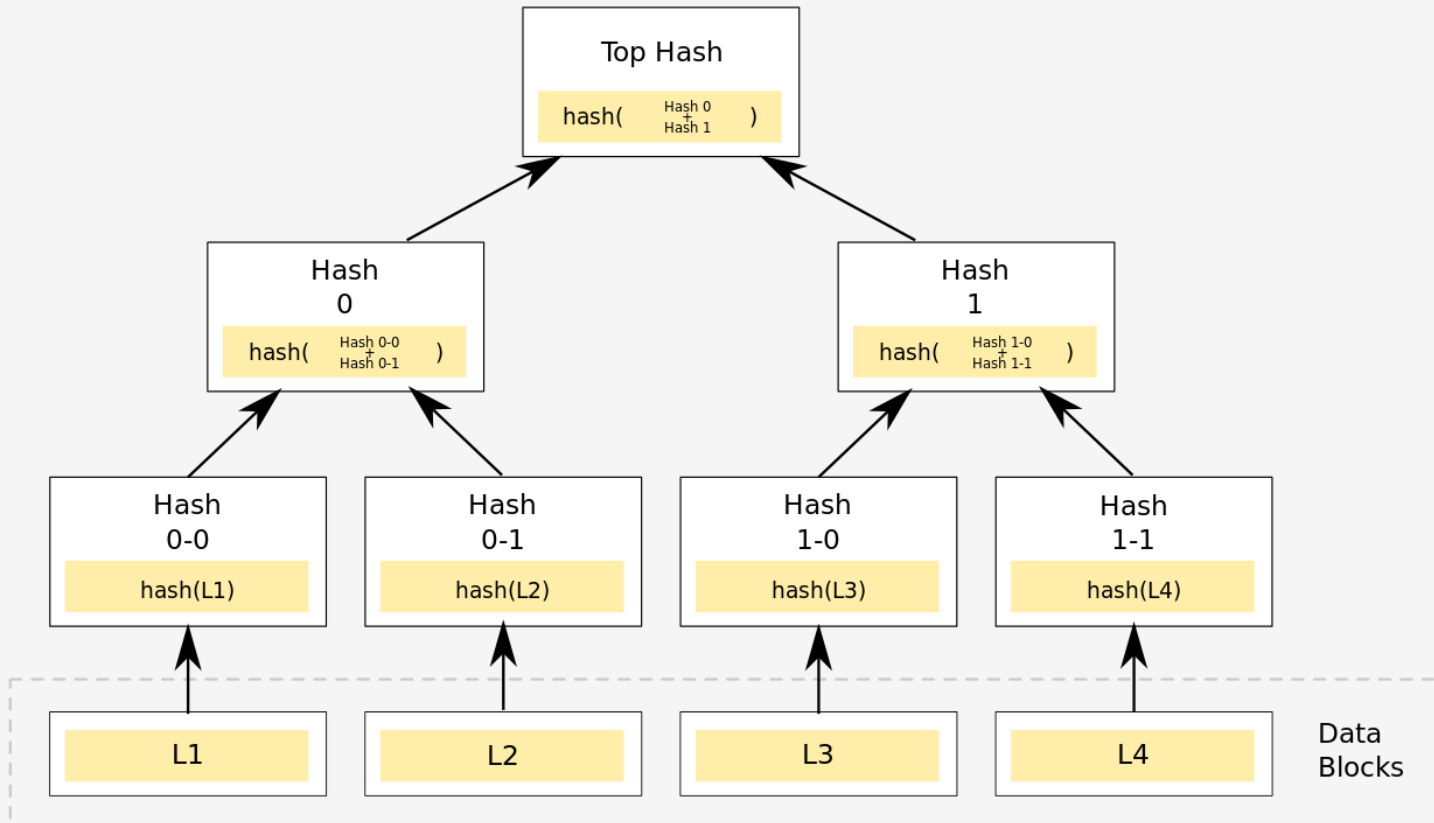
Crypto



Purpose	Description	Examples
Data Integrity and Verification	Verifies that data (e.g., transactions) has not been altered.	Bitcoin blockchain structure, SHA-256 for Bitcoin, Keccak-256 for Ethereum
Mining in PoW	Miners solve cryptographic puzzles to validate blocks by finding a hash with specific properties.	Bitcoin, Litecoin
Address Generation	Creates unique addresses by hashing public keys.	Bitcoin address creation using SHA-256 and RIPEMD-160
Digital Signatures	Uses hashes to sign transactions securely.	ECDSA in Bitcoin
Merkle Trees	Efficiently verifies transaction inclusion in blocks.	Merkle trees in Bitcoin
Key Derivation	Secures password-based keys using multiple hash iterations.	Wallets using PBKDF2 or Argon2
Random Selection in PoS	Uses hashing for random validator selection.	Ethereum 2.0

Hash functions

Merkle Trees

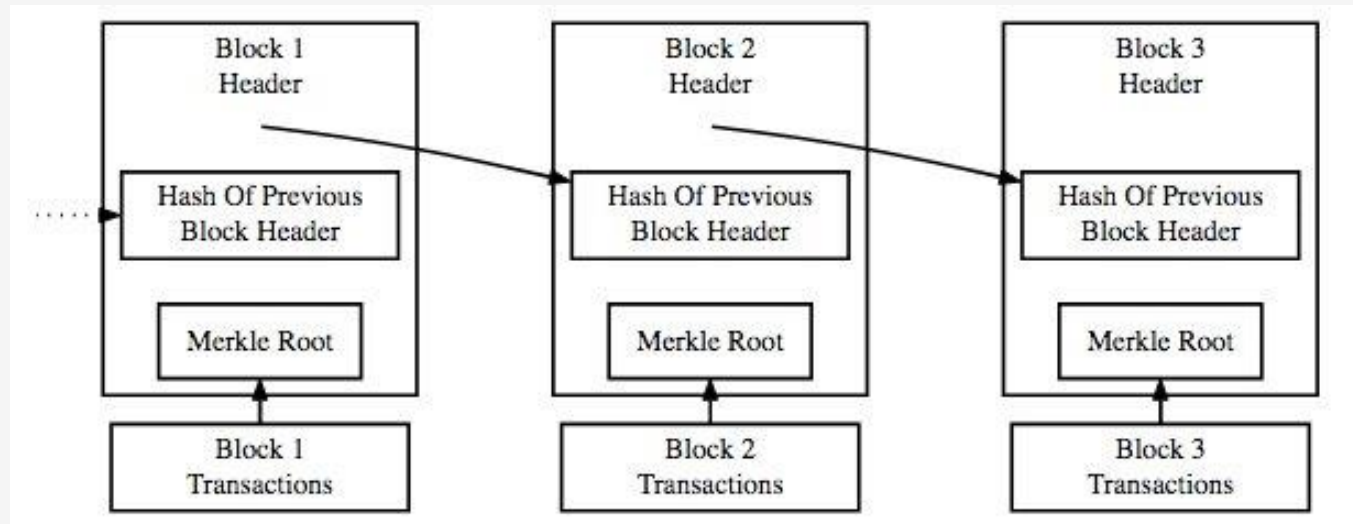


A hash tree or Merkle tree is a tree in which every leaf node is labelled with the cryptographic hash of a data block, and every non-leaf node is labelled with the cryptographic hash of the labels of its child nodes,
https://en.wikipedia.org/wiki/Merkle_tree

- Used to verify integrity of data
- NOT used for searching
- Are efficient to check data integrity because you don't need the entire tree to verify a block

Hash functions

Crypto Block Chain Overview



Example of Hash functions:

- Bitcoin uses SHA 256
- Ethereum uses Keccak-256

Protect the chain for future changes: all the transactions and blocks are hashed and chained together

Hash functions

Proof of Work vs Proof of Stake



Feature	Proof of Work (PoW)	Proof of Stake (PoS)
Mechanism	Mining: Solve computational puzzles	Staking: Validators chosen based on stake amount
Energy Usage	High (energy-intensive mining)	Low (no need for mining hardware)
Security Model	Based on computational power	Based on economic incentives
Attack Costs	51% attack requires controlling mining power	51% attack requires controlling stake amount
Environmental Impact	High carbon footprint	Environmentally friendly
Rewards	Block rewards and transaction fees to miners	Rewards in transaction fees or new coins to validators
Decentralization	More decentralized in theory but prone to mining pools	Potential for centralization if large holders dominate
Examples	Bitcoin (BTC), Ethereum (ETH) (until it moved to PoS in 2022), Litecoin (LTC)	Ethereum 2.0 (ETH), Cardano (ADA), Polkadot (DOT), Solana (SOL)

Hash functions

Bitcoin Blockchain Hash

version	02000000
previous block hash (reversed)	17975b97c18ed1f7e255adf297599b55 330edab87803c817010000000000000
Merkle root (reversed)	8a97295a2747b4f1a0b3948df3990344 c0e19fa6b2b92b3a19c8e6badc141787
timestamp	358b0553
bits	535f0119
nonce	48750833
transaction count	63
coinbase transaction	
transaction	
...	



Block hash

```
0000000000000000  
e067a478024addfe  
cdc93628978aa52d  
91fabd4292982a50
```

Is impossible to modify transactions included in any block without modifying all subsequent blocks

The *proof of work* takes advantage of the apparently random nature of cryptographic hashes: hashing random data produces an apparently random hash value and is very difficult to predict the output

<http://www.righto.com/2014/02/bitcoin-mining-hard-way-algorithms.html>



Hash functions

Bitcoin Blockchain Hash

version	02000000
previous block hash (reversed)	17975b97c18ed1f7e255adf297599b55 330edab87803c817010000000000000
Merkle root (reversed)	8a97295a2747b4f1a0b3948df3990344 c0e19fa6b2b92b3a19c8e6badc141787
timestamp	358b0553
bits	535f0119
nonce	48750833
transaction count	63
coinbase transaction	
transaction	
...	



Block hash

```
0000000000000000  
e067a478024addfe  
cdc93628978aa52d  
91fabd4292982a50
```

1. The hash of the contents of the new block is computed
2. A nonce (random string) is appended to the hash
3. The new value is hashed
4. The new hash is compared to the difficulty level (a 64-character string value = 256 bits) and checked whether is less than that or not:
 1. If not, then the nonce is changed and the process repeats again
 2. If yes, then the block is added to the chain and the public ledger is updated

<http://www.righto.com/2014/02/bitcoin-mining-hard-way-algorithms.html>



Hash functions

Bitcoin Blockchain Hash

- A new block is added to the block chain if its hash is at least as challenging as a difficulty (starts with a given number of 0s)
- Every 2,016 blocks, the network uses timestamps stored in each block header to calculate the number of seconds elapsed between generation of the first and last of those last 2,016 blocks. The ideal value is 1,209,600 seconds ~ two weeks (around 10 minutes between each new block)
 - For less than 2 weeks, the expected difficulty value is increased proportionally (by as much as 300%) so that the next 2,016 blocks should take exactly two weeks to generate if hashes are checked at the same rate.
 - For more than two weeks to generate the blocks, the expected difficulty value is decreased proportionally (by as much as 75%) for the same reason.
- https://developer.bitcoin.org/devguide/block_chain.html



Message Authentication Codes (MAC)

Definition

- is a key-dependent one-way hash function
- a one-way hash function generates a MAC if the hash is encrypted with a symmetric algorithm
- used **to authenticate** files between users
- used to provide **data integrity**
- requires a secret key known only by the communication parties
- **NOT used** for securing data

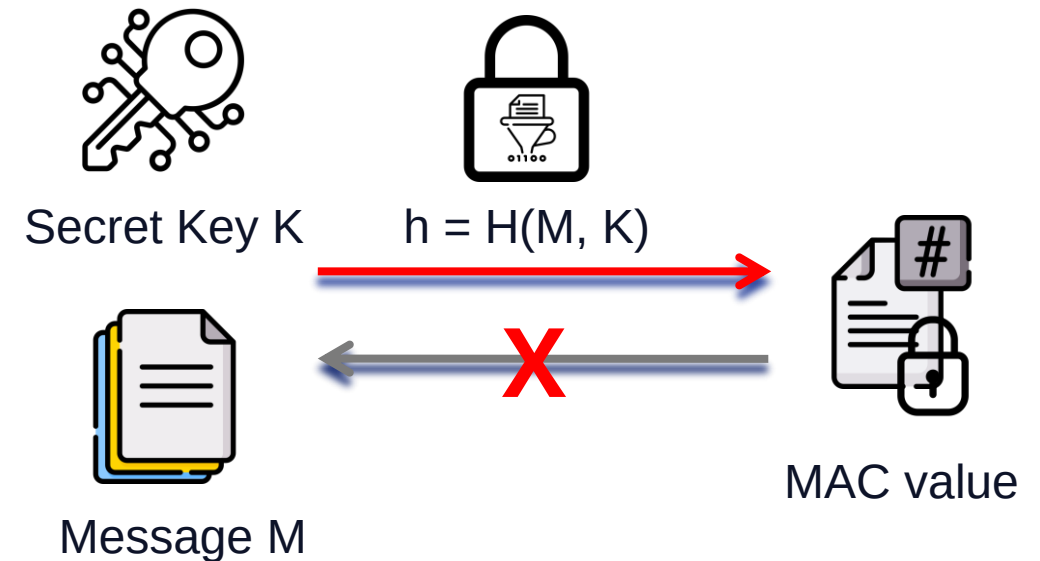


Message Authentication Codes (MAC)

- has the same properties of a hash function
- the function requires a secret key
- is generated by taking a message and a secret key as inputs and applying a MAC algorithm
 - Hash-based MAC (HMAC)
 - Cipher Block Chaining MAC (CBC-MAC)
 - Cipher-based MAC (CMAC)
- The secret key prevents the attacker to recompute the MAC value
- The MAC value should be long enough to prevent guessing

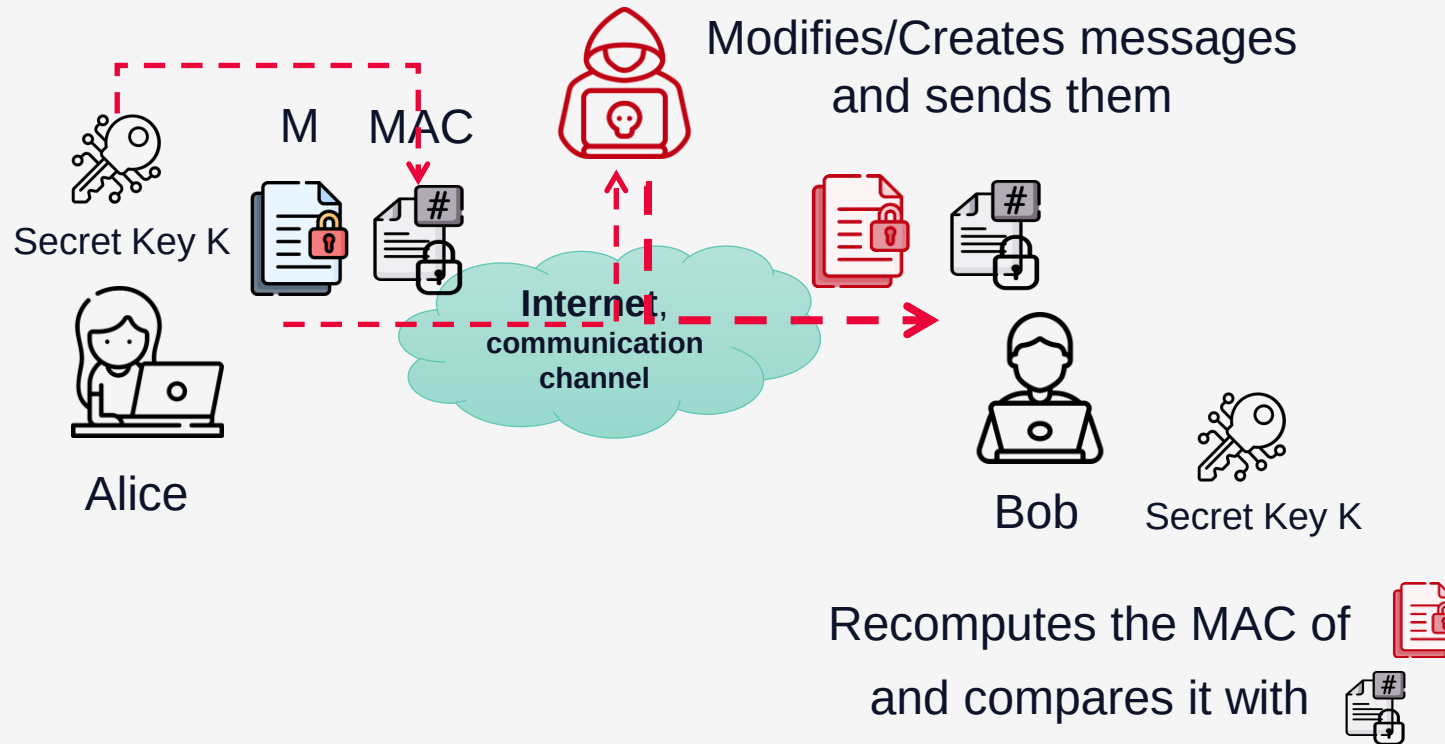
MD5("a") = "0C C1 75 B9 C0 F1 B6 A8 31 C3 99 E2 69 77 26 61"

MD5 MAC("a","secret") = B0 EB A8 AE E3 FB 9B 23 A5 F8 77 96 59 D3 2F 23



Message Authentication Codes

Use case for Message Integrity and Data Authentication

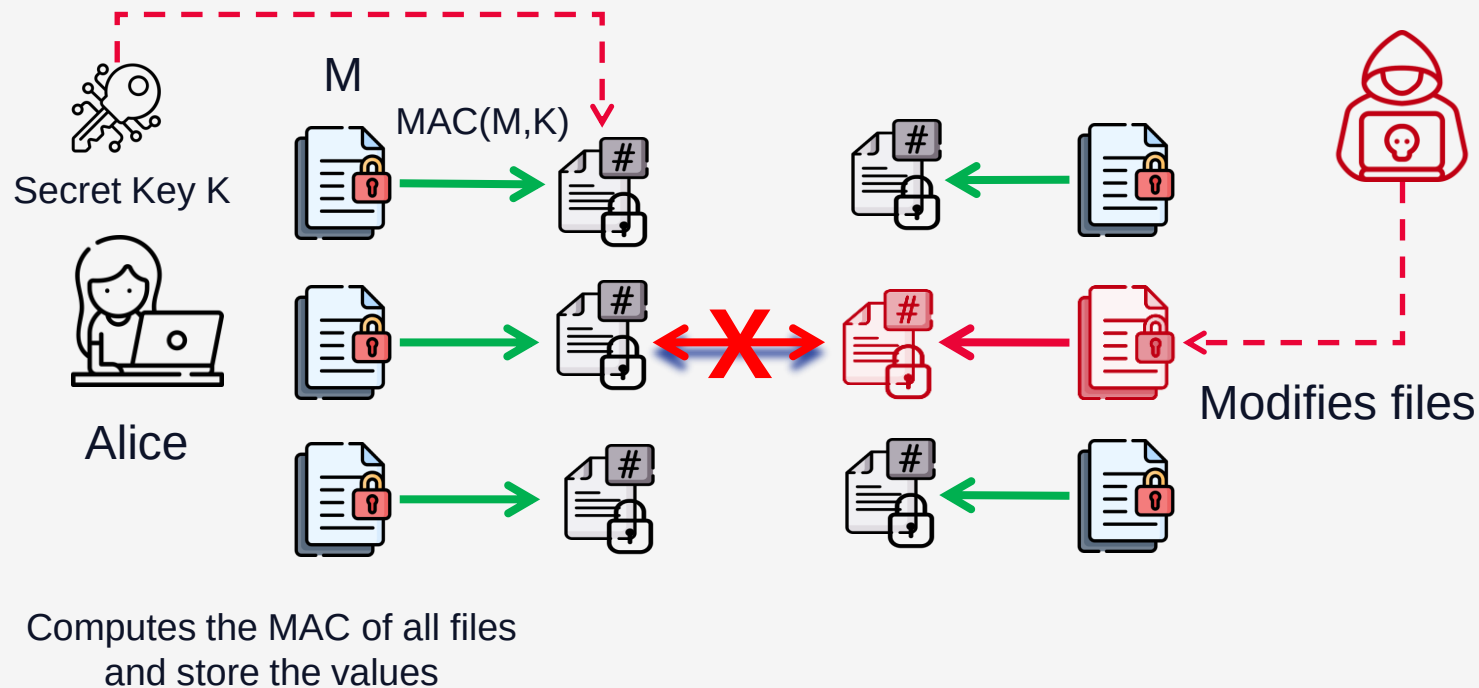


Man-in-the-Middle Mitigation

- Attacker intercepts and alters the communication between two parties.
- Alice and Bob share a secret key **K**
- Alice sends along the message its MAC value
- Bob can verify the integrity of the received message by recomputing the MAC value and compare it with the one received from Alice
- If the MAC value is different, it means the message is different
- **The attacker does not have the secret key and therefore it can't recompute it for altered/new messages**

Message Authentication Codes

Use case for Intrusion Detection Systems



Intrusion Detection

- Alice has a secret key
- Alice computes the MAC value for all the files and store the values (not needed a secure storage)
- Attacker accesses the system and modify files.
- Alice recomputes the MAC values of all files and check them against the initial values. If the MAC is different, it means someone altered it
- **The attacker does not have the secret key and he can't recompute a valid MAC**

Message Authentication Codes

Hash-based MAC (HMAC)

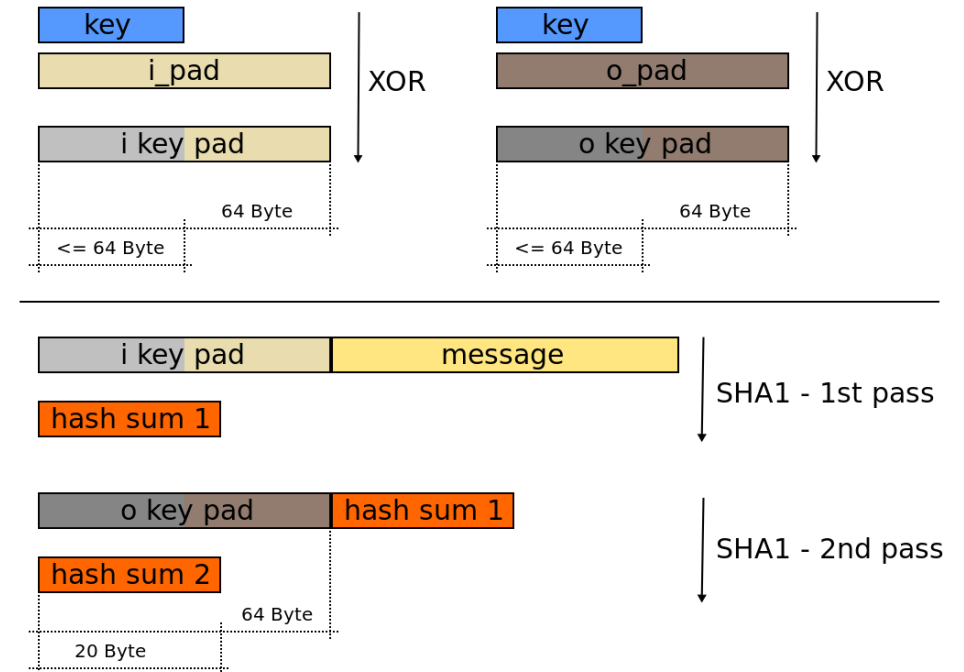
- HMAC - **keyed-hash message authentication code**
- uses a cryptographic hash function (SHA-1, MD5, etc.) in combination with a secret cryptographic key
- used to simultaneously verify both the *data integrity* and the *authentication of a message*
- first published in 1996 by Mihir Bellare, Ran Canetti, and Hugo Krawczyk, who also wrote RFC 2104
- HMAC-SHA1 and HMAC-MD5 are used within the IPsec and TLS protocols
- Internet protocols: SSL, IPSec, SS
- <https://en.wikipedia.org/wiki/HMAC>



Message Authentication Codes

HMAC – RFC 2104

- H is a cryptographic hash function,
- K is a secret key **padded** to the right with extra zeros to the input block size of the hash function, or the hash of the original key if it's longer than that block size,
- m is the message to be authenticated,
- $|$ denotes **concatenation**
- $\oplus$ denotes **exclusive or** (XOR),
- **opad** is the outer padding (0x5c5c5c...5c5c, one-block-long **hexadecimal** constant),
- **ipad** is the inner padding (0x363636...3636, one-block-long **hexadecimal** constant).

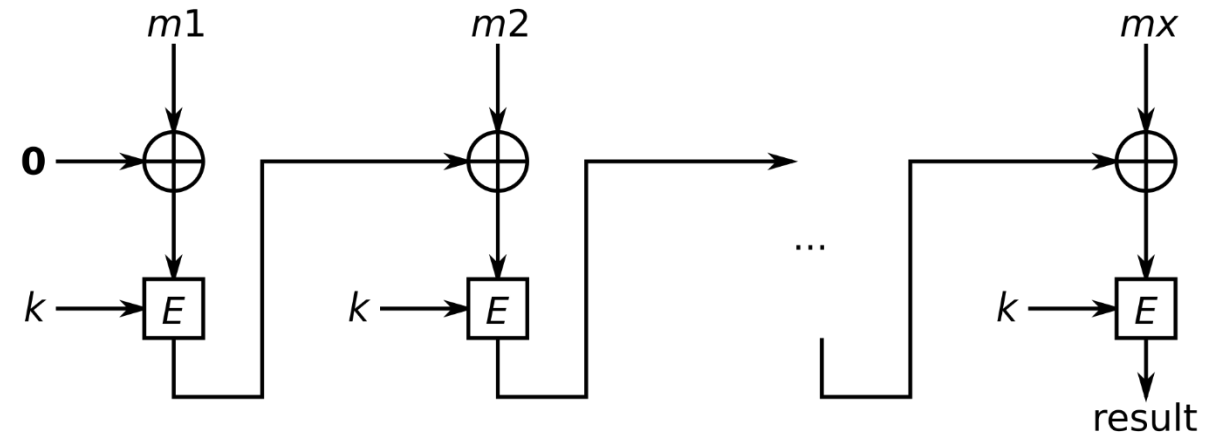


http://en.wikipedia.org/wiki/Hash-based_message_authentication_code

Message Authentication Codes

Cipher Block Chaining MAC (CBC-MAC)

- [CBC-MAC](#) (banking – ANSI X9.9, X9.19, FIPS 186-3)
- purpose is to create a chain of blocks such that each block depends on the proper encryption of the previous block



<https://en.wikipedia.org/wiki/CBC-MAC>

Message Authentication Codes MAC

Attacks on a MAC

- Chosen message attack – the attacker gets the tag for some messages
- Existential forgery – to generate some new valid message, tag pairs
- Alternatives without encryption are not safer:
 - $MAC = H(key \parallel message)$ - with most hash functions, it is easy to append data to the message without knowing the key and obtain another valid MAC (length-extension attack).
 - $MAC = H(message \parallel key)$, an attacker who can find a collision in the (unkeyed) hash function has a collision in the MAC (as two messages m_1 and m_2 yielding the same hash will provide the same start condition to the hash function before the appended key is hashed, hence the final hash will be the same).
 - $MAC = H(key \parallel message \parallel key)$ is better, but not

Conditions

- Attacker cannot generate a valid tag
- Given (M_1, tag) attacker CANNOT generate (M, tag') for $tag \neq tag'$



Message Authentication Codes MAC

Length-extension attack

- A **length-extension attack** is a cryptographic attack that exploits a specific property of some **hash functions** used in **Message Authentication Codes (MACs)**.
- It typically applies to hash functions that use the **Merkle-Damgård construction**, like **MD5**, **SHA-1**, or **SHA-256**.
- This attack allows an attacker to **extend** a message for which they have a valid hash without knowing the secret key used in the MAC
- A simpler MAC construction that directly uses a hash function can be vulnerable if it uses a construction like $\text{MAC} = H(\text{secret} || \text{message})$.
- Due to how the Merkle-Damgård construction works, if an attacker knows $H(\text{secret} || \text{message})$, they can compute $H(\text{secret} || \text{message} || \text{padding} || \text{additional_data})$ by using the internal state of the hash function at the end of $H(\text{secret} || \text{message})$ as the starting point.
- Padding is typically required by hash functions to make the total length a multiple of the block size, and the attacker must correctly simulate this padding for the length-extension attack to work.



Message Authentication Codes MAC

Length-extension attack

Step 1: Original Message and MAC

- Suppose message = "data".
- The MAC is computed as $MAC = H(secret \parallel "data")$.
- The attacker knows the message and MAC but does not know secret.

Step 2: Length-Extension Attack

- The attacker aims to compute a valid MAC for "data $\parallel$ additional_data", such as "data $\parallel$ moredata".
- Using the known value of $H(secret \parallel "data")$, the attacker can extend it with "moredata" and calculate $H(secret \parallel "data \parallel padding \parallel moredata")$.
- The attacker then produces a valid MAC for "data $\parallel$ moredata" without needing to know secret.



Message Authentication Codes MAC

Length-extension attack

- This attack leverages the internal state of the hash function after processing the original message.
- Hash functions like MD5, SHA-1, and SHA-256 process data in fixed-size blocks. When a hash function is used to create a MAC in an insecure manner ($H(\text{secret} \parallel \text{message})$), it exposes this intermediate state, allowing attackers to extend the message by treating it as if they were simply appending to the existing data.



Message Authentication Codes MAC

Length-extension attack - Mitigations

- **Use HMAC Instead of Plain Hashing:**
 - HMAC (Hash-based Message Authentication Code) construction is designed to be resistant to length-extension attacks.
 - It uses a construction like $\text{HMAC} = H(\text{outer_pad} || H(\text{inner_pad} || (\text{key} || \text{message})))$, which ensures that the internal state of the hash function cannot be manipulated by attackers.
- **Avoid Insecure Constructions:**
 - Never use a simple $H(\text{secret} || \text{message})$ construction for MACs if the hash function is susceptible to length-extension attacks.
- **Use SHA-3 or Other Sponge Construction Hashes:**
 - SHA-3 and other sponge-based hash functions do not have the same vulnerability because of their internal structure, making them inherently resistant to length-extension attacks.

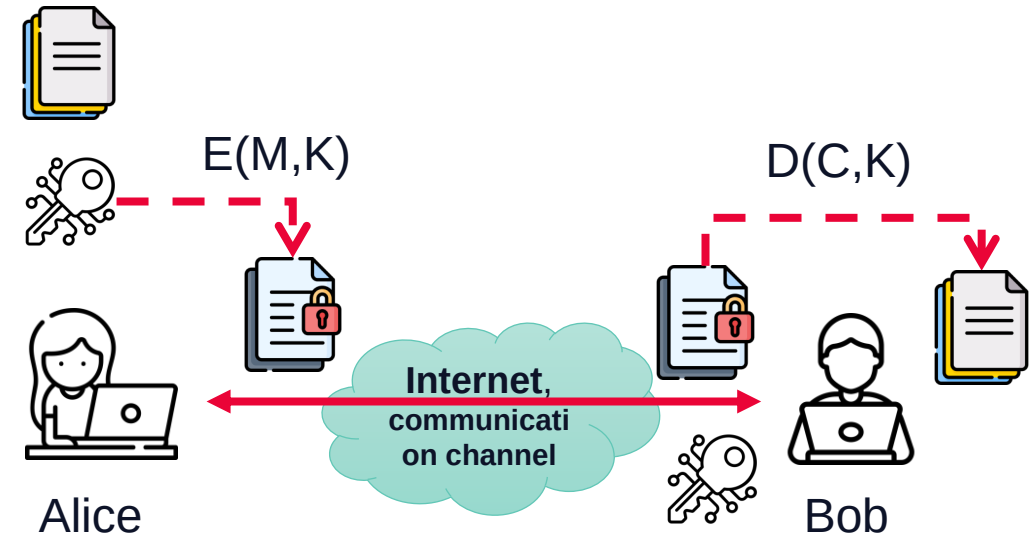


4. Symmetric Cryptographic Systems



Symmetric Cryptographic Systems

- a.k.a conventional / private-key / single-key
- the only cryptographic solution prior to 1970 (public-key)
- encryption key has the same value as the decryption one
- both source and destination know the key
- the key must be protected
- implemented by algorithms that use transposition and substitution
- the most used type of encryption
- Requires:
 - A secret key
 - A strong encryption algorithm
 - A secure way to distribute the key



Symmetric Cryptographic Systems

Don't forget

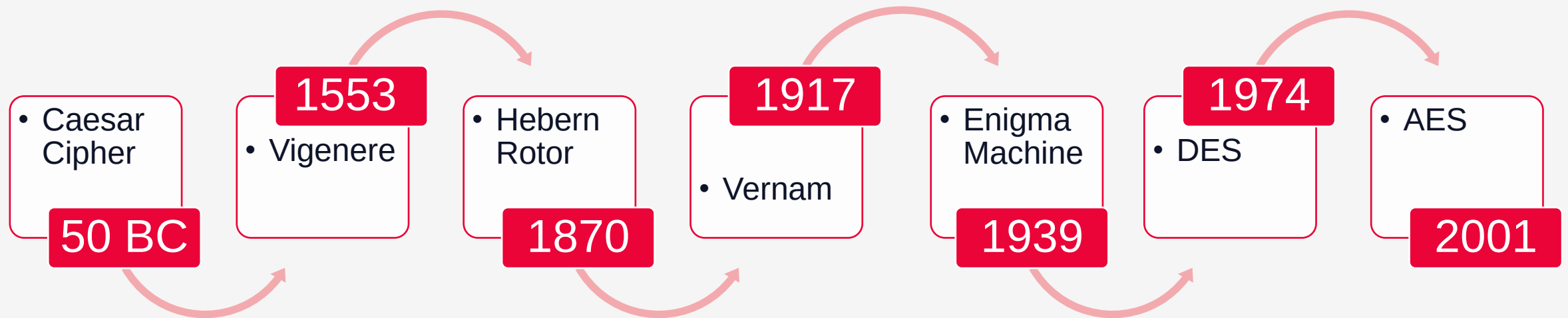
Encryption algorithm is publicly
known !

DO NOT USE proprietary
solutions !



Symmetric Cryptographic Systems

History



Symmetric Cryptographic Systems

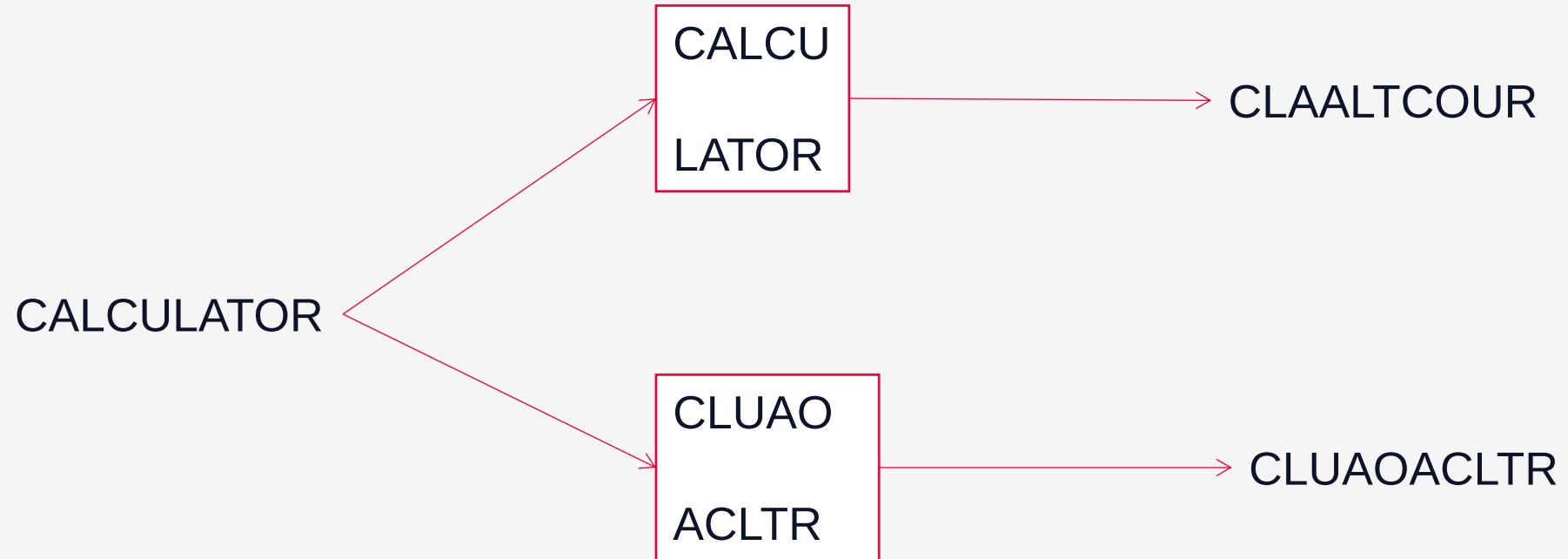
Transposition ciphers

- change the position of the plaintext characters
- there are changed blocks of chars or the entire message
- the encryption key, $K = (d, f)$, where d represent the length of consecutive char blocks that will be enciphered accordingly to the permutation, f
- The alphabet of the clear text remains unchanged
- By the number of transpositions:
 - Mono-phase
 - Multi-phase
- By the target element:
 - Monographic – for chars
 - Multigraphic – for groups of characters



Symmetric Cryptographic Systems

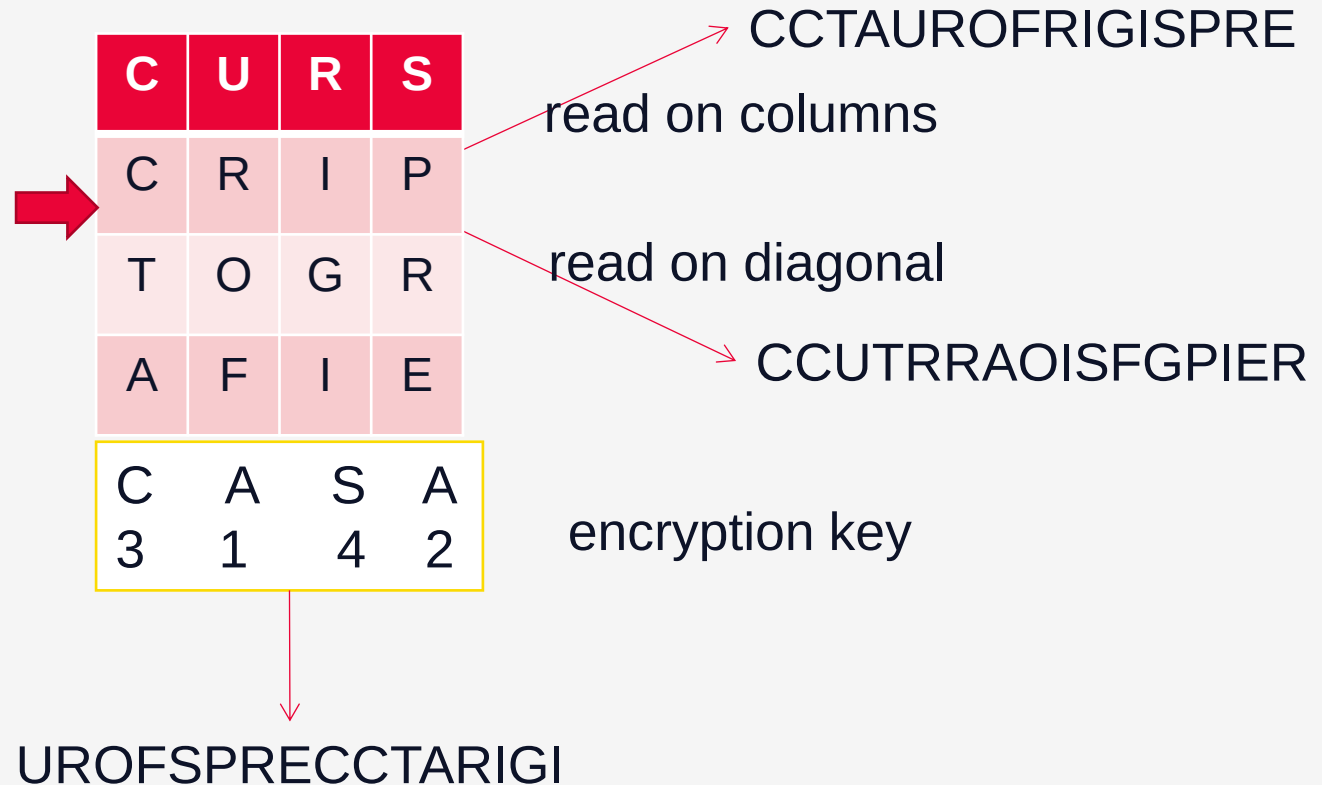
Monographic Transposition cipher



Symametric Cryptographic Systems

Monographic Transposition cipher

CURS CRIPTOGRAFIE



Symmetric Cryptographic Systems

Transposition cipher

Scytale

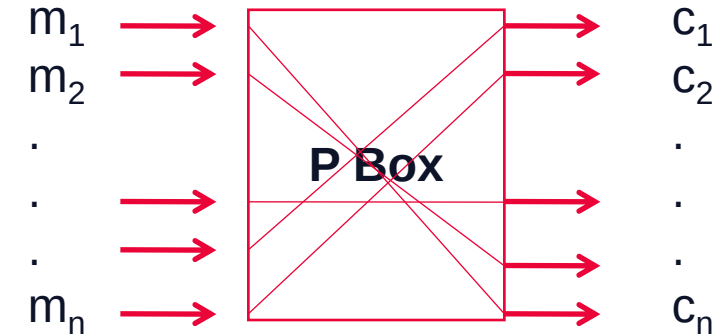
- is a tool used to perform a transposition cipher, consisting of a cylinder with a strip of parchment around it on which is written a message.
- The recipient uses a rod of the same diameter on which the parchment is wrapped to read the message.
- Used by ancient Greek in military campaigns
- <https://en.wikipedia.org/wiki/Scytale>



Symmetric Cryptographic Systems

Transposition cipher

- Easy to implement
- Vulnerable to statistic attacks (character frequency remains the same)
- **Transpositions are implemented by P boxes**



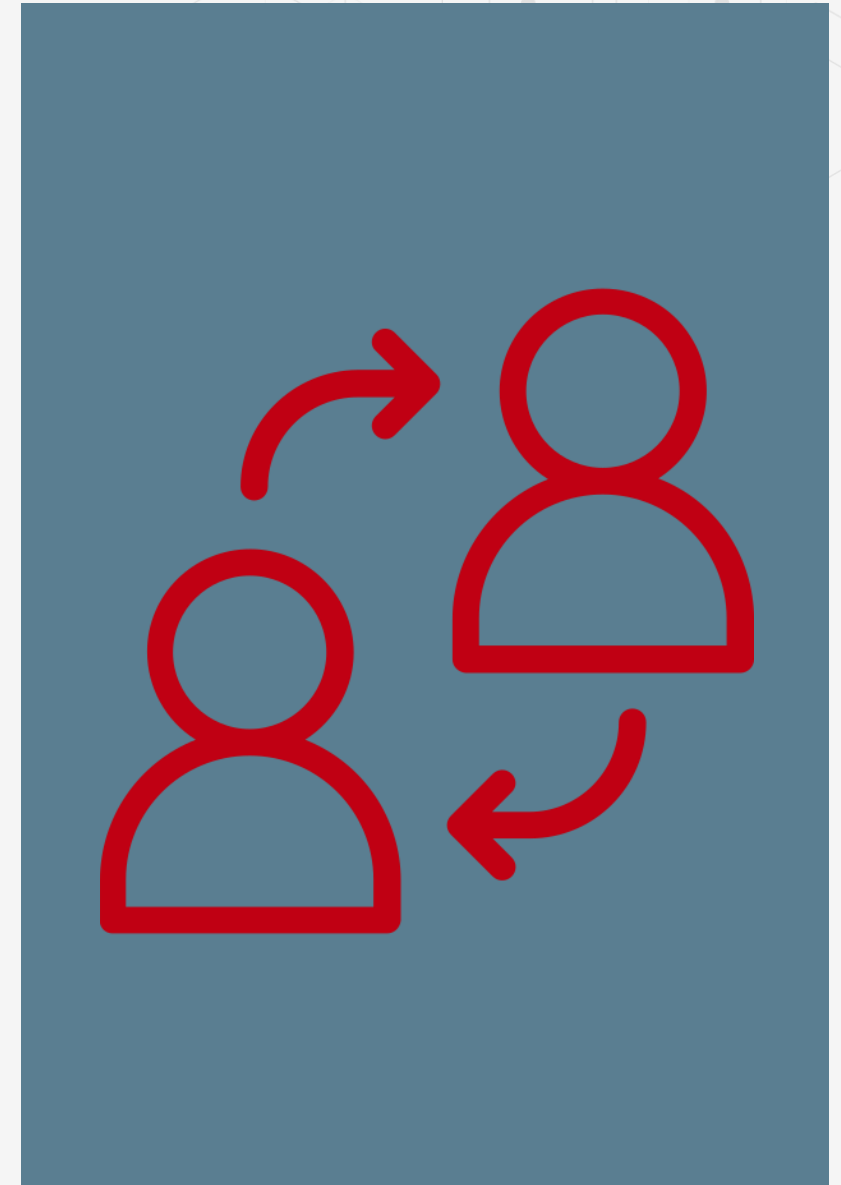
Symmetric Cryptographic Systems

Substitution ciphers

- Replace each character from the plaintext alphabet, A , with one from the ciphers alphabet, C
- If $A = \{a_1, a_2, \dots, a_n\}$ and $C = \{f(a_1), f(a_2), \dots, f(a_n)\}$, $f: A \rightarrow C$ is the substitution function, the cipher key
- In real solutions, f is implemented by linear transformations

$$C = (a * M + b) \bmod N$$

- a – amplification factor / selection factor for $b = 0$
- b – shifting coefficient
- the pair (a, b) – the substitution key



Symmetric Cryptographic Systems

Substitution ciphers

Caesar Cipher

- Mono-alphabetic substitution
- $A = \{A, B, C, \dots, X, Y, Z\} = C$
- $C(e_i) = e_i + 3 \pmod{26}$, with $e_i = \{0, 1, 2, \dots, 25\}$

$A\ B\ C\ \dots\ X\ Y\ Z$
 $D\ E\ F\ \dots\ A\ B\ C$
- a general function: $C(e_i) = (e_i + b_i) \pmod{26}$
- very vulnerable to attacks – mainly brute-force attacks
- the number of possible keys is 26

CURS $A\ B\ C\ \dots\ X\ Y\ Z$
 $D\ E\ F\ \dots\ A\ B\ C$ FXUV
CRIPTOGRAFIE ➡ ➡ FULSWRJUDILH



Symmetric Cryptographic Systems

Substitution ciphers

Random substitution cipher

- increase protection
- the characters of the substitution alphabet are statistical independent
- the key is a set $\{(a_1, b_1), (a_2, b_2), \dots, (a_{26}, b_{26})\}$, where a_i, b_i has values in $\{0, 1, 2, \dots, 25\}$

Substitution by mnemonic keys:

- the substitution rule is given by a literal key
- the mnemonic key generated by the literal key
- the number of correlations is bigger



encryption key

C	H	E	I	E
1	4	2	5	3

alphabet

A	B	C	D	E
F	G	H	I	J
K	L	M	N	O
P	Q	R	S	T
U	V	W	X	Y
Z				

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	F	K	P	U	Z	C	H	M	R	W	E	J	O	T	Y	B	G	L	Q	V	D	I	N	S	X

P_1 permutation

Symmetric Cryptographic Systems

Substitution ciphers

S	E	C	U	R	I	T	A	T	E
7	3	2	10	6	5	8	1	9	4

encryption key

1								A	B	C
2			D	E	F	G	H	I	J	K
3		L	M	N	O	P	Q	R	S	T
4										U
5						V	W	X	Y	Z
6										

alphabet

permutation P_1

ABCDEFGHIJKLMNOPQRSTUVWXYZ
LDMENFOGPHQWAIRXBJSYCKTUZ

Substitution by
stair-shaped
table

Symmetric Cryptographic Systems

Substitution ciphers

Homophonic substitution

- ciphers based on simple substitution are vulnerable to attacks that take into account characters frequency
- characters are replaced with symbols from $f(a)$, where $f:A \rightarrow 2^C$
- the frequency of the code symbols is almost constant
- the number of possible keys is $(26!)^n$

Poly-alphabetic substitutions

- $C_1, C_2, \dots, C_d$ – d cipher alphabets
- $f_1, f_2, \dots, f_d$ – d substitution functions $f_{i=1..d}: A \rightarrow C_i$
- $M = m_1 m_2 \dots m_d m_{d+1} \dots m_{2d} \dots$ – plaintext
- $Ek(M) = f_1(m_1)f_2(m_2)\dots f_d(m_d)f_1(m_{d+1})\dots$ – cipher



Symmetric Cryptographic Systems

Substitution ciphers

Poly-alphabetic substitutions – Vigenere cipher

- the key: $K = \{k_1, k_2, \dots, k_d\}$
- the substitution function $f_i(a) = (a + k_i) \pmod n$, where n – alphabet length
- another version is to use a binary alphabet –Vernam cipher

It is defined the equivalences $A = 0, B = 1, C = 2, \dots$

Plaintext: **SUBSTITUTIE POLIALFABETICA**

Key: **ACADEMIE**

$$S + A = 18 + 0 \pmod{26} = 18 \pmod{26} = 18 = S$$

$$U + C = 20 + 2 \pmod{26} = 22 \pmod{26} = 22 = W$$

$$B + A = 1 + 0 \pmod{26} = 1 \pmod{26} = 1 = B$$

Ciphertext: **SWBVXUBYTKE SSXQELHAEIFQGA**



Symmetric Cryptographic Systems

Substitution ciphers

Polygram substitution:

- substitutes block of chars (polygrams) from the plaintext
- Hides the frequency of different characters
- the simplest form is for $n=2$ when the digram m_1m_2 from the plaintext is substituted by the c_1c_2 digram from the ciphertext
- The correspondence between m_1m_2 and c_1c_2 digrams is defined by a square table

	A	B	C	D	E
A	QX	FN	LB	YE	HJ
B	AS	EZ	BN	RD	CO
C	PD	RA	MG	LU	OP



Symmetric Cryptographic Systems

Substitution ciphers

Polygram substitution :

- **PLAYFAIR cipher**- in the first line of the square it is placed a key word; the rest of the lines are completed with alphabet chars, without repeating them
- **algebraic encryption method** – linear transformation based on: $f(M)=P \cdot M^T$ where P is a square matrix with $n \times n$ lines and columns, and M is a column vector with n elements from the plaintext



Symmetric Cryptographic Systems

Substitution ciphers

- a 5X5 matrix of letters based on a keyword
- fill in letters of keyword (sans duplicates)
- fill rest of matrix with other letters
- eg. using the keyword **MONARCHY**

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Playfair Cipher:

- not even the large number of keys in a monoalphabetic cipher provides security
- improves security by encrypting multiple letters;
- invented by Charles Wheatstone in 1854, but named after his friend Baron Playfair

Symmetric Cryptographic Systems

Substitution ciphers

Playfair Encrypting and Decrypting

- plaintext is encrypted two letters at a time
 1. if a pair is a repeated letter, insert filler like 'X'
 2. if both letters fall in the same row, replace each with letter to right (wrapping back to start from end)
 3. if both letters fall in the same column, replace each with the letter below it (again wrapping to top from bottom)
 4. otherwise, each letter is replaced by the letter in the same row and in the column of the other letter of the pair

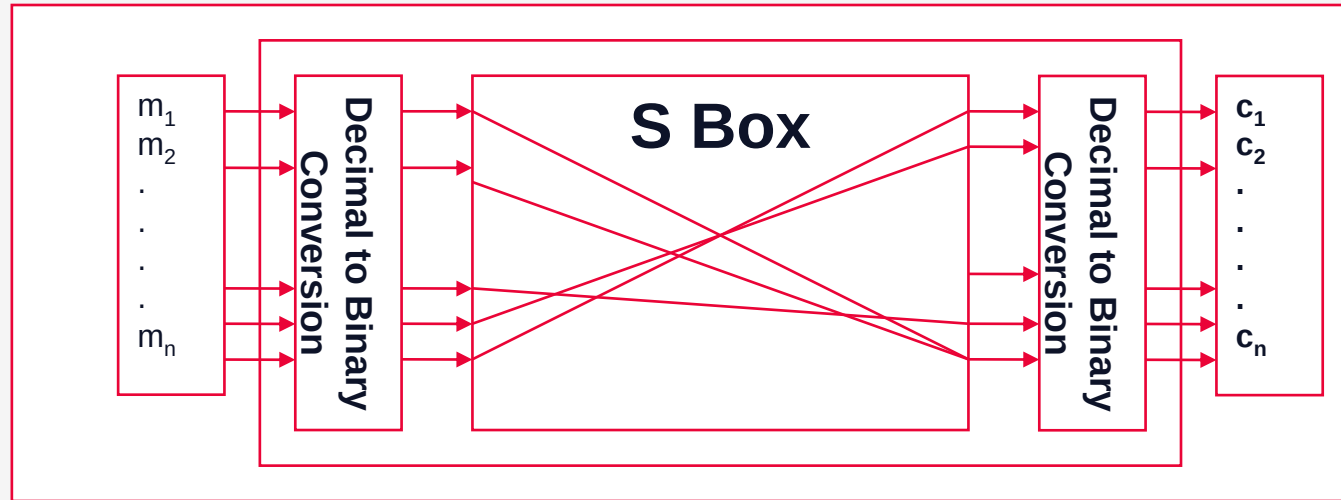
Security of Playfair Cipher

- security much improved over monoalphabetic
- since have $26 \times 26 = 676$ digrams
- would need a 676 entry frequency table to analyze (versus 26 for a monoalphabetic) and correspondingly more ciphertext
- was widely used for many years eg. by US & British military in WW1
- it can be broken, given a few hundred letters since still has much of plaintext structure



Symmetric Cryptographic Systems

Substitution cipher



Simple S Box

Symmetric Cryptographic Systems

OTP ciphers (One Time Pad)

- Each bit/character from the plaintext is encrypted by a modular addition (XOR) with a bit/character from a secret random key
- secure till our days (!!! if properly used = random key as large or greater than the plaintext)
- patented by Vernam (1917).
- Vernam has its own encryption algorithm but is not OTP



Symmetric Cryptographic Systems

Substitution ciphers

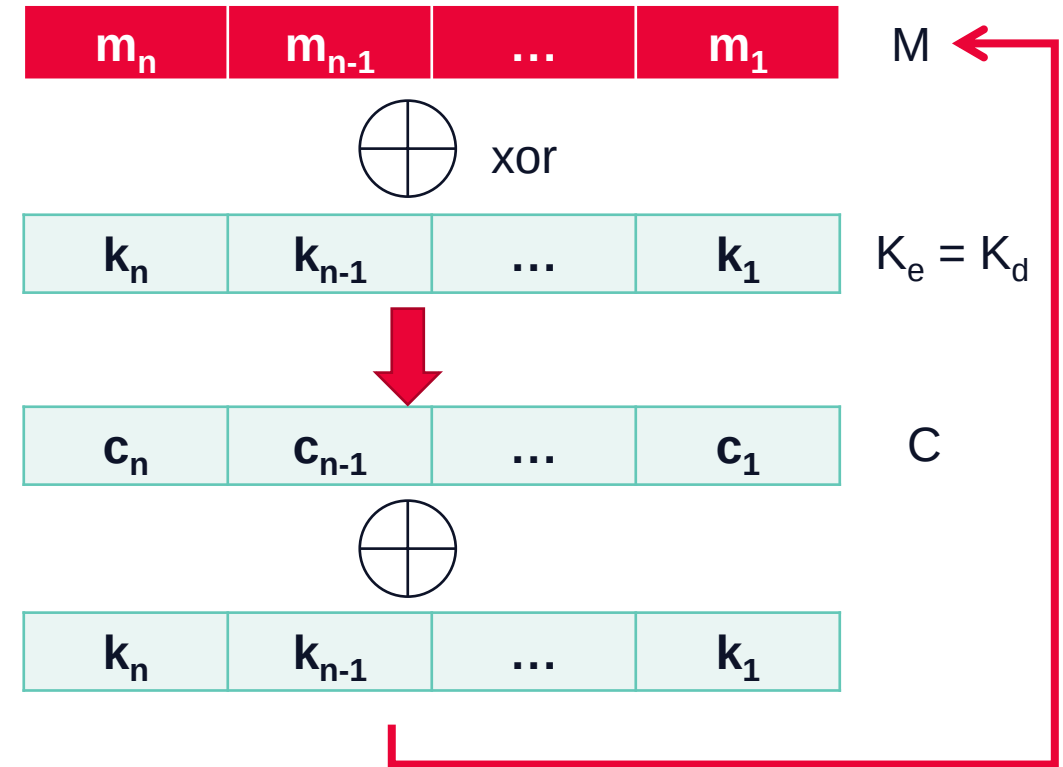
- defined in 1917 by Vernam
- **Property:**
- key length = message length

Advantages:

- impossible to break if the key is secured
- low complexity

Disadvantages:

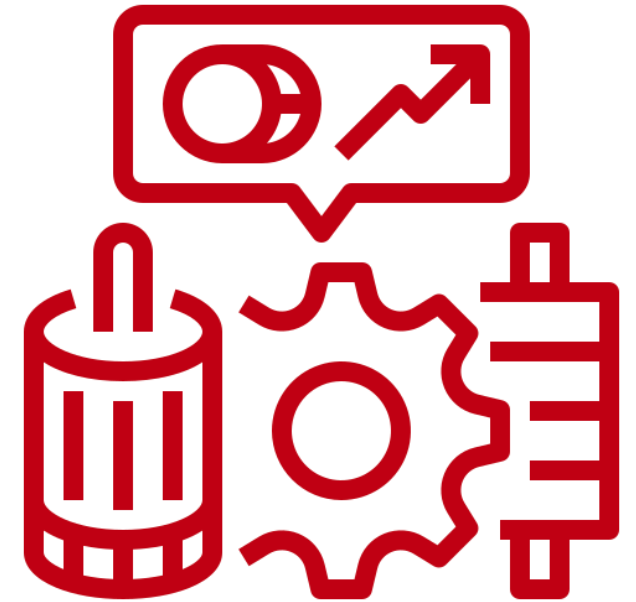
- the key length is the message length
- the key becomes a message that must be transmitted



Symmetric Cryptographic Systems

Rotor Machines

- The next major advance in ciphers required use of mechanical cipher machines which enabled to use of complex varying substitutions
- Before modern ciphers, rotor machines were most common complex ciphers in use
- Widely used in WW2
- German Enigma, Allied Hagelin, Japanese Purple
- Implemented a very complex, varying substitution cipher
- Used a series of cylinders, each giving one substitution, which rotated and changed after each letter was encrypted
- With 3 cylinders have $26^3=17576$ alphabets



Symmetric Cryptographic Systems

Enigma

- Implements a poly-alphabetic Vigenere encryption
- Designed in 1918 in Germany
- The security of the cipher:
 - number of disks (rotors): for 3 -> $26 \cdot 26 \cdot 26 = 17.576$ possible solutions
 - 6 ways to transpose disks -> $6 \cdot 17.576 = 105.456$ solutions
 - connectionn table with 10 pairs of chars -> $90 \cdot 105.456 = 9.491.040$
- A machine cu n rotors does the encryption of a symbol in $2n + 1$ substitutions



Symmetric Cryptographic Systems

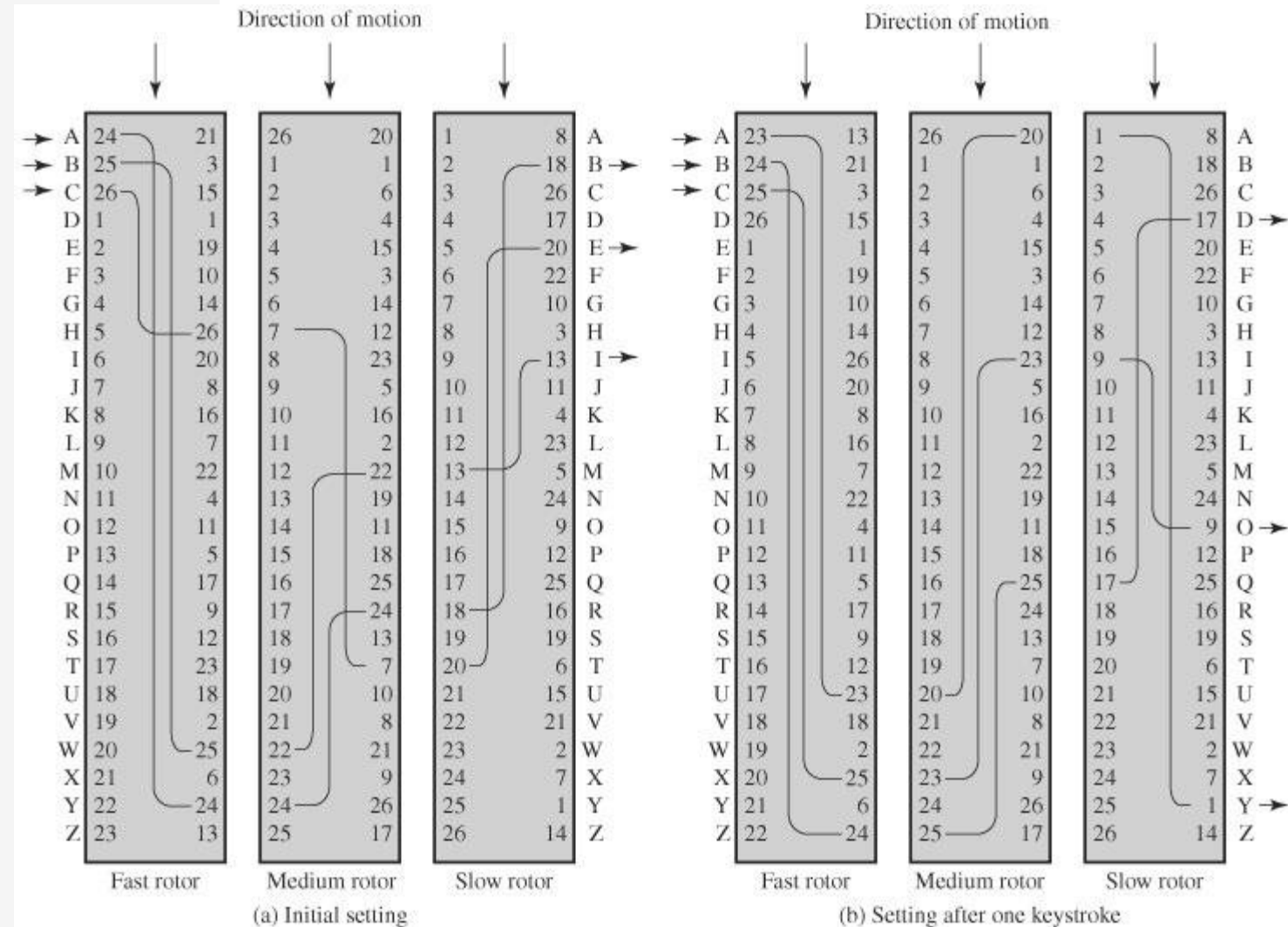
Enigma

Components:

- Keyboard
- Electric Circuits
- Rotors with 26 symbols (3 -> 7)
- Reflector
- Connections table

Settings:

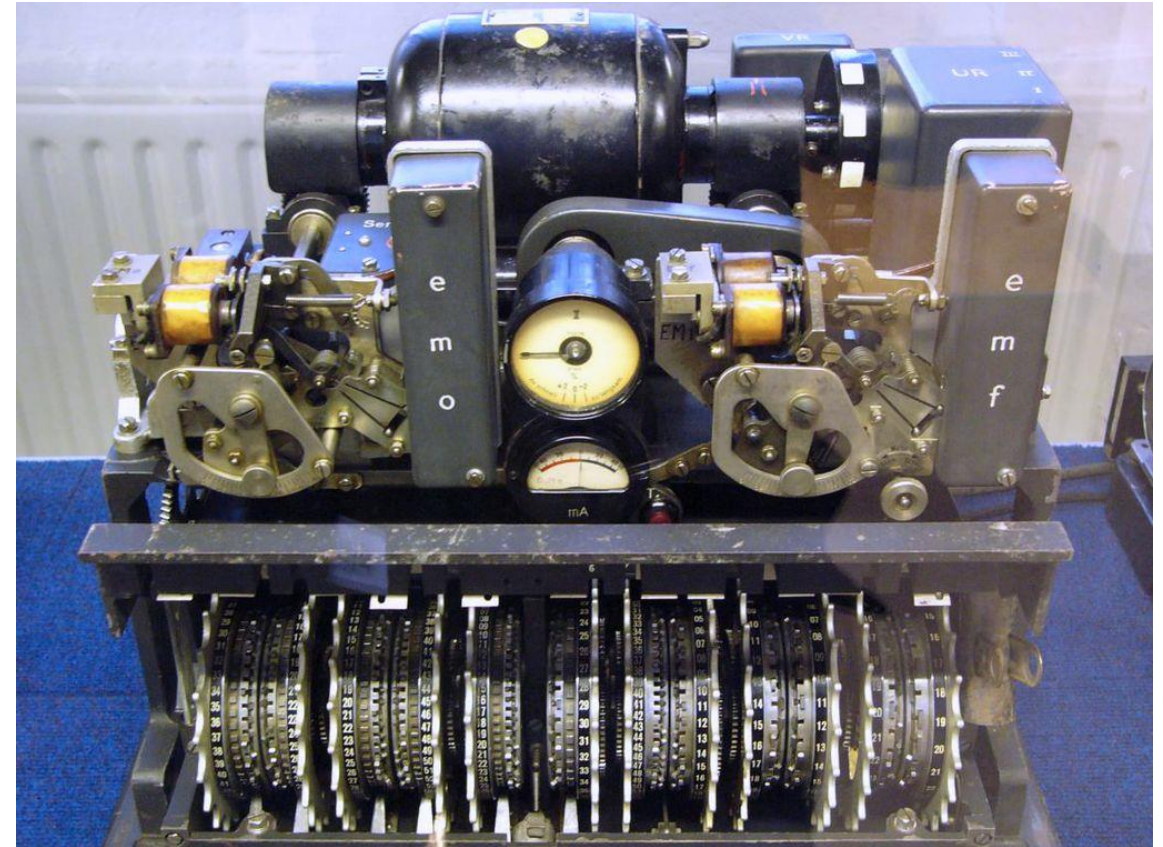
- The rotors order and their initial position
- The initialization of the symbols ring
- The initialization of connections



Symmetric Cryptographic Systems

Lorenz

- Declassified in 2002.
- Used from 1940 by the German Army.
- Used by high command and top generals.
- More advanced, complex, faster and more secure than Enigma.
- Bill Tutte broke Lorenz system in spring 1942 (without ever having seen the machine)
- https://en.wikipedia.org/wiki/Lorenz_cipher



Symmetric Cryptographic Systems

Rotor Machines

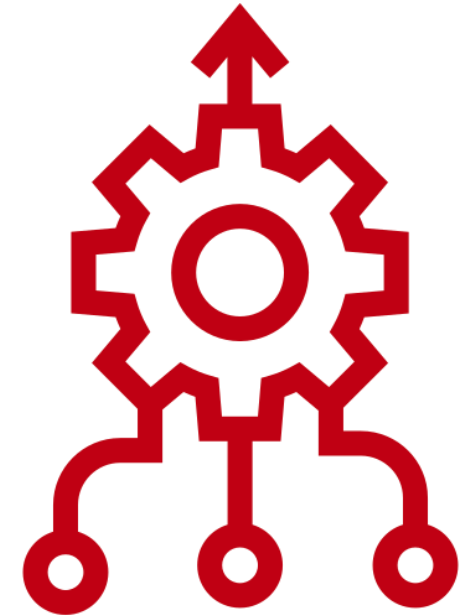


Enigma	Lorenz
~100.000 made	~200 made
3 or 4 rotors	12 rotors and 501 pins
103.325.660.891.587.134.000.000 different possible settings	$6.546.781 \cdot 10^{144}$ different possible settings

Symmetric Cryptographic Systems

Product ciphers

- A product/generated algorithm (also called product cipher) is a composition of t functions (ciphers) $f_1, f_2, \dots, f_t$, where each f_i can be a substitution or a permutation
- Are based on S-P boxes networks, resulting the cryptogram $C = E_k(M) = S_t P_{t-1} \dots S_2 P_1 S_1(M)$, each S_i being dependent of a k key, part of K cipher
- ciphers using only substitutions or transpositions are not secure because of language characteristics, hence consider using several ciphers in succession to make harder, but:
 - two substitutions make a more complex substitution
 - two transpositions make more complex transposition
 - **a substitution followed by a transposition makes a new much harder cipher**
- this is bridge from classical to modern ciphers



Symmetric Cryptographic Systems

Product ciphers

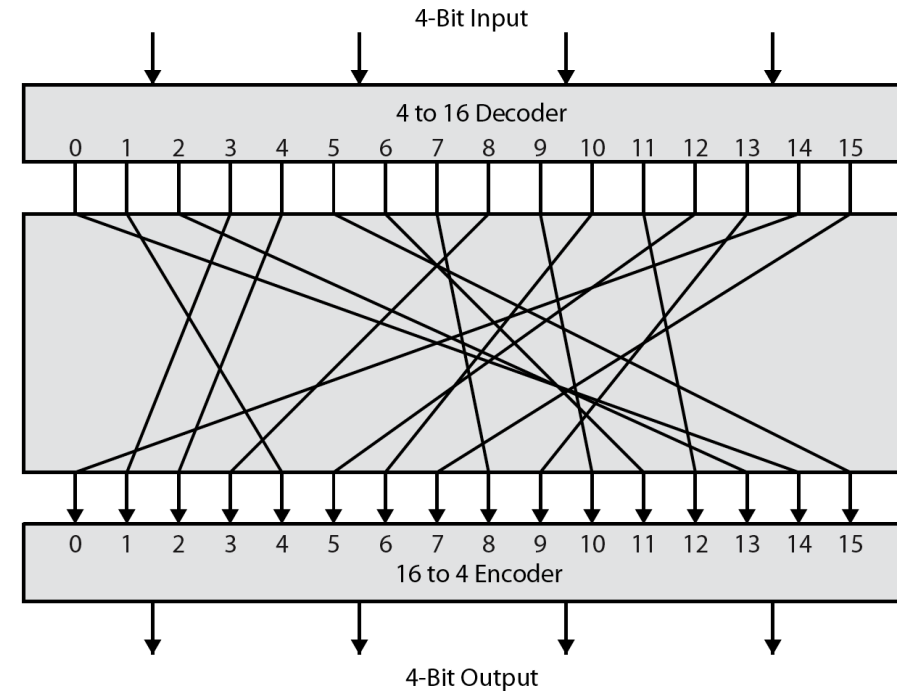
- **S-boxes** – maps entrances of n bits in exits of m bits (often $m=n$)
 - **Feistel networks** - method of transforming a cryptographic function into a permutation or building bits blocks, used by de cipher, of simple functions
 - **key scheduling** – the process of key expanding from N bits in $N*r$ bits
 - **Logical operations** on bits groups(bit slice operations) - AND, OR, XOR, NOT
-
- Claude Shannon introduced idea of substitution-permutation (S-P) networks in 1949 paper
 - the foundation of modern block ciphers
 - S-P nets are based on the two primitive cryptographic operations seen before:
 - substitution (S-box)
 - permutation (P-box)
 - provide confusion & diffusion of message & key



Symmetric Cryptographic Systems

Confusion and Diffusion

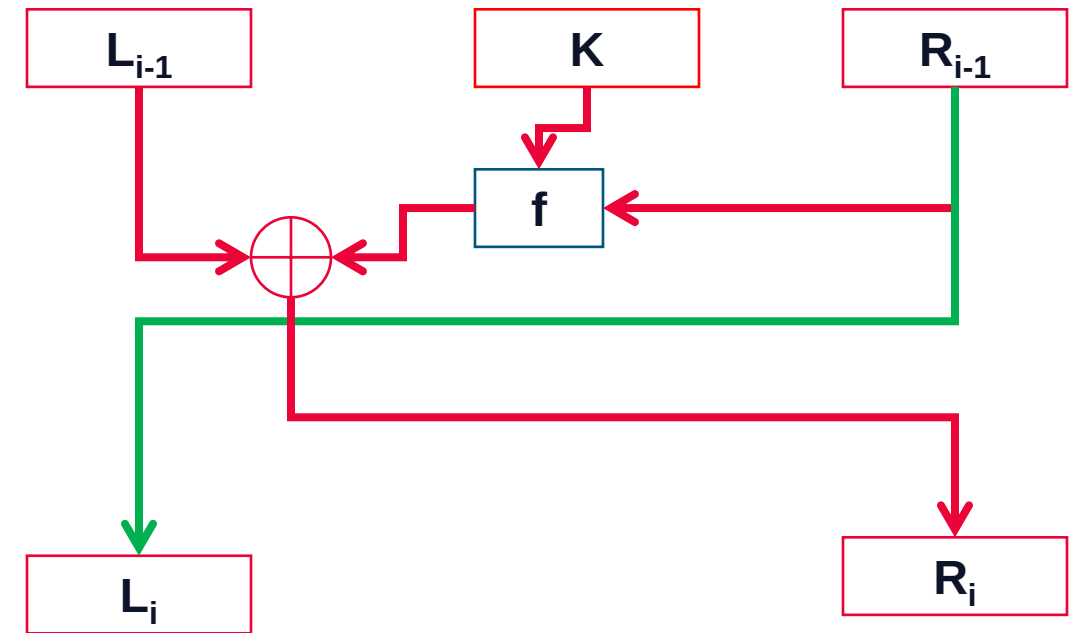
- cipher needs to completely obscure statistical properties of original message
- a one-time pad does this
- more practically Shannon suggested combining S & P elements to obtain:
- **diffusion** – dissipates statistical structure of plaintext over bulk of ciphertext
- **confusion** – makes relationship between ciphertext and key as complex as possible



Symmetric Cryptographic Systems

Feistel Network

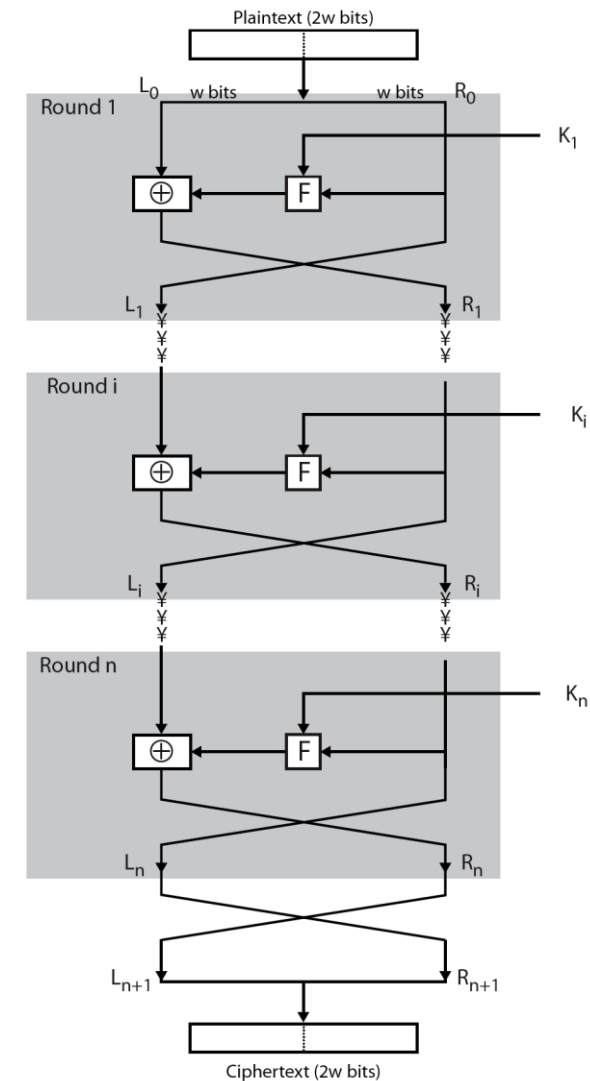
- Defined by Horst Feistel in the '60
- Used for the 1st time in Lucifer algorithm (IBM)
- The network takes a function f , $f:\{0,1\}^{n/2} \times \{0,1\}^N \rightarrow \{0,1\}^{n/2}$ and generates a reversible function $ff:\{0,1\}^n \rightarrow \{0,1\}^n$
 - $n/2$ is the lengths in bits of each L and R block
 - N is the number of bits of the key used by f function.
- Function ff is often called round function.
- If a round function depends on N key bits, then a cipher using Feistel networks with r rounds (r round functions meaning r ff functions) need $N \cdot r$ key bits.
- For designing f functions are typically used S boxes
- implements Shannon's S-P net concept



Symmetric Cryptographic Systems

Feistel Cipher Design Elements

- block size
- key size
- number of rounds
- subkey generation algorithm
- round function
- fast software en/decryption
- ease of analysis



Symmetric Cryptographic Systems

Complex ciphers

64 bits
(->1997)

- **Lucifer** (except 128 bits)
- **DES** – Data Encryption Standard
- **IDEA** – International Data Encryption Standard
- **FEAL** – Japanese Fast Data Encryption Algorithm
- **LOKI** – Australian symmetrical cipher
- **RC2** – Rivest Cipher

128 bits
(1997 ->)

- **AES** – Advanced Encryption Standard (Rijndael)
- **Twofish**
- **Serpent**
- **RC6**
- **MARS**
- **Blowfish**

Symmetric Cryptographic Systems

Cipher types

- **Block ciphering**

- Operates on blocks of plaintext and ciphertext – usually of 64, 128 bits and larger
- Most known block ciphering types: ECB, CBC, PCBC, OFB/NCF
- The same plaintext block will be always encrypted to the same ciphertext block, using the same key

- **Stream ciphering**

- Operates at bit/byte level
- Very simple, mostly based on XOR
- The security is in the key generator
- Modes: sequential cipher, self-synchronizing sequential cipher, feedback cipher, synchronous sequential cipher, output-feedback sequential cipher, counter cipher.
- The same plaintext will be encrypted to a different bit or byte in case of repeated encryptions (the key will be different)

Define ways to use symmetrical algorithms

The algorithm used does not matter

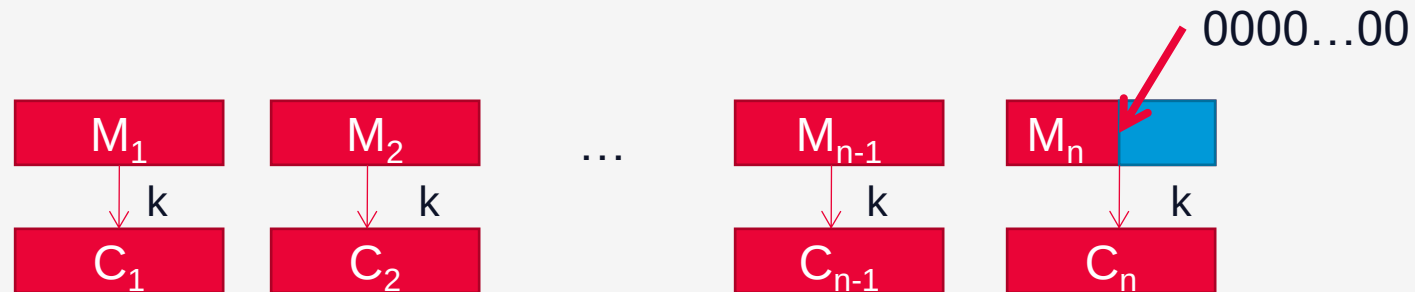
Are simple because the security is the attribute of ciphering and not of the way the ciphering scheme is done



Block Cipher Mode

Padding

- allows for processing of messages that are not evenly divisible into blocks of defined size (ex 64 bits)
- the last block could be shorter than required size
- the last block is padded by adding regular pattern (ex. zeroes, #, ...)



Block Cipher Mode

Padding

- **Bit padding** – you add a single 1 bit followed by needed number of 0 bits
- **Zero padding** – adds needed 0 value bits

```
. . . | 1101 0100 0010 0010 1000 0000 |
```

is not reversible

```
. . . | 1101 0100 0010 0010 0000 0000 |
```



Block Cipher Mode

Padding

- **ANSI X.923** – use needed 0 value bytes and the last byte defines the number of padded bytes

```
... | CC CC CC CC CC CC CC CC | CC CC  
00 00 00 00 00 06 |
```

- **ISO 10126** – use random bytes and the last byte defines the number of padded bytes

```
... | CC CC CC CC CC CC CC CC | CC CC  
16 4F 5D A4 11 06 |
```

Block Cipher Mode

Padding

- **PKCS#7** - [RFC 5652](https://www.rfc-editor.org/rfc/rfc5652) - value of each added byte is the number of bytes that are added

```
... | CC CC CC CC CC CC CC CC | CC CC  
06 06 06 06 06 06 |
```

the number of bytes added will depend on the block boundary to which the message needs to be extended; previous example is for 8 bytes (64 bit) blocks

- **PKCS#5** – same as PKCS7 but only for 64 bit block ciphers

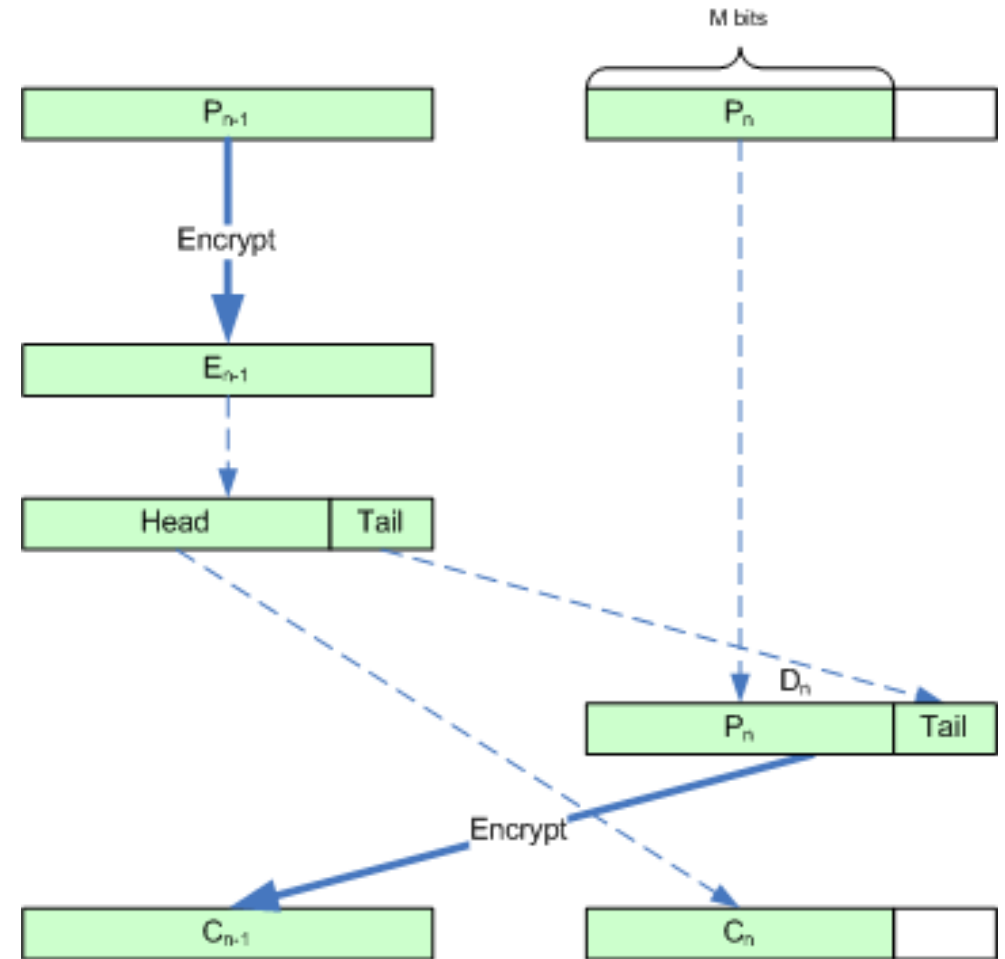
[https://en.wikipedia.org/wiki/Padding_\(cryptography\)](https://en.wikipedia.org/wiki/Padding_(cryptography))



Block Cipher Mode

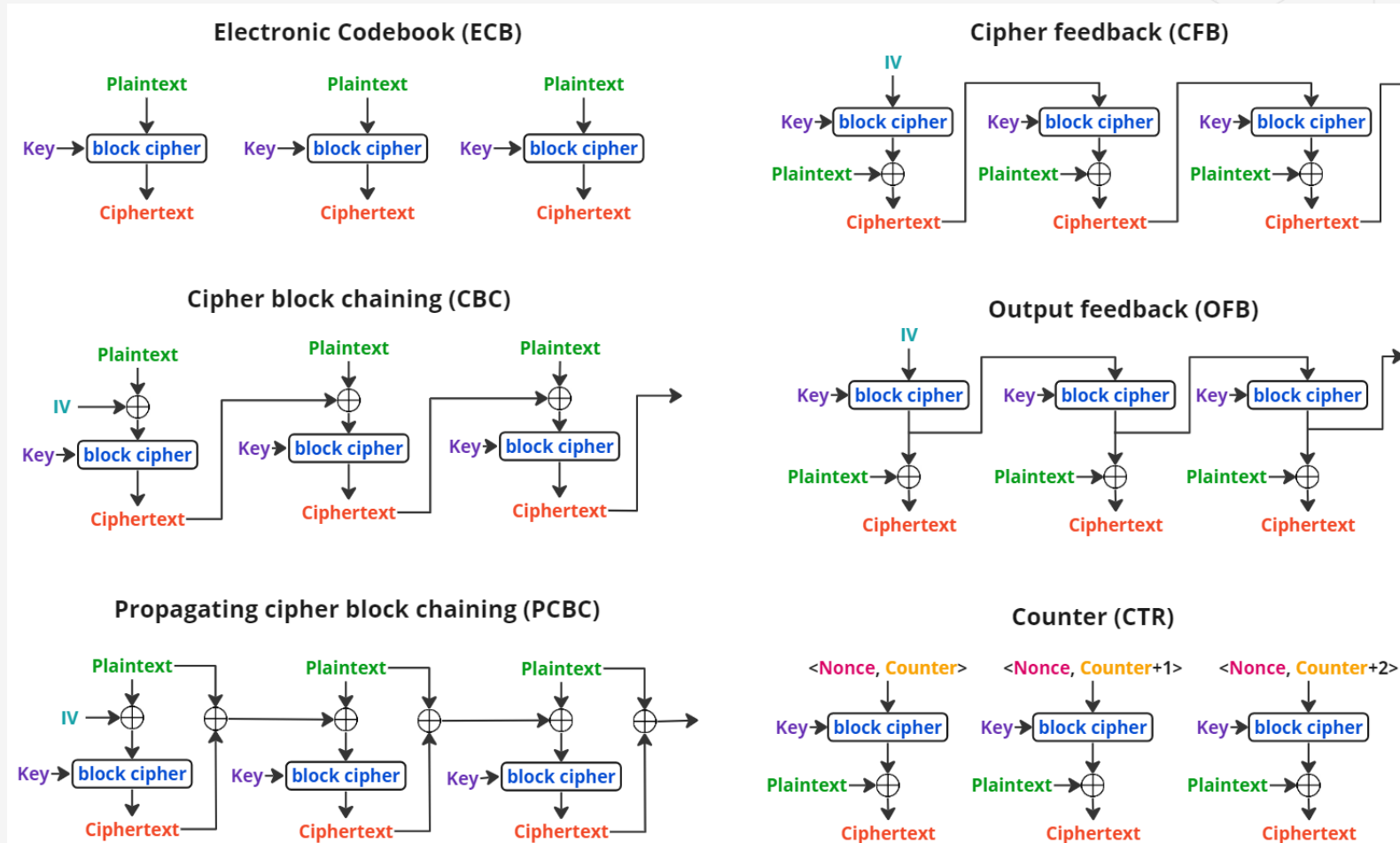
Padding

- Ciphertext stealing (CTS) is an alternative to padding
- http://en.wikipedia.org/wiki/Ciphertext_stealing



Symmetric Cryptographic Systems

Block Cipher Mode of Operation

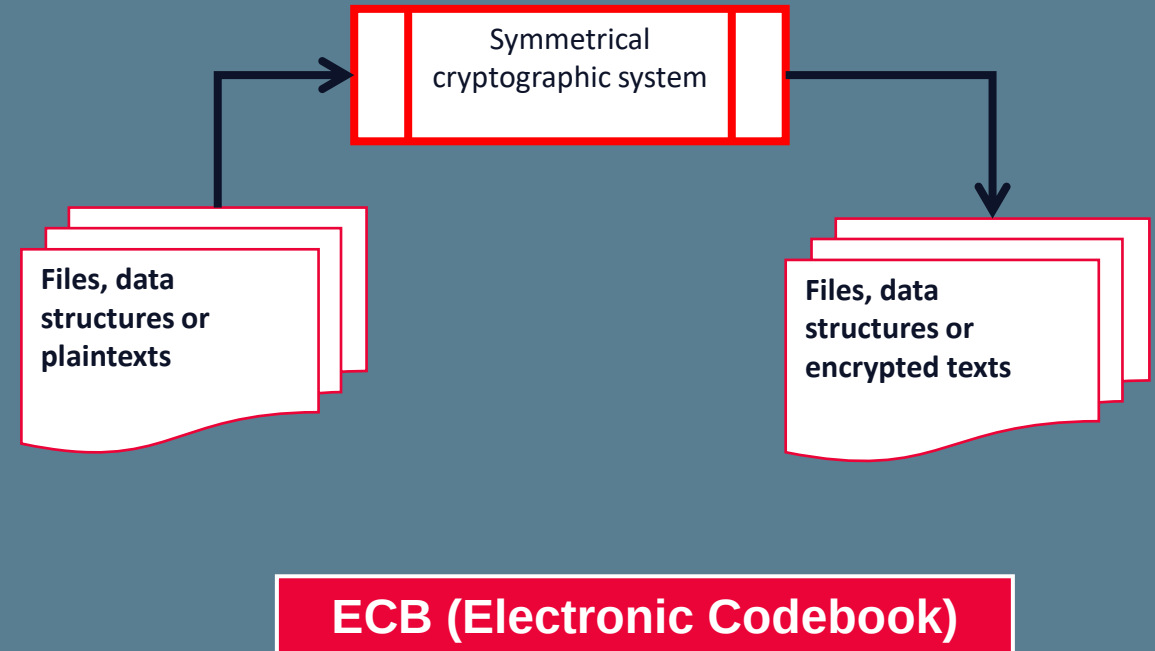


Source https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation

Block Cipher Mode

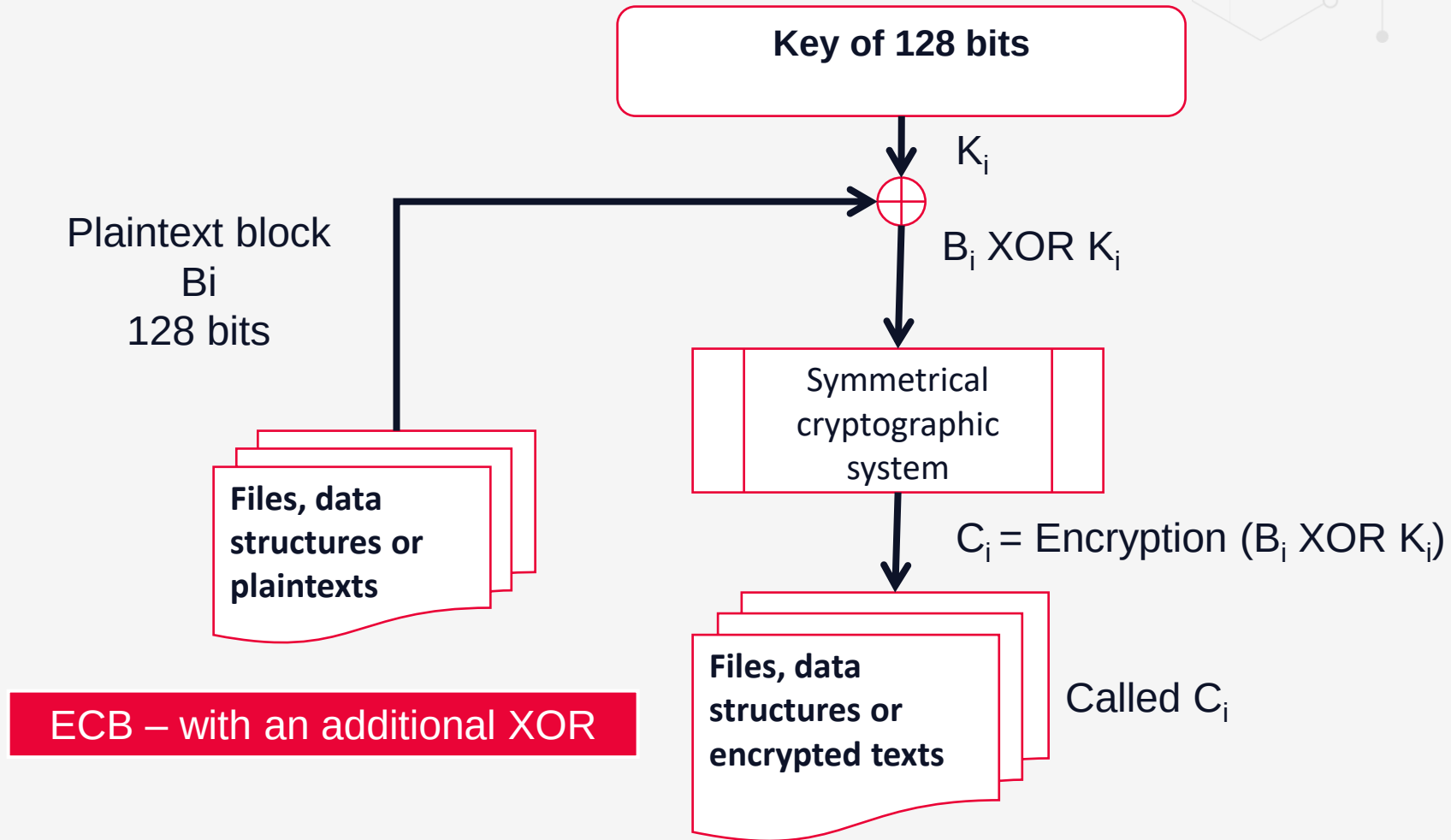
Electronic Codebook (ECB)

- Same block of plaintext encrypts into the same block of ciphertext every time is encrypted
- It is theoretically possible to create a code book of plaintexts and corresponding ciphertexts (not feasible because every key need its own code book)
- Each plaintext block is encrypted independently.
- Very vulnerable to block replay



Block Cipher Mode

Electronic Codebook (ECB)



Block Cipher Mode

Electronic Codebook (ECB)



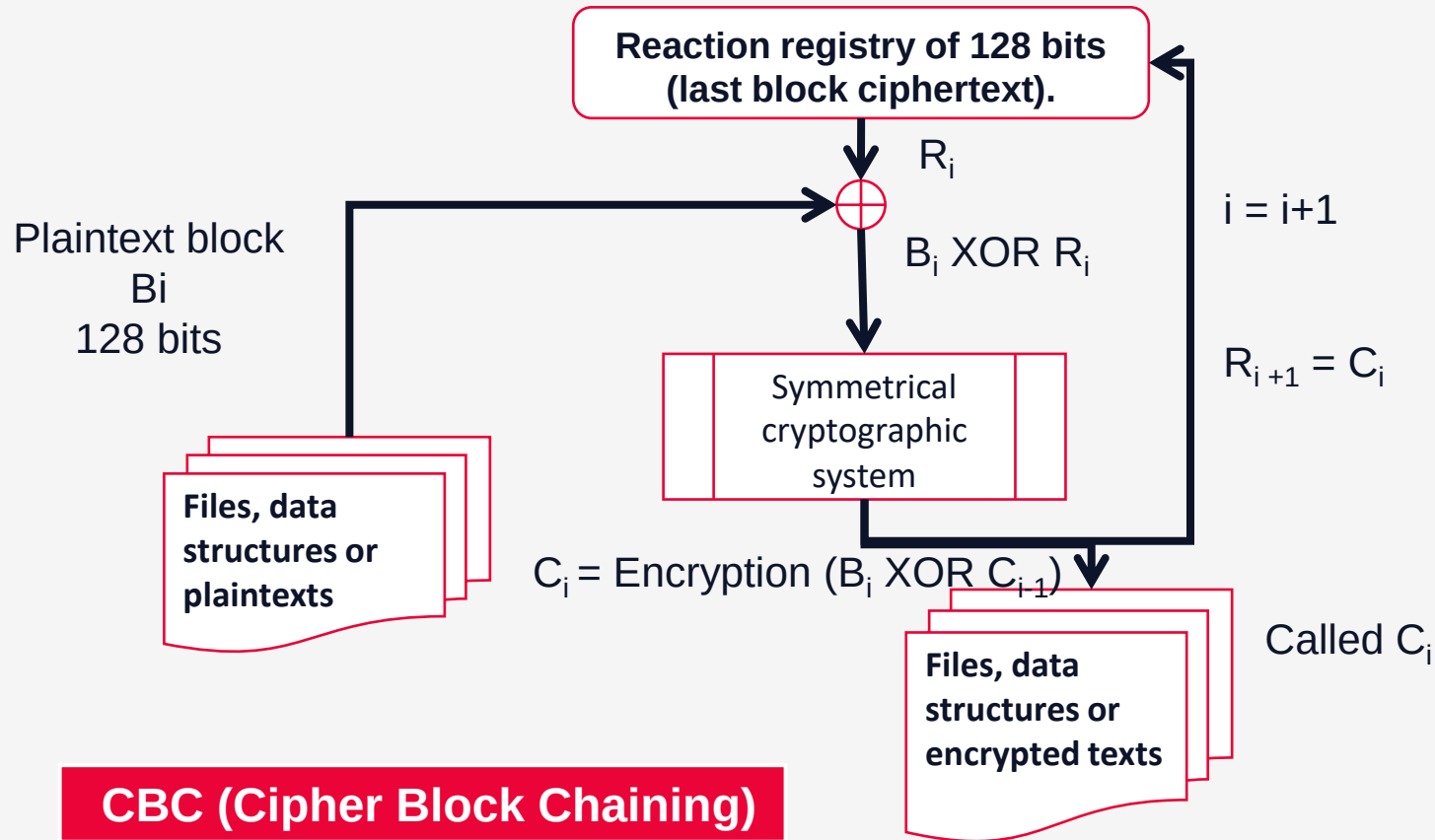
	Advantages	Disadvantages
Security	• More than one message can be encrypted with the same key.	• Input to the block cipher is not randomized; it is the same as the plaintext. • Plaintext patterns are not concealed. • Ciphertext is easy to manipulate; blocks can be removed, repeated, or interchanged.
Efficiency	• Speed is the same as the block cipher. • Processing is parallelizable.	• No preprocessing is possible. • Ciphertext is up to one block longer than the plaintext, due to padding.
Fault-tolerance		A ciphertext error affects one full block of plaintext.

[3]



Block Cipher Mode

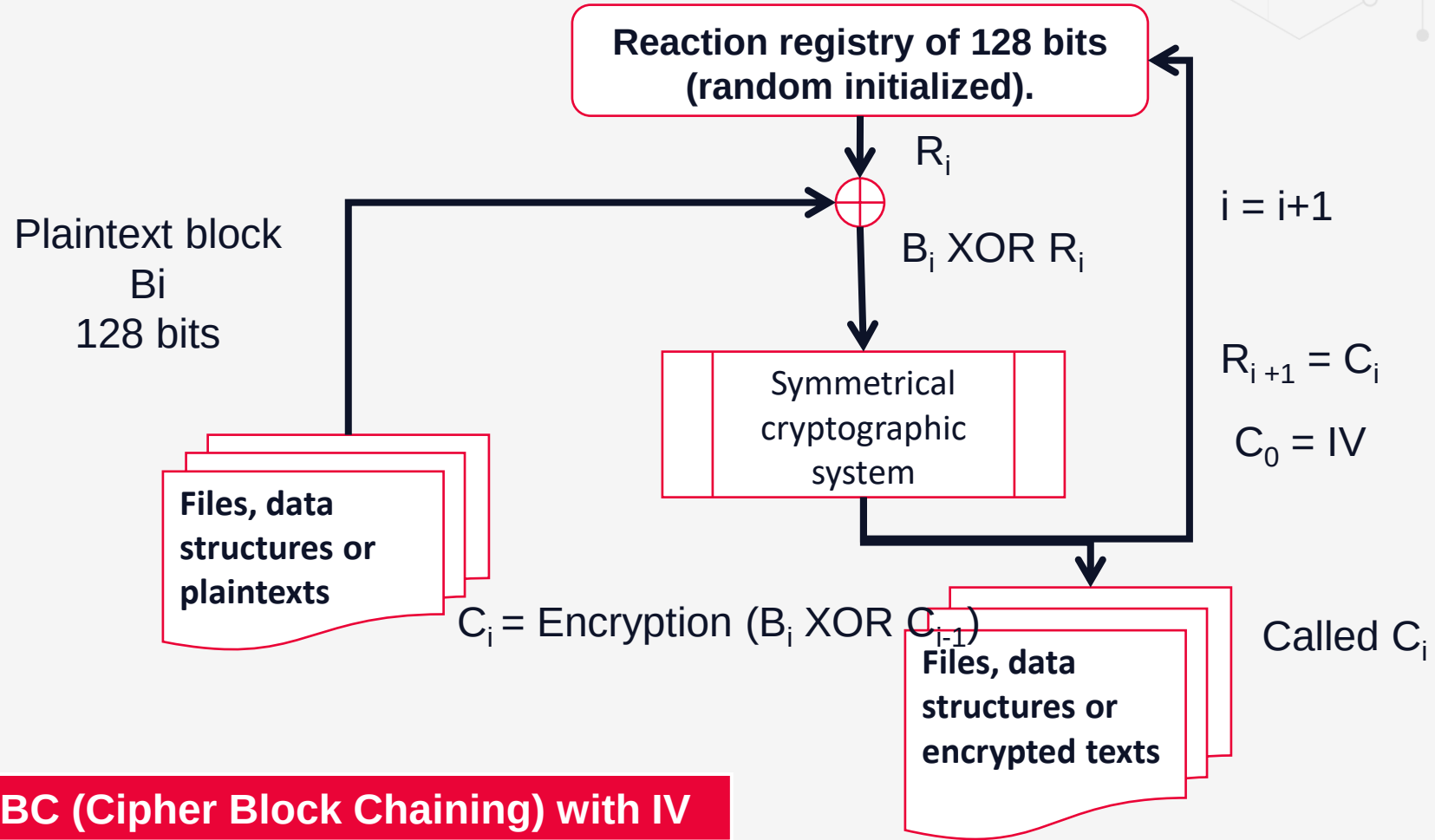
Cipher Block Chaining (CBC)



- Adds a feedback mechanism to a block cipher
- The result of the encryption of previous block are fed back into the encryption of current block
- Each ciphertext block is dependent not just on the plaintext block that generated it but on all the previous plaintexts blocks.

Block Cipher Mode

Cipher Block Chaining (CBC)



CBC (Cipher Block Chaining) with IV

Block Cipher Mode

Cipher Block Chaining (CBC)

	Advantages	Disadvantages
Security	• More than one message can be encrypted with the same key. • Plaintext patterns are concealed. • Input to the block cipher is randomized. • Ciphertext is hard to manipulate;	
Efficiency	• Speed is the same as the block cipher.	• No preprocessing is possible. • Ciphertext is up to one block longer than the plaintext, due to padding. • Encryption is not parallelizable.
Fault-tolerance		A ciphertext error affects one full block of plaintext and corresponding bit in the next block.

Block Cipher Mode

Cipher Block Chaining (CBC)



	Advantages	Disadvantages
Security	• More than one message can be encrypted with the same key (with different IV). • Plaintext patterns are concealed. • Input to the block cipher is randomized. • Ciphertext is hard to manipulate;	• Blocks can be removed from the beginning and end of the message, bits of the first block can be changed
Efficiency	• Speed is the same as the block cipher.	• No preprocessing is possible. • Ciphertext is the same size as the plaintext, not counting IV. • Encryption is not parallelizable.
Fault-tolerance		A ciphertext error affects the corresponding bit of plaintext and the next block.

Block Cipher Mode

ECB vs CBC

Electronic Codebook (ECB)

Same block of plaintext encrypts into the same block of ciphertext every time is encrypted

It is theoretically possible to create a code book of plaintexts and corresponding ciphertexts (not feasible because every key need its own code book)

Each plaintext block is encrypted independently.

Very vulnerable to block replay

Cipher Block Chaining (CBC)

Adds a feedback mechanism to a block cipher

The result of the encryption of previous block are fed back into the encryption of current block

Each ciphertext block is dependent not just on the plaintext block that generated it but on all the previous plaintexts blocks.

Two identical messages will still encrypt to the same ciphertext

Two messages that begin the same will encrypt in the same way up to the first difference

Increase security with a IV – initialization vector



Block Cipher Mode

ECB vs CBC

Electronic Codebook (ECB)

aa

encrypted with AES in ECB mode with key
"passwordpassword" leads to

53 32 60 BC 31 BF C6 2B 99 6D 2B C5 3F
F0 E1 72 53 32 60 BC 31 BF C6 2B 99 6D
2B C5 3F F0 E1 72 3E 8C 67 1F 85 72 F0
FA A3 F8 6C 6A 53 32 41 90

Cipher Block Chaining (CBC)

aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaa

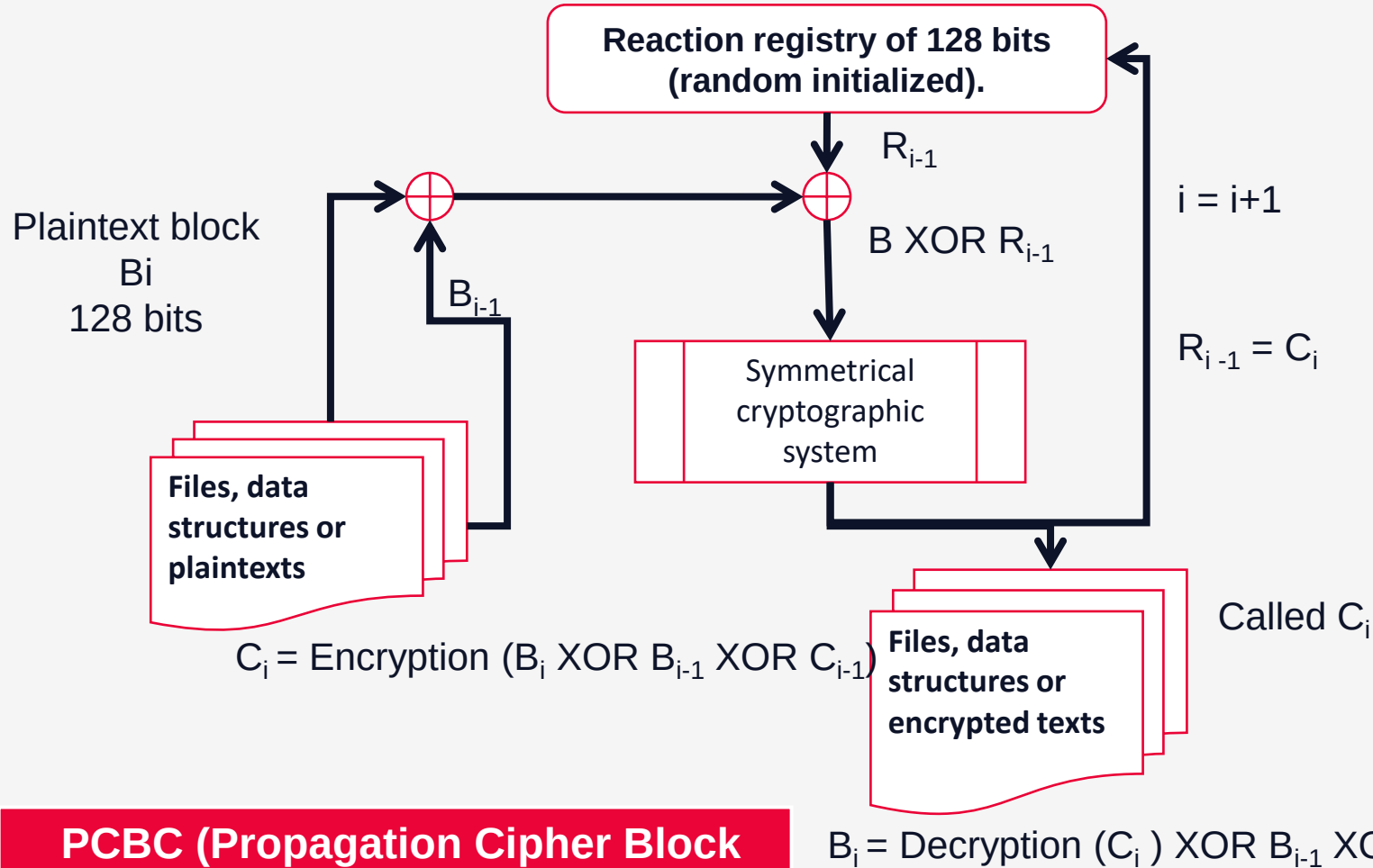
encrypted with AES in CBC mode with key
"passwordpassword" and IV (zero byte) leads to

53 32 60 BC 31 BF C6 2B 99 6D 2B C5
3F F0 E1 72 03 53 72 20 88 79 1F
B7 85 A9 27 38 18 2A 52 C6 1D 8A
81 9B A4 5C 24 A3 7A F5 D7 E1 0A
84 95 CB



Block Cipher Mode

Propagation Cipher Block Chaining (PCBC)



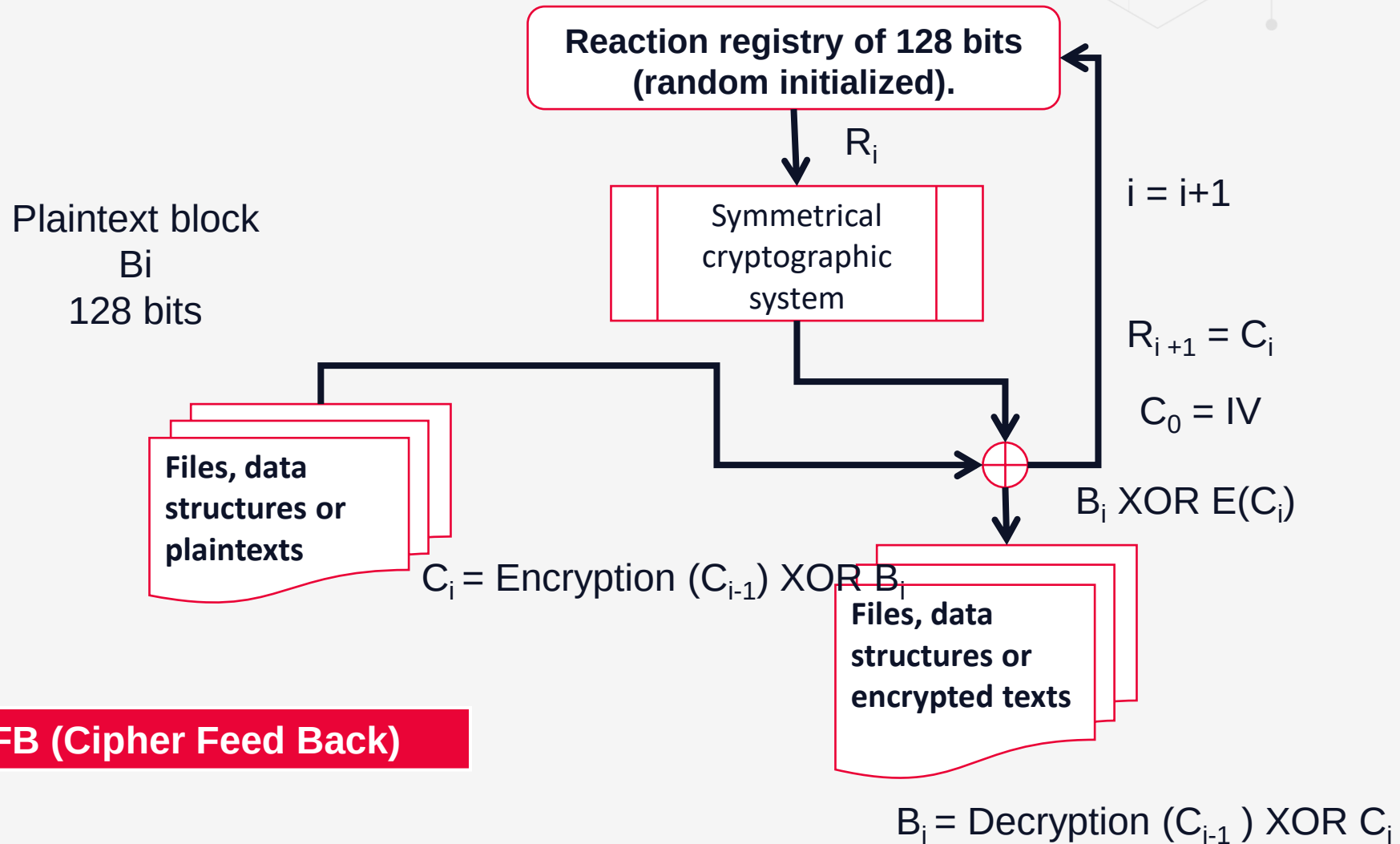
- Like CBC mode
- Both previous plaintext block and ciphertext blocks are XORed with the current plaintext block before encryption (or after decryption)
- PCBC was used in Kerberos version 4 to perform both encryption and data integrity checking in one pass.

PCBC (Propagation Cipher Block Chaining)



Block Cipher Mode

Cipher FeedBack (CFB)



Block Cipher Mode

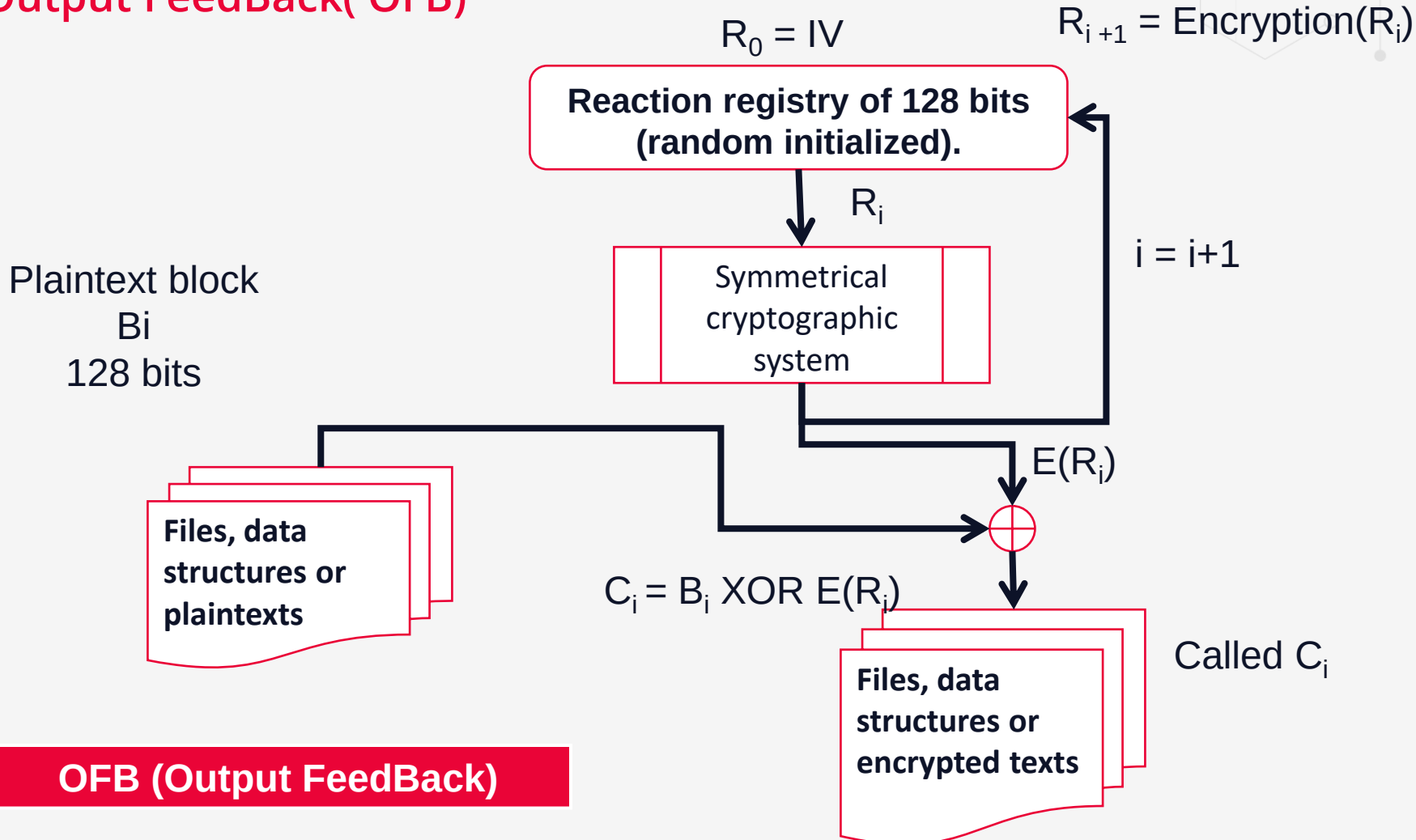
Cipher FeedBack (CFB)



	Advantages	Disadvantages
Security	•Plaintext patterns are concealed. •Input to the block cipher is randomized. •Ciphertext is hard to manipulate;	
Efficiency	•Speed is the same as the block cipher.	•No preprocessing is possible. •Ciphertext is up to one block longer than the plaintext, due to padding. •Encryption is not parallelizable.
Fault-tolerance		A ciphertext error affects one full block of plaintext and corresponding bit in the next block.

Block Cipher Mode

Cipher Output FeedBack(OFB)



OFB (Output FeedBack)

Decryption: $B_i = C_i \text{ XOR } R_i$

Block Cipher Mode

Cipher Output FeedBack (OFB)

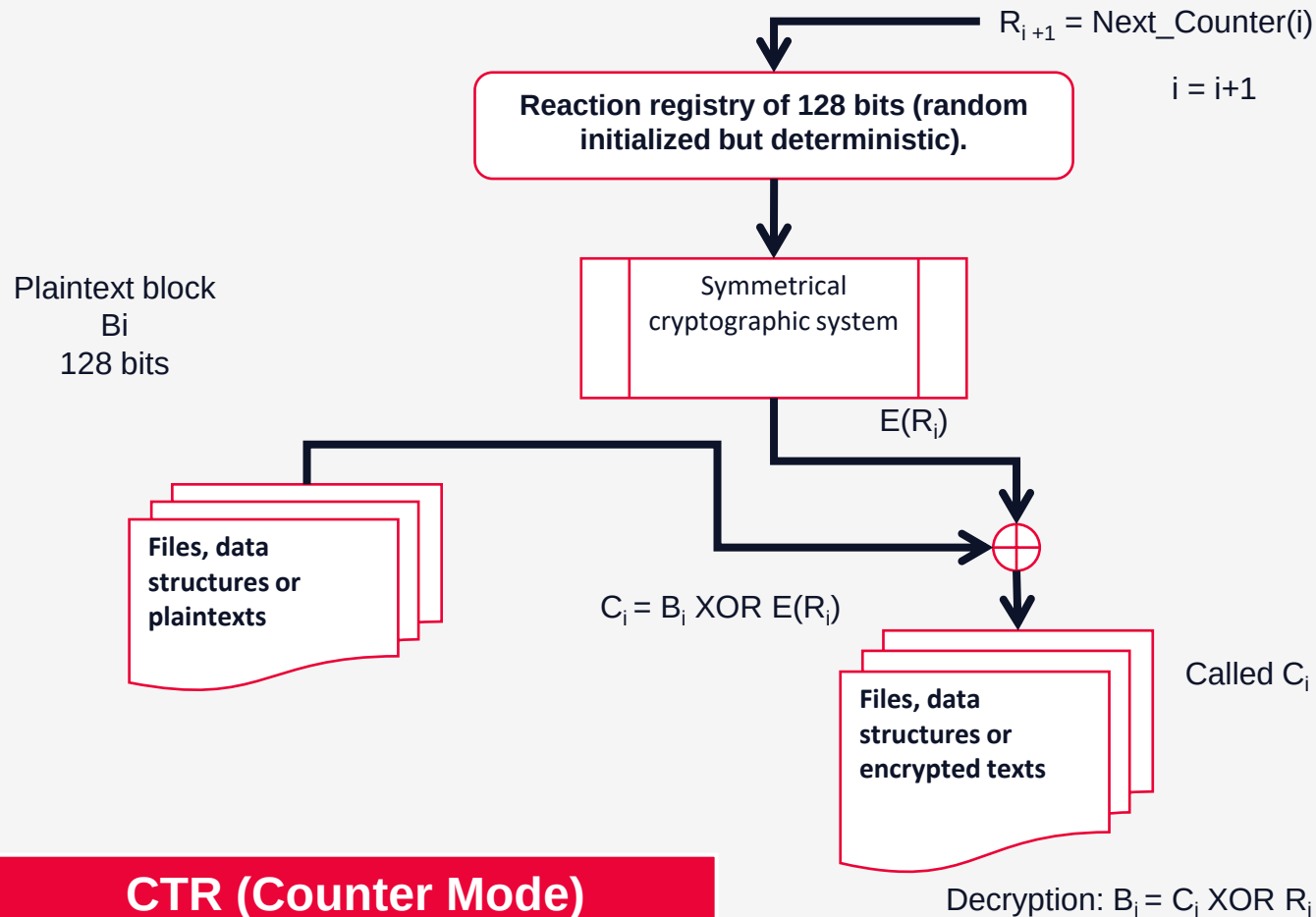
Advantages and Limitations

- bit errors do not propagate
- more vulnerable to message stream modification
- a variation of a Vernam cipher
- hence must never reuse the same sequence (key+IV)
- sender & receiver must remain in sync
- originally specified with m-bit feedback
- subsequent research has shown that only full block feedback (ie CFB-64 or CFB-128) should ever be used



Block Cipher Mode

Counter (CTR)



- a “new” mode, though proposed early on
- similar to OFB but encrypts counter value rather than any feedback value
- must have a different key & counter value for every plaintext block (never reused)
$$C_i = P_i \text{ XOR } O_i$$
$$O_i = \text{DES}_{K1}(i)$$
- uses: high-speed network encryptions

Block Cipher Mode

Counter (CTR)

- efficiency
 - can do parallel encryptions in h/w or s/w
 - can preprocess in advance of need
 - good for bursty high speed links
- random access to encrypted data blocks
- provable security (good as other modes)
- but must ensure never reuse key/counter values, otherwise could break (cf OFB)



Block Cipher Mode

Summary

- **cipher feedback** (CFB) mode, transforms a block cipher into a self-synchronizing stream cipher
- **output feedback** (OFB) mode makes a block cipher into a synchronous stream cipher
- **counter mode** (CTR) turns a block cipher into a stream cipher



Symmetric Cryptographic Systems

DES (Data Encryption Standard)

- The 1st standard for data cryptographic protection
 - Studied by IBM starting in 1970 for NBS (National Bureau of Standards)
 - Published as FIPS PUBS 46 (Federal Information Processing Standards Publications)
 - In 1977 is named DES and used until 1998 when it was hacked/cracked
 - standard ANSI X3.92 and named DEA (Data Encryption Algorithm)
- Symmetric block cipher developed based on IBM Lucifer algorithm
- Encrypts data in 64-bit blocks
- Key lengths of 64 bits – 56 bits random generated (or from password) and 8 bits for detecting transmission errors (each bits represents the odd parity of key's 8 octets)
- Was considerable controversy over design
 - in choice of 56-bit key (vs Lucifer 128-bit)
 - and because design criteria were classified



Symmetric Cryptographic Systems

DES (Data Encryption Standard)

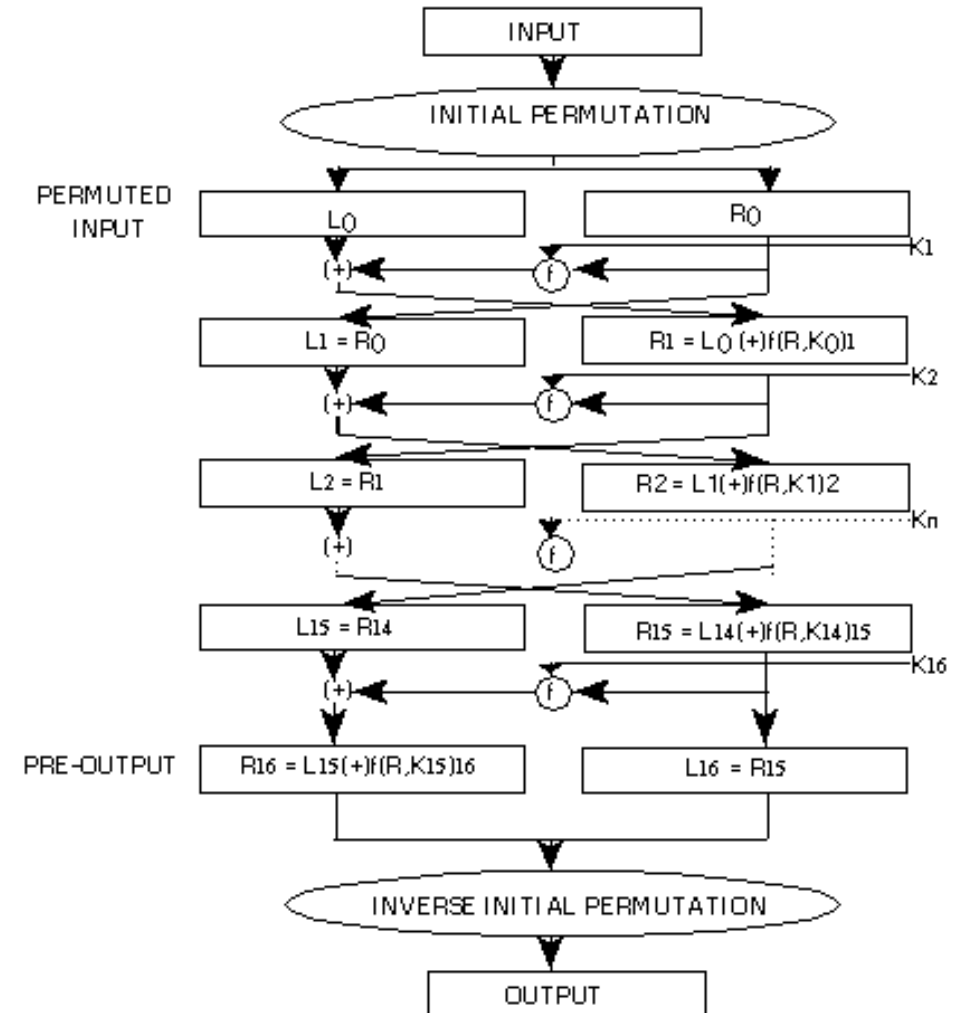
- In 90's many researcher show that with current technology DES can be broken
 - 1994, the first experimental cryptanalysis of DES is performed using linear cryptanalysis (Matsui, 1994)
 - 1997, The [DESCHALL Project](#) breaks a message encrypted with DES for the first time in public
 - 1998, The EFF's [DES cracker \(Deep Crack\)](#) breaks a DES key in 56 hours (The machine was testing over 90 billion keys per second. It would take about 9 days to test every possible key at that rate. On average, the correct key would be found in half that time)
 - ...
- As a temporary solution, DES security is increased by using a multiple encryption scheme, [Triple DES \(3DES\)](#)
 - applies the DES cipher algorithm three times to each data block
 - 1978, a triple encryption method using DES with two 56-bit keys was proposed by Walter Tuchman; EDE (Encrypt Decrypt Encrypt) with 2 keys, $E(D(E(M, K1), K2), K1)$
 - in 1981, Merkle and Hellman proposed a more secure triple-key version of 3DES with 112 bits of security, $E(E(E(M, K1), K2), K3)$



Symmetric Cryptographic Systems

DES (Data Encryption Standard)

- Combines two encryption techniques: confusion and diffusion, a substitution followed by a permutation)
- A processing round: Feistel network with permutation between 2 blocks (32 bits) of initial message block and a substitution through f function that will become a nonreversible ff function due to Feistel network.
- Made of 16 rounds
- Each round uses a different 48 key bits selected from an initial 56 key bits



Symmetric Cryptographic Systems

Brute-force on DES (Data Encryption Standard)

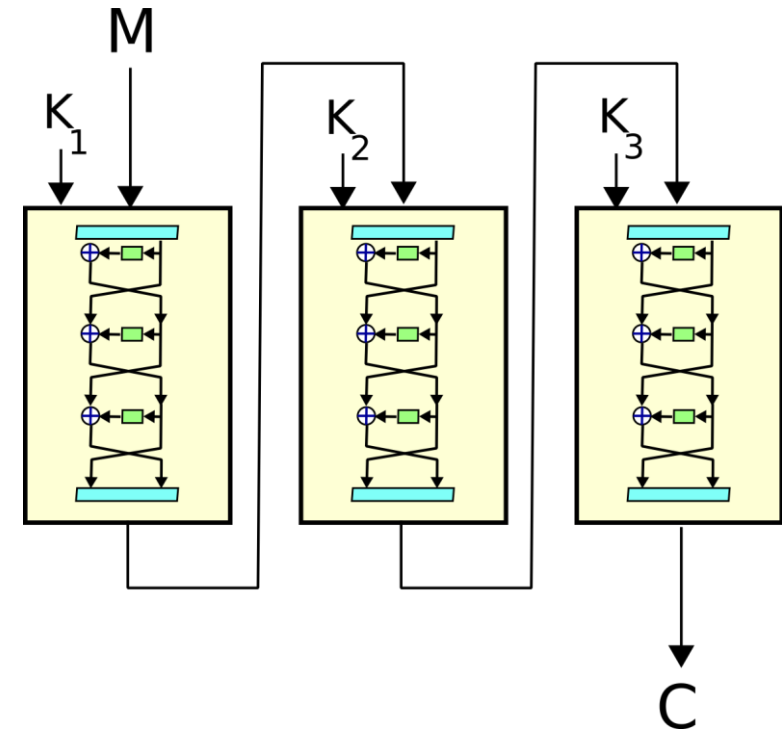
- 56-bit keys have $2^{56} = 7.2 \times 10^{16}$ values
- brute force search looks hard
- history have shown is possible
 - in 1997 on Internet in a few months
 - in 1998 on dedicated h/w (EFF) in a few days
 - in 1999 above combined in 22hrs!



Symmetric Cryptographic Systems

Triple DES (3DES)

- In response to DES's vulnerabilities, **3DES** was introduced in the late 1990s as an interim solution.
- Apply the DES algorithm three times to each block of data with 3 different keys
- 3DES was seen as a stronger encryption method because it used three 56-bit keys, which effectively provided 168-bit key security, though later analysis showed the effective security was about 112 bits
- The term **EDE** specifically describes the mode of operation used in **3DES**, where the data is encrypted, then decrypted, and finally encrypted again
 - **Encrypt** the data with the first key (Key1).
 - **Decrypt** the output using the second key (Key2).
 - **Encrypt** the result again with the third key (Key3).
- Sometimes, a simpler version of 3DES is used, known as 2-key 3DES, where the 1st and 3rd keys are the same (Key1 = Key3), reducing the number of independent keys but still providing more security than DES alone



https://en.wikipedia.org/wiki/Triple_DES

Symmetric Cryptographic Systems

NIST Requirements for New Standard

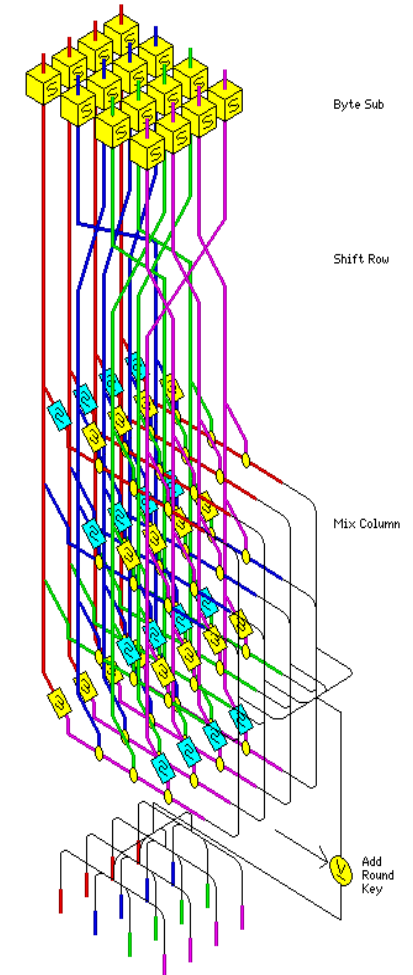
- A system of symmetric encryption based on 128 bits blocks
- Keys of 128, 192 and 256 bits length
- Does not contain weak keys
- Efficient on Intel platforms as well as other software or hardware platforms
- Able to be implemented on 32 bits processors and smart-cards (8 bits processors)
- Faster than DES and offering a higher security than 3DES
- private key symmetric block cipher
- 128-bit data, 128/192/256-bit keys
- stronger & faster than Triple-DES
- active life of 20-30 years (+ archival use)
- provide full specification & design details
- both C & Java implementations
- NIST have released all submissions & unclassified analyses



Symmetric Cryptographic Systems

AES (Advanced Encryption Standard)

- finalist and winner of AES contest launched by NIST 1997
- designed by Joan Daemen and Vincent Rijman
- became standard from 2000 (FIPS PUB 197)
- **uses 128, 192 or 256 bits keys**
- It is a symmetric cryptographic algorithm
- **processes blocks of 128 bits**
- can be efficiently implement on 8-bit CPUs or 32-bit CPUs



Rijndael Round

Symmetric Cryptographic Systems

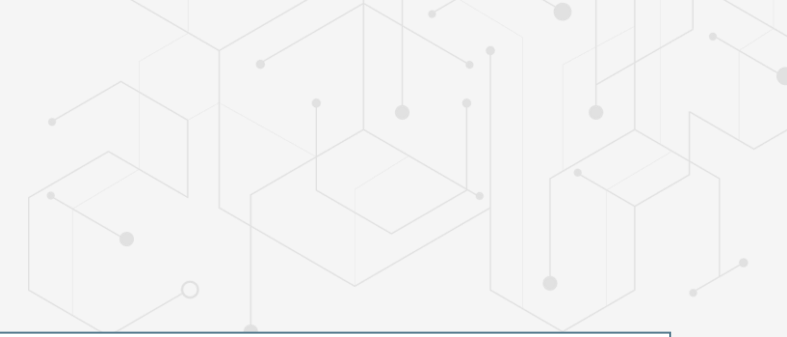
AES (Advanced Encryption Standard)

- finalist and winner of AES contest launched by NIST 1997 (shortlist in Aug-99)
 - MARS (IBM) - complex, fast, high security margin
 - RC6 (USA) - v. simple, v. fast, low security margin
 - Rijndael (Belgium) - clean, fast, good security margin
 - Serpent (Euro) - slow, clean, v. high security margin
 - Twofish (USA) - complex, v. fast, high security margin
- initial criteria:
 - security – effort for practical cryptanalysis
 - cost – in terms of computational efficiency
 - algorithm & implementation characteristics
- final criteria
 - general security
 - ease of software & hardware implementation
 - implementation attacks
 - flexibility (in en/decrypt, keying, other factors)



Symmetric Cryptographic Systems

AES (Advanced Encryption Standard)



8-BIT

- byte substitution works on bytes using a table of 256 entries
- shift rows is simple byte shift
- add round key works on byte XOR's
- mix columns requires matrix multiply in $GF(2^8)$ which works on byte values, can be simplified to use table lookups & byte XOR's

32-BIT

- redefine steps to use 32-bit words
- can precompute 4 tables of 256-words
- then each column in each round can be computed using 4 table lookups + 4 XORs
- at a cost of 4Kb to store tables
- designers believe this very efficient implementation was a key factor in its selection as the AES cipher



Symmetric Cryptographic Systems

Brute-force attacks

Brute-force attack with a **million/second** key search computer [Schneier, 97]:

- for a 56 bits key – 2285 years
- for a 64 bits key – 585000 years
- for a 128 bits key – 10^{25} years

Brute-force attack with a **2^{30} encryptions per second** key search computer ([link](#))

- for a 64 bits key – 545 years on a single machine or in 1 year on 640 cores in cloud
- for a 128 bits key – 10^{22} years

An efficient brute-force attack is implemented in a parallel architecture (Ex The Chinese lottery [Schneier])



Symmetric Cryptographic Systems

Brute-force attacks

Key Size (bits)	Number of Alternative Keys	Time required at 1 decryption/ μ s	Time required at 10^6 decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

Symmetric Cryptographic Systems

Multiple encryptions systems

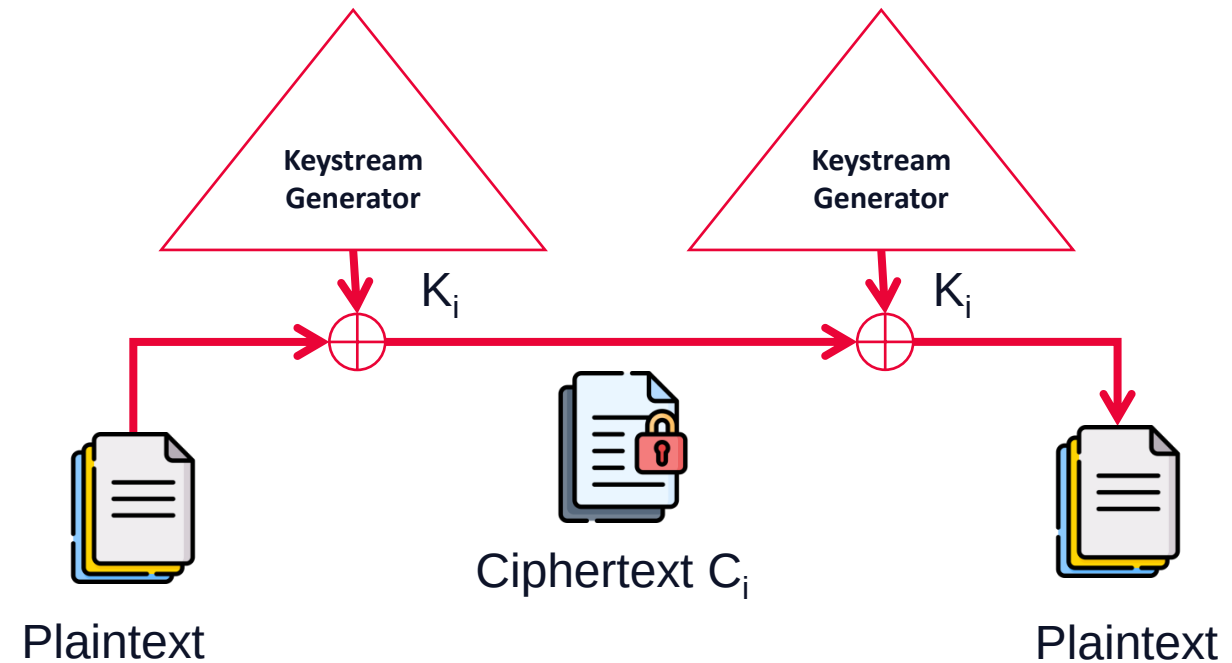
- Ways to combine block algorithms to get new algorithms
- Increase security by others ways then designing a new algorithm
- Combining techniques
 - *Multiple encryption* – using an algorithm to encrypt the same plaintext multiple times with multiple keys
 - *Cascading* - using different algorithms
- **Encryption with 2 keys:**
 - Encrypts with 2 different keys
 - $C = E_{K1}(E_{K2}(P)); P = D_{K1}(D_{K2}(C));$
- **Encryption on Davies-Price method:**
 - CBC variant
 - $C_i = E_{K1}(P_i) E_{K2}(C_{i-1}); P_i = D_{K1}(C_i) D_{K2}(C_{i-1});$
- **Triple encryption with 2 keys**
 - $C = E_{K1}(D_{K2}(E_{K1}(P))); P = D_{K1}(E_{K2}(D_{K2}(C)));$
 - EDE – encrypt-decrypt-encrypt
- **Triple encryption with 3 keys**
 - $C = E_{K3}(D_{K2}(E_{K1}(P))); P = D_{K1}(E_{K2}(D_{K3}(C)));$



Symmetric Cryptographic Systems

Stream Ciphers

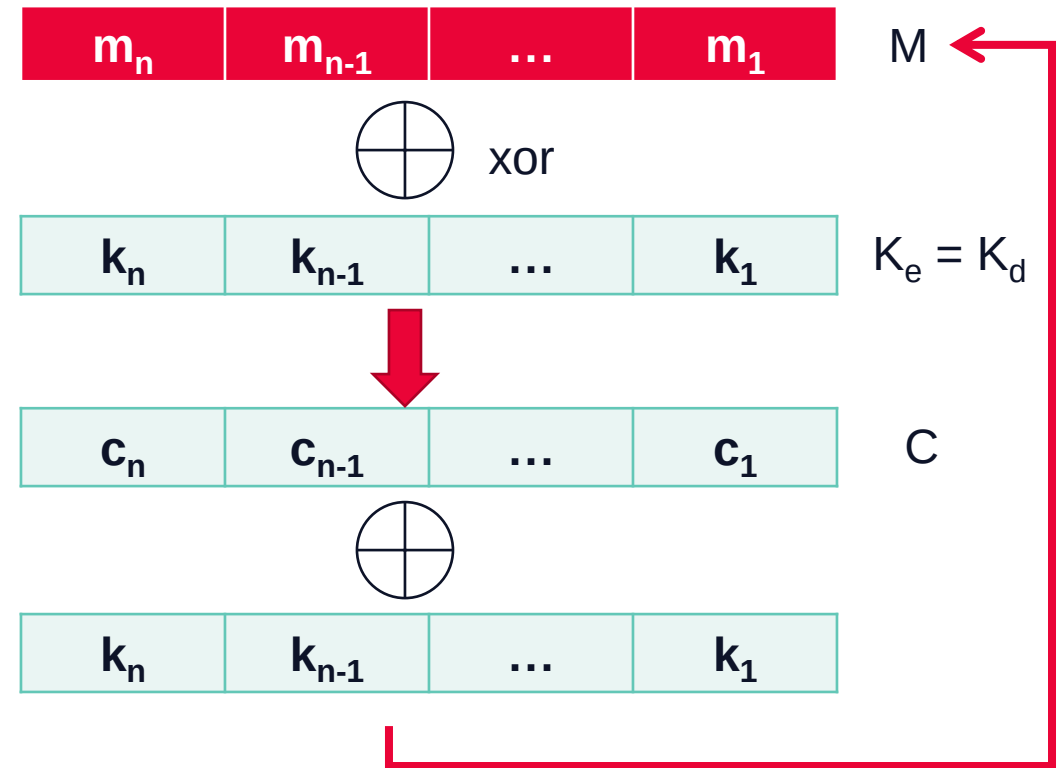
- Convert plaintext to ciphertext 1 bit/byte at a time
- Are based on XOR to encrypt the data (Vernam example)
- Security depends entirely on the insides of keystream generator.
- some design considerations for key generator are:
 - long period with no repetitions
 - statistically random
 - depends on large enough key
 - large linear complexity
- properly designed, can be as secure as a block cipher with same size key
- but usually simpler & faster



Symmetric Cryptographic Systems

One Time Pad (OTP)

- Each bit/character from the plaintext is encrypted by a modular addition (XOR) with a bit/character from a secret random key
- patented by Vernam (1917)
- secure till our days **with a random key as large or greater than the plaintext which is not reused**
- advantages:
 - impossible to break if the key is secured
 - low complexity
- disadvantages:
 - the key length is the message length
 - the key becomes a message that must be transmitted



Symmetric Cryptographic Systems

Stream Ciphers

- RC4 (1987) – used in HTTPS, 802.11 WEP
- CSS (Content Scrambling System)– implements 2 LFSRs to encrypt DVDs
- A5 – for GSM based on 3 LFSRs (19, 22 and 23 bits)
- E0 – used in Bluetooth based on 4 LFSRs (25, 31, 33 and 39 bits)



Symmetric Cryptographic Systems

Stream Ciphers

Synchronous stream cipher:

- the keystream is generated independent of the message stream
- Key Auto-Key (KAK)
- the two keystream generators (at encryption and decryption) are synchronized and generate the same output -> are deterministic -> are periodic
- Except for OTP, all keystream generators are periodic

Self-synchronizing stream ciphers:

1. each message begins with a random header n bits long;
2. the header is encrypted, transmitted, and then decrypted
3. the decryption will be incorrect, but after those n bits both keystream generators will be synchronized.



Symmetric Cryptographic Systems

Attacks on Stream Ciphers

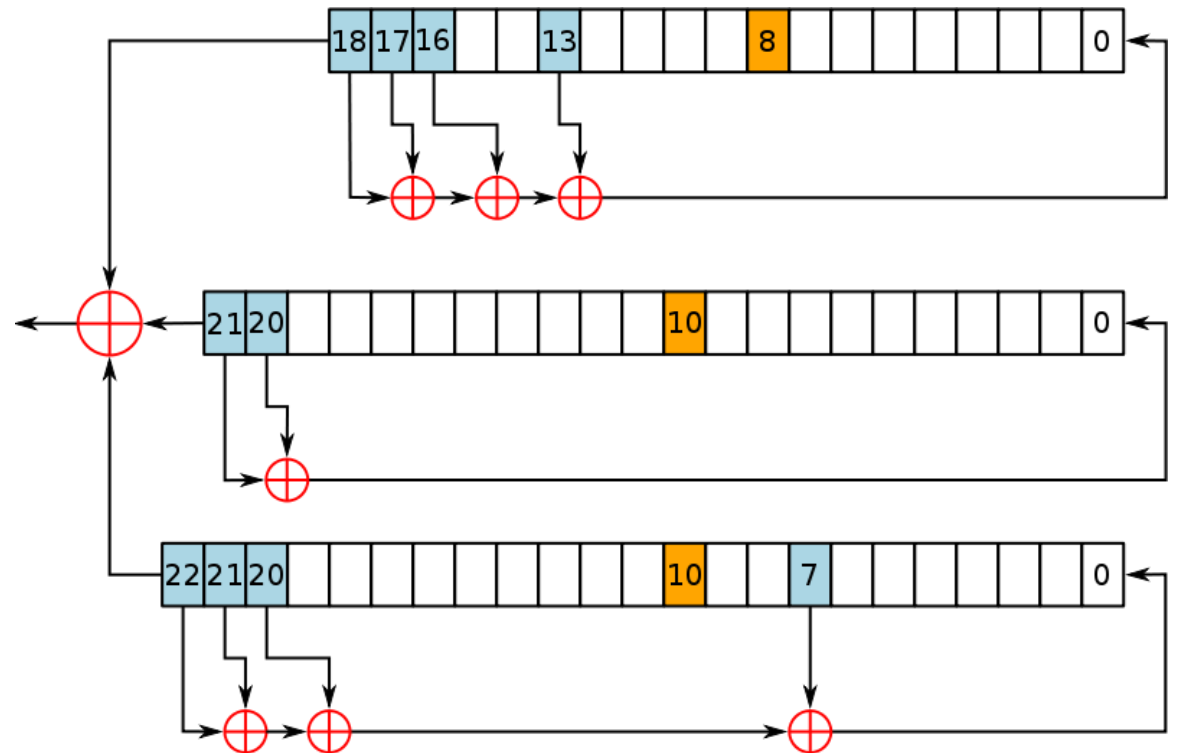
- Vulnerable to scenarios when the **same key is used more than once**
- The key is short enough
- The keys are predictable based on PRNG vulnerabilities
- Examples:
 - [Project Venona](#) (1943 – 1980)
 - [MS-PPTP](#) (Windows NT) - Point-to-Point Tunneling Protocol



Stream Ciphers

A5

- Stream cipher used to encrypt GSM – the link between from the phone to base station
- Is a French design cipher
- Consists in 3 LFSRs (their length is 19, 22 and 23)
- It is very efficient and pass all known statistical test
- <http://en.wikipedia.org/wiki/A5/1>



<http://en.wikipedia.org/wiki/A5/1>

Stream Ciphers

RC4

- a proprietary cipher owned by RSA DSI (designed in 1987)
- another Ron Rivest design, simple but effective
- variable key size, byte-oriented stream cipher
- RC4 is probably the most widely used stream cipher (web SSL/TLS, wireless WEP)
- key forms random permutation of all 8-bit values
- uses that permutation to scramble input info processed a byte at a time
- the period of the cipher is overwhelmingly likely to be greater than 10^{100} . Eight to sixteen machine operations are required per output byte, and the cipher can be expected to run very quickly in software
- RC4 was kept as a trade secret by RSA Security, but in September 1994 was anonymously posted on the Internet on the Cypherpunks anonymous remailers list



Stream Ciphers

RC4

Key Schedule

- starts with an array S of numbers: $0..255$
- use key to well and truly shuffle , the total number of possible states is $256!$
- S forms **internal state** of the cipher
- initializes the state S to the numbers $0..255$, and then walks through each entry in turn, using its current value plus the next byte of key to pick another entry in the array, and swaps their values over
- does this 256 times

for $i = 0$ to 255 do

$S[i] = i$

$T[i] = K[i \bmod \text{keylen}]$

$j = 0$

for $i = 0$ to 255 do

$j = (j + S[i] + T[i]) \bmod 256$

swap ($S[i]$, $S[j]$)



Stream Ciphers

RC4

Encryption

- encryption continues shuffling array values
- sum of shuffled pair selects "stream key" value from permutation
- XOR $S[t]$ with next byte of message to en/decrypt

$i = j = 0$

for each message byte $M[i]$

$i = (i + 1) \pmod{256}$

$j = (j + S[i]) \pmod{256}$

swap($S[i]$, $S[j]$)

$t = (S[i] + S[j]) \pmod{256}$

$C[i] = M[i] \text{ XOR } S[t]$

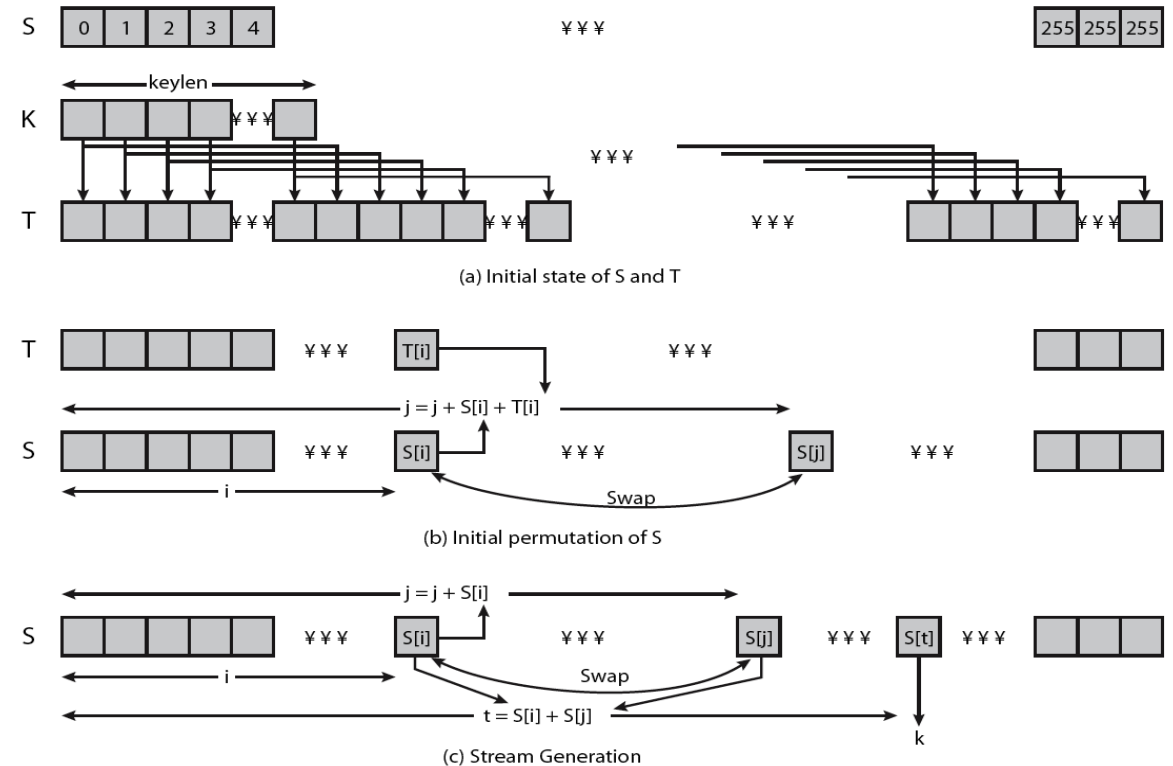


Stream Ciphers

RC4 Security

Encryption

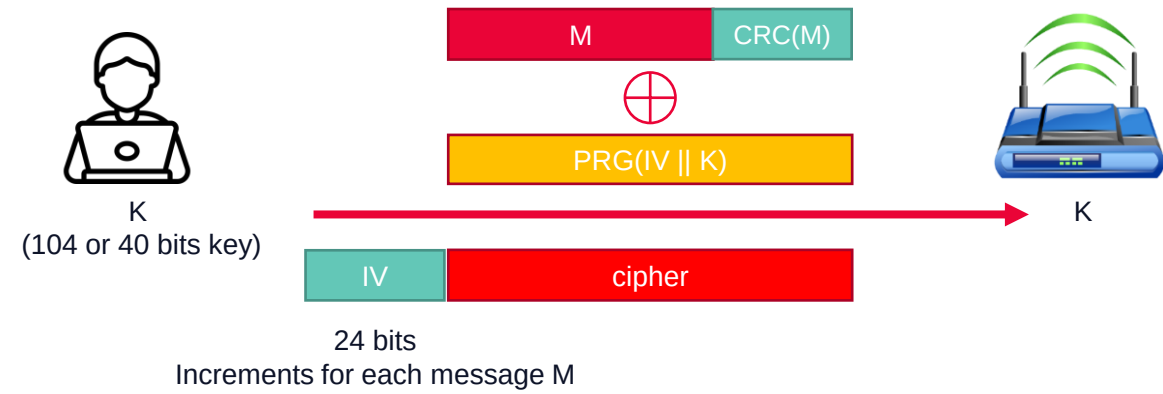
- claimed secure against known attacks
 - have some analyses, none practical
- result is very non-linear
- since RC4 is a stream cipher, must **never reuse a key**
- have a concern with WEP, but due to key handling rather than RC4 itself



Stream Ciphers

WEP problem

- 802.11 standard introduced in 1999 - Wired Equivalent Privacy (**WEP**)
- Used a key of 10 or 26 hexadecimal digits
- Standard 64-bit WEP uses a 40-bit key (also known as WEP-40), which is concatenated with a **24-bit initialization vector (IV)** to form the RC4 key
- User's 5 chars passwords reduced more the key strength
- because RC4 is a stream cipher, the same traffic key must never be used twice



Stream Ciphers

WEP problem

The problem

- the purpose of an IV, which is transmitted as plain text, is to prevent any repetition, but a 24-bit IV is not long enough to ensure this on a busy network
- for a 24-bit IV, there is a 50% probability the same IV will repeat after 5000 packets
- It is possible to perform the attack with a personal computer, off-the-shelf hardware and freely available software such as [aircrack-ng](https://aircrack-ng.org/) to crack *any* WEP key in minutes (now less than a minute)

The result

- In 2001 they were able to recover the key in 10^6 frames
- Today 40000 frames are sufficient
- On some cards the IV is reset to 0 after a power cycle
- Because IV is generated incrementally, it repeats after 2^{24} values (between 5 and 7 hours on a busy network)
- The keys are related (differ only in 24 bits)



Another protocol problem

WPA2 problem

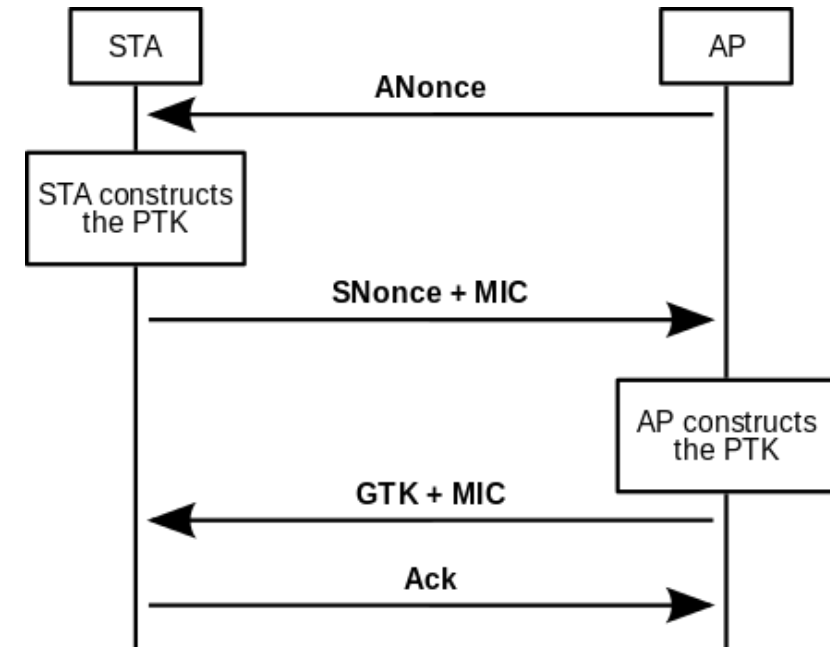
- [Wi-Fi Protected Access II \(WPA2\)](#) is a security algorithm for IEEE 802.11i wireless networks standard introduced in 2004
- replaces [Wired Equivalent Privacy \(WEP\)](#), which was shown to have security vulnerabilities
- Wi-Fi Protected Access (WPA) has been a temporary solution to WEP
- uses the [Advanced Encryption Standard \(AES\)](#) block cipher, whereas WEP and WPA used the [RC4](#) stream cipher
- On October 16, 2017, researchers revealed the KRACK (Key Reinstallation AttaCKs) exploit, <https://www.krackattacks.com/>
- Allows attackers eavesdrop on traffic between computers and wireless access points
- The attack **DOES NOT** recover the password of the Wi-Fi network



Another protocol problem

WPA2 problem

- The attack is against the 4-way handshake of the WPA2 protocol by replaying cryptographic handshake messages
- The 4-way handshake protocol
 - It allows clients and APs to check their pre-shared key
 - Generates a fresh encryption key that will be used to encrypt all subsequent traffic
- The client will reinstall an already-in-use key and will reset its incremental transmit packet number (i.e. nonce) and receive packet number (i.e. replay counter)
- For Android (above 6.0) and Linux the client can be tricked to install an all-zero encryption key

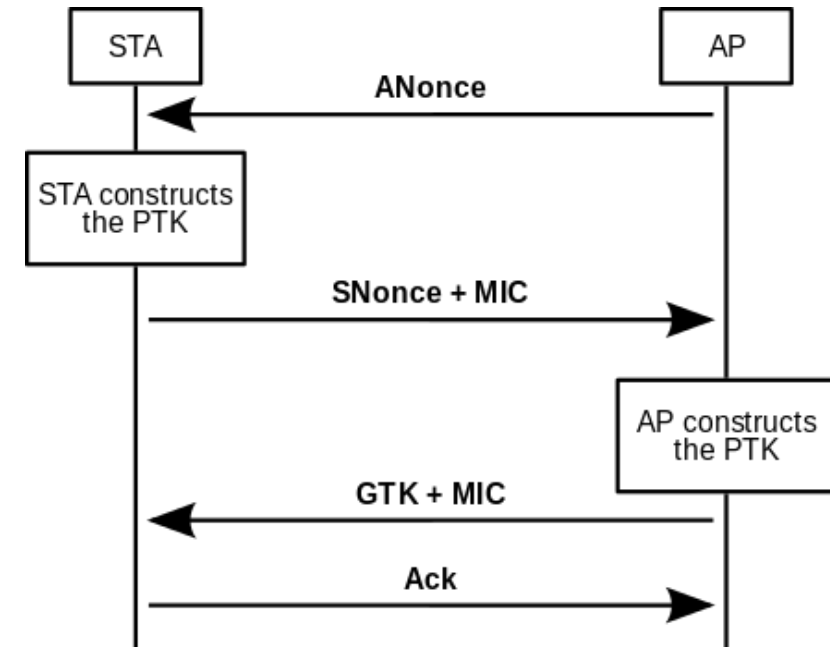


[The four-way handshake in 802.11i](#)

Another protocol problem

WPA2 problem

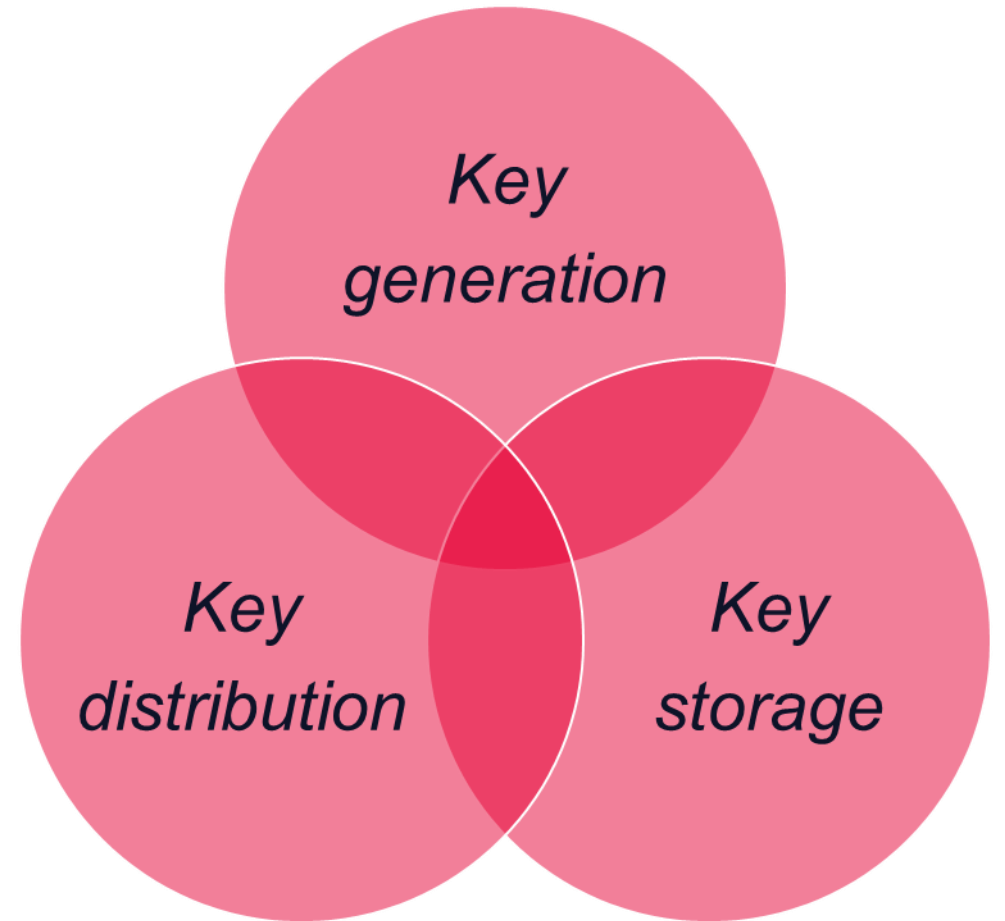
- The attacker can
 - inject and manipulate data (to inject ransomware or other malware into websites that you visit)
 - steal sensitive information such as credit card numbers, passwords, chat messages, emails, photos, and so on
 - read information that was previously assumed to be safely encrypted
- As with all stream ciphers you should NOT reuse the same key again
- ***the same encryption key is used with nonce values that have already been used in the past***
- The client will reuse keystream when encrypting packets and for known content it will be easy for an attacker to determine it



[The four-way handshake in 802.11i](#)

Symmetric Cryptographic Systems

The only thing to protect is the key



Symmetric Cryptographic Systems

Key generation

What is the key-length ?

- it depends on what is the value of the data
- it depends on the use span of the cryptographic algorithm

Do not reduce the keyspace

- a 64-bit key with only lowercase-letters -> 2^{40} possible keys
- random keys – for a 64 bits key, every possible 64-bit value must be equally likely
- pass phrases – take entire phrases and convert them in keys using hash functions
- ANSI X9.17 standard for random key generation



Symmetric Cryptographic Systems

Key storage

Lifetime of keys:

- the longer a key is used, the greater the chance that it will be compromised
- the longer a key is used, the greater the loss if the key is compromised
- the longer a key is used, the greater the temptation for someone to break it
- cryptanalysis gives better results with more ciphertext encrypted with the same key

Use:

- **Hardware Security Modules (HSMs):** physical devices specifically designed to manage, generate, and store cryptographic keys securely
- **Trusted Platform Module (TPM):** hardware-based security feature included in many modern computers, designed to secure hardware through integrated cryptographic keys
- **Secure Elements and Smart Cards:** Specialized hardware components like secure elements and smart cards designed to securely store cryptographic keys
- **Key Wrapping:** Encrypting the encryption keys themselves with another key (known as a key-encryption key or KEK) before storing them



Symmetric Cryptographic Systems

Hardware Security Modules (HSMs)

- Are dedicated physical devices used to provide a secure environment for performing cryptographic operations, such as encryption, decryption, key generation, and key management. These modules are specifically designed to safeguard and manage sensitive data
- **Key Management:** HSMs generate, store, and manage cryptographic keys securely, protecting them from unauthorized access or tampering.
- **High Security:** They provide strong physical security mechanisms, such as tamper-evident seals, tamper-resistant hardware, and secure key storage.
- **Compliance:** HSMs are often used in industries that require strict security and compliance, such as financial services, healthcare, and government sectors. They help meet regulatory standards like FIPS 140-2/3.
- **Performance:** HSMs are designed to perform cryptographic operations efficiently and securely at high speeds, even for high-demand applications like large-scale transactions or secure communications.
- **Application Integration:** HSMs can integrate with applications requiring cryptographic security, such as SSL/TLS, digital signatures, PKI (Public Key Infrastructure), and encryption systems.

Symmetric Cryptographic Systems

Hardware Security Modules (HSMs)

- **Thales Luna HSM** - One of the most widely used HSMs, designed for general-purpose cryptographic needs and integrated with several cloud services and enterprise applications.
- **AWS CloudHSM** - A cloud-based HSM service provided by Amazon Web Services (AWS). It allows organizations to generate and manage encryption keys in the cloud while maintaining control over their sensitive data
- **IBM 4765/4767 HSM** - IBM's HSMs are known for their high levels of security and FIPS 140-2 Level 4 certification. They are commonly used in financial institutions and government agencies.
- **Microsoft Azure Dedicated HSM** - A cloud-based HSM solution provided by Microsoft Azure, enabling secure key management in a cloud environment while maintaining full control of the keys
- **Yubico YubiHSM** - A compact and cost-effective HSM designed for smaller-scale applications or organizations needing strong cryptographic security for servers and cloud environments.



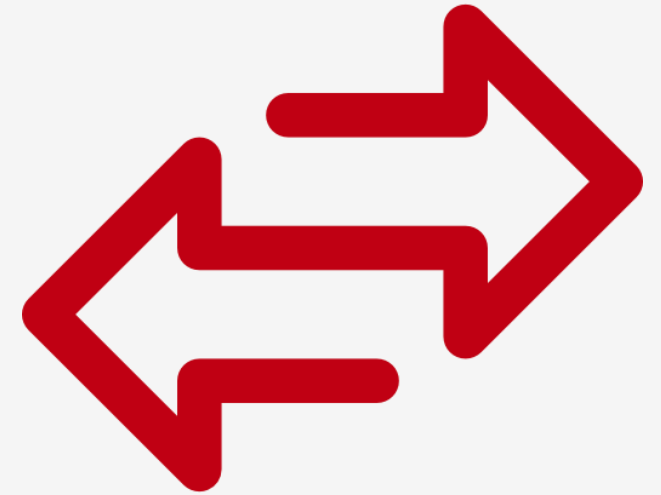
<https://www.yubico.com/product/yubihsm-2/>

5. Asymmetric Cryptographic Systems



Asymmetric Cryptographic Systems

- aka public key systems
- one of the biggest breakthrough in modern Cryptography
- designed to solve the problem of symmetric systems, the keys distribution
- Objective to fix vulnerabilities of symmetric encryption algorithms
 - **key distribution**
 - **non-repudiation**
 - **authentication**
- Requires:
 - A pair of keys
 - A strong encryption algorithm



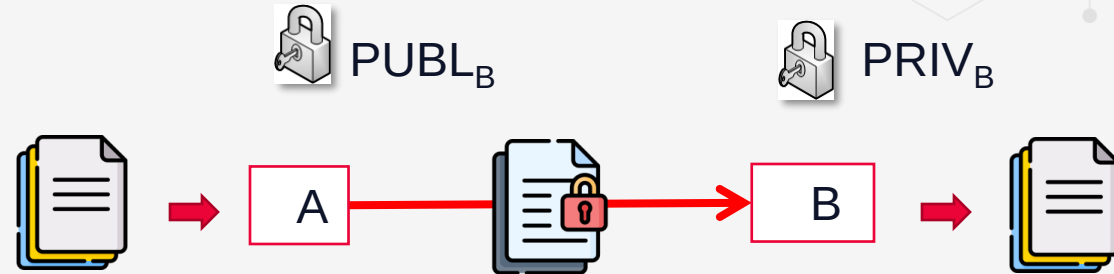
Asymmetric Cryptographic Systems

History

- 1976 – idea of public key encryption system, Diffie and Hellman (key distribution design)
- Probably most significant advance in the 3000-year history of cryptography
- Use distinct keys of encryption and decryption (but dependent on each other)
- It is impossible to extract a key from the other key
- One of the keys is made public, being available for everyone wishing to send an encrypted message.
- The receiver owning the 2nd key, can decrypt and use the message
- The public key technique is also used in digital (electronic) signature

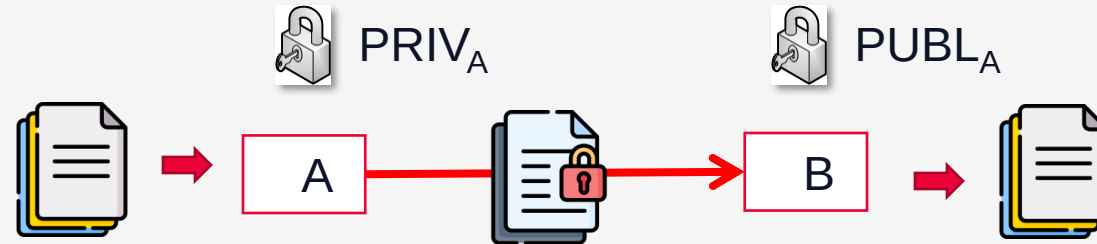
Asymmetric Cryptographic Systems

Confidentiality

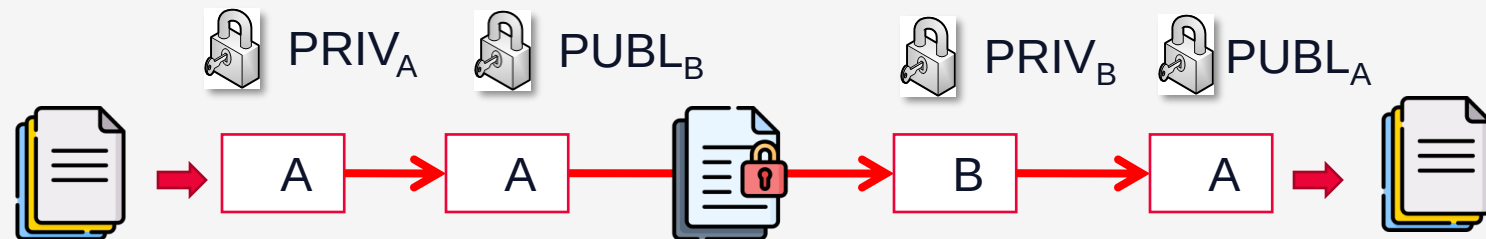


Digital signature course

Authentication (digital signature)



Authentication & Confidentiality



ENCRYPT

DECRYPT

Asymmetric Cryptographic Systems

Algorithms

- **RSA** (Rivest-Shamir-Adleman), **Rabin** – factorization problem
- **Diffie-Hellman**, **El Gamal**, **DSS**, **LUC**, **XTR** – based on discrete logarithms
- **BrandStorm**, **PIEPRZYK** – based on equations in finite fields, work with polynomial rings
- **Miller and Kobitz** – based on elliptic curves
- **Rivest-Chor**, **MH** – Merkle-Hellman, variants:
 - iterative variant
 - with additive trapdoor
 - with multiplicative trapdoor
- **GS** (Graham-Shamir), **SH** (Shamir) – based on Knapsack problem
- **NTRU** – based on lattice operations ([http://en.wikipedia.org/wiki/Lattice_\(order\)\)](http://en.wikipedia.org/wiki/Lattice_(order))))
- **Elliptic curves** (https://en.wikipedia.org/wiki/Elliptic_curve)



Asymmetric Cryptographic Systems

El Gamal

- Presented for the 1st time in 1985 by Taher ElGamal
- Derived from Diffie and Hellman's keys distribution schema
- Funds its cryptographic strengths on difficulty to calculate logarithms in large Galois fields, discrete logarithm problem

Key generation:

- It is generated a large prime number and an α generator for Z_p group of modulo p integers
- It is selected a random number a , with $1 \leq a \leq p-2$ and it is calculated $\alpha^a \bmod p$
- Public key is (p, α, α^a)
- Private key is a



Asymmetric Cryptographic Systems

El Gamal

Encryption

- It is obtained the public key (p, α, α^a)
- It is considered message m represented as integer value in $\{0, 1, \dots, p-1\}$
- It is chosen a random integer value k , with $1 \leq k \leq p - 2$.
- It is calculated $\gamma = \alpha^k \bmod p$ si $\delta = m \cdot (\alpha^a)^k \bmod p$
- Cipher is $c = (\gamma, \delta)$

Decryption

- Private key is a
- Cipher is $\mathbf{c} = (\gamma, \delta) = (\alpha^k \bmod p, m \cdot (\alpha^a)^k \bmod p)$
- Using key a it is calculated $\gamma^{p-1-a} \bmod p$ because $\gamma^{p-1-a} = \gamma^{-a} = \alpha^{-ak}$
- m is obtained calculating $(\gamma^{-a}) \cdot \delta \bmod p$
- It is possible because $\gamma^{-a} \cdot \delta \equiv \alpha^{-ak} m \alpha^{ak} \equiv m \pmod{p}$.



Asymmetric Cryptographic Systems

RSA (Rivest-Shamir-Adleman)

- Created by 3 researchers from MIT (Massachusetts Institute of Technology)
- “de facto” standard in digital signature field and of encryption with public keys
- The safest method to secure and authenticate commercially available
- Is based on the present quite impossibility to factorize very large integer numbers
- The encryption/decryption functions are of exponential type, where the exponent is the key and the calculation is made in the ring of rest modulo n classes
- Can be used for electronic signature as well as encryption/decryption
- Uses large integers (eg. 1024 bits)
- Security due to cost of factoring large numbers, factorization takes $O(e \log n \log \log n)$ operations (hard)



Asymmetric Cryptographic Systems

RSA (Rivest-Shamir-Adleman)



The cryptographic system's parameters are:

- **p** and **q** are 2 very big prime numbers (secret, eventually known only by the owner, of minimum 1024 bits).
- The module **n**, made public, is **$n = p * q$** .
- The Euler indicator **$\varphi(n) = (p-1) * (q-1)$** , impossible to be determined by an attacker, because its prime factors of **n** (**p** and **q**) are not known.
- The secret key, **PRIV**, chosen as being a big integer number relatively prime with **$\varphi(n)$** , preferable in the period **$[\max(p, q) + 1, n - 1]$** .
- The public key, **PUB**, an integer calculated by a version of the algorithm of Euclid, as being inverse of **PRIV** modulo **$\varphi(n)$** ; **$PUB = \text{inv}(\text{PRIV}, \varphi(n))$** .
- **M** the document in electronic form (the file), the message, the object
- **H(M)**, the digest of the document, calculated with a hash dispersion function



Asymmetric Cryptographic Systems

RSA (Rivest-Shamir-Adleman)

RSA configuration:

1. Are generated 2 big prime numbers p, q ;
2. It is calculated $n = p * q$ and $\varphi(n) = (p - 1)(q - 1)$;
3. It is chosen a random number b , ($1 < b < \varphi(n)$) with $\gcd(b, \varphi(n)) = 1$
4. It is calculated $a = b^{-1} \bmod (\varphi(n))$
5. Make public n and a . Keep private b .



Asymmetric Cryptographic Systems

RSA (Rivest-Shamir-Adleman)

RSA encryption:

1. The intention is to send message M (simplification 1 byte)
2. It is calculated $C = M^b \bmod n$
3. It is send C

RSA decryption:

1. It is received message C (simplification 1 byte)
2. It is calculated $M = C^a \bmod n$

Decryption is possible because

$$\begin{aligned} M &= C^a \bmod n = (M^b)^a \bmod n = M^{ba} \bmod n = \\ &= M^{b \bmod n} \bmod n = M^1 \bmod n = M \end{aligned}$$



Asymmetric Cryptographic Systems

RSA (Rivest-Shamir-Adleman) Example

1. Select primes: $p=17$ & $q=11$
2. Compute $n = pq = 17 \times 11 = 187$
3. Compute $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; choose $e=7$
5. Determine d : $de \equiv 1 \pmod{160}$ and $d < 160$ Value is $d=23$ since $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish public key $\mathbf{PU} = \{23, 187\}$
7. Keep secret private key $\mathbf{PR} = \{7, 187\}$

sample RSA encryption/decryption is:

given message $M = 88$ (nb. $88 < 187$)

encryption:

$$C = 88^7 \pmod{187} = 11$$

decryption:

$$M = 11^{23} \pmod{187} = 88$$



Asymmetric Cryptographic Systems

RSA (Rivest-Shamir-Adleman)

- function $E_k(x) = x^b \bmod n$ is not reversible because of its complexity
- Secret trapdoor owned by the receiver for decryption is factorization $n = p \cdot q \rightarrow$ determines $\varphi(n) \rightarrow$ determines a
- p and q are big numbers
- The effort of encryption/decryption depends on number of bits, k , of n ($k = \log_2(n) + 1$)
- Sum of 2 numbers of k bits - $O(k)$;
- Product of 2 numbers of k bits - $O(k^2)$
- Number $x \cdot y \bmod n$ - $O(k^2)$
- Modular exponentiation $x^y \bmod n$ - $O(k^2)$

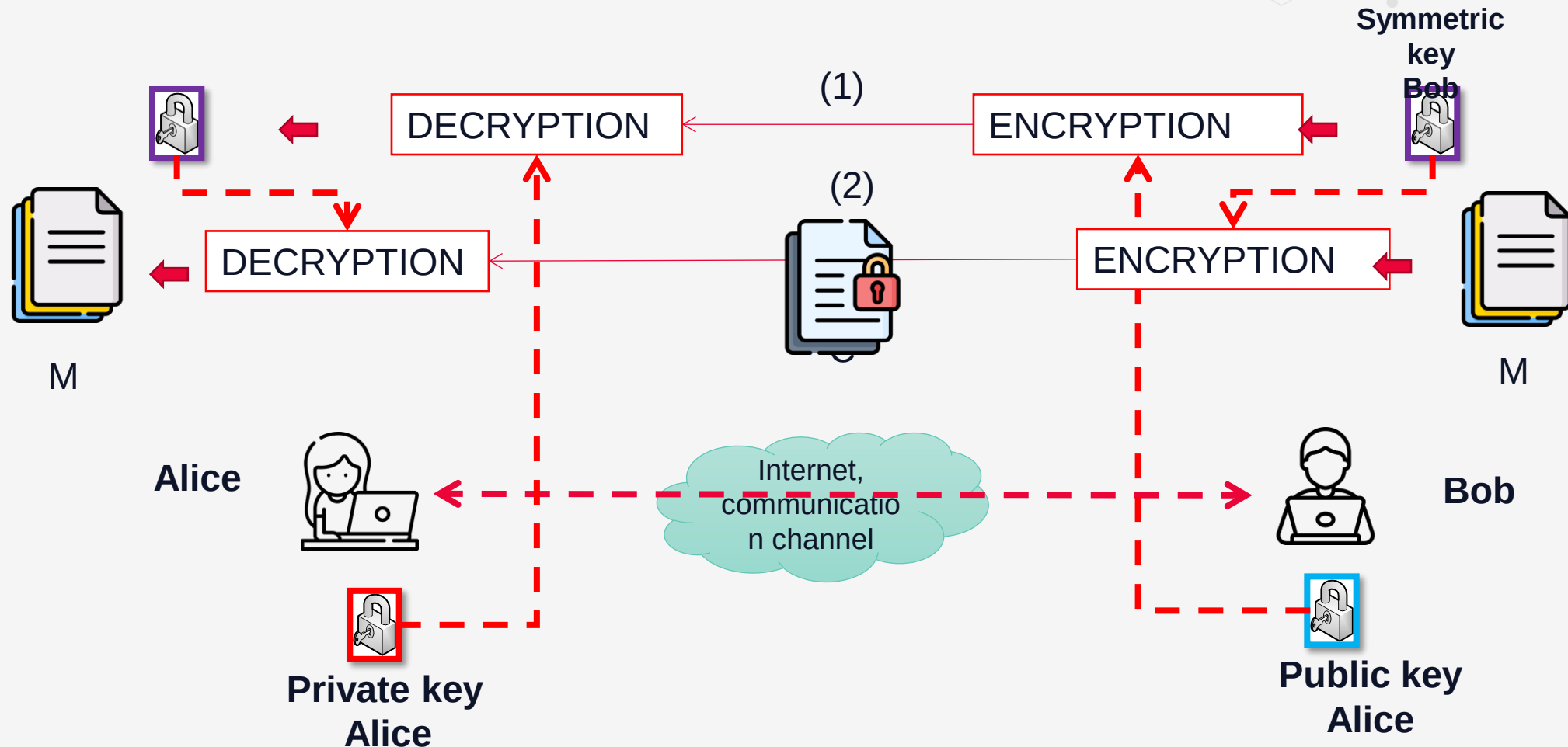
RSA is approximately 1000 times slower than DES for a hardware implementation and approximately 100 times for a software implementation

For efficiency reasons hybrid encryption is used



Asymmetric Cryptographic Systems

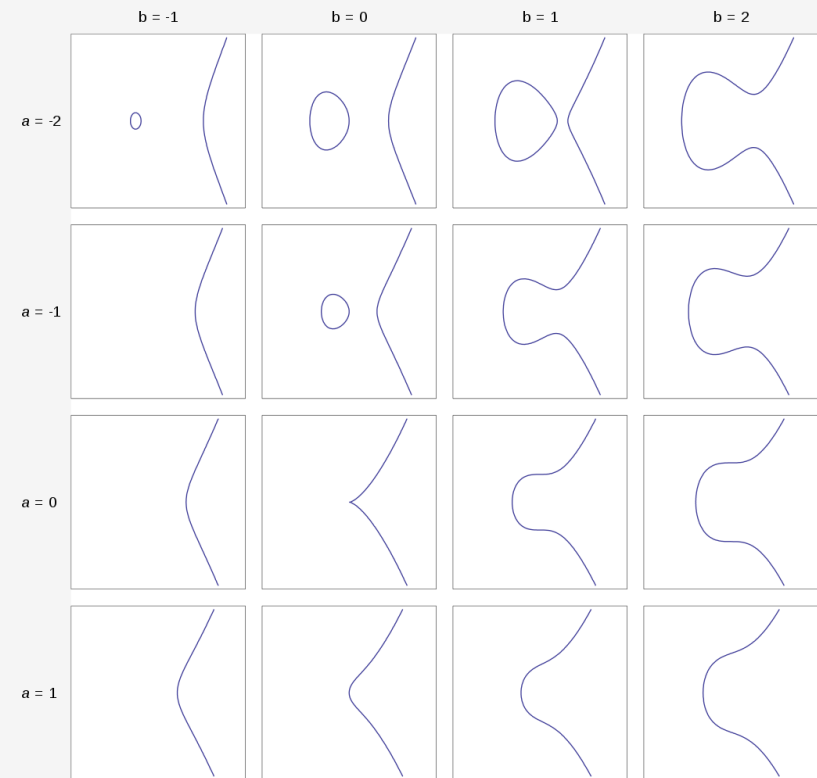
Hybrid encryption systems



Elliptic Curves Cryptography

The next phase

- In mathematics, an **elliptic curve** is a plane algebraic curve defined by an equation of the form that has no cusps or self-intersections
 - $Y^2 = X^3 + aX + b$
 - Multiplying a point on the curve by a number will produce another point on the curve
 - It is very difficult to find what number was used, even if you know the original point and the result.
 - Equations based on elliptic curves are relatively easy to perform, and extremely difficult to reverse
- ECC generates keys using the properties of the elliptic curve equation (RSA uses product of very large prime numbers)
- The use of elliptic curves in cryptography was suggested independently by [Neal Koblitz and Victor S. Miller](#) in 1985
- Elliptic curve cryptography algorithms entered wide use around 2004
- <http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-8951-CryptoAuth-RSA-ECC-Comparison-Embedded-Systems-WhitePaper.pdf>

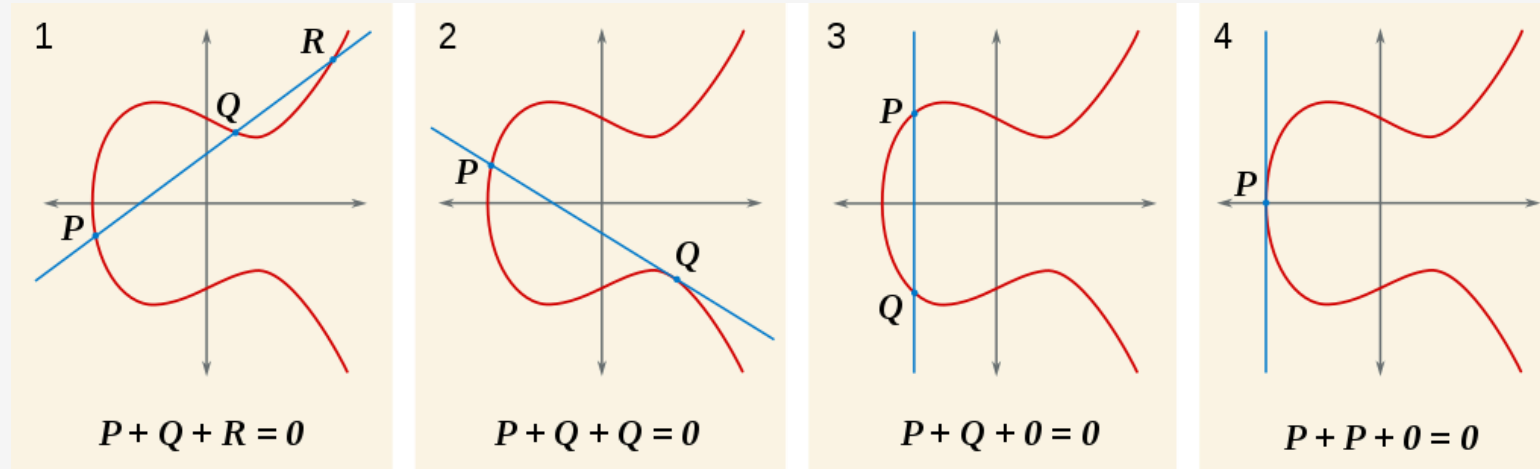


https://en.wikipedia.org/wiki/Elliptic_curve



Elliptic Curves Cryptography

- The curve is symmetrical about the x-axis; any point P , has an opposite point, $-P$
- If P and Q are two points on the curve, then we can uniquely describe a third point, $P + Q$, in the following way. First, draw the line that intersects P and Q . This will generally intersect the cubic at a third point, R . We then take $P + Q$ to be $-R$, the point opposite R



https://en.wikipedia.org/wiki/Elliptic_curve

Elliptic Curves Cryptography

Performance

- [Lenstra, Kleinjung and Thome](#) defined in 2013 the concept of "Global Security." - compute how much energy is needed to break a cryptographic algorithm and compare that with how much water that energy could boil.
- Breaking a 228-bit RSA key requires less energy than it takes to boil a teaspoon of water. Comparatively, breaking a 228-bit elliptic curve key requires enough energy to boil all the water on earth. For this level of security with RSA, you'd need a key with 2,380-bits

Minimum size (bits) of Public Keys			Key Size Ratio	
Security (bits)	DSA / RSA	ECC	ECC to RSA / DSA	Valid
80	1024	160-223	1:6	Until 2010
112	2048	224-255	1:9	Until 2030
128	3072	256-383	1:12	Beyond 2031
192	7680	384-511	1:20	
256	15360	512+	1:30	

<https://www.ssl2buy.com/wiki/rsa-vs-ecc-which-is-better-algorithm-for-security>



Elliptic Curves Cryptography



- The [Elliptic Curve Diffie-Hellman](#) (ECDH) key agreement scheme is based on the [Diffie-Hellman](#) scheme,
- The Elliptic Curve [Integrated Encryption Scheme](#) (ECIES), also known as Elliptic Curve Encryption Scheme,
- The [Elliptic Curve Digital Signature Algorithm](#) (ECDSA) is based on the [Digital Signature Algorithm](#),
- The [Edwards-curve Digital Signature Algorithm](#) (EdDSA) is based on [Schnorr signature](#) and uses [twisted Edwards curves](#),
- The [ECMQV](#) key agreement scheme is based on the [MQV](#) key agreement scheme

Controversy

- [NIST Special Publication 800-90](#) (.pdf) on DRBGs, or "Deterministic Random Bit Generators."
 - Describes 4 PRNGs, one based on ECC
 - [Dual_EC_DRBG](#) (**Dual Elliptic Curve Deterministic Random Bit Generator**) – contains a lost of constants which are not explained ([CRYPTO 2007 paper](#))
 - If you know the secret numbers, you can predict the output of the random-number generator after collecting [just 32 bytes of its output](#). By monitoring the [TLS](#) internet encryption connection you can break the protocol security
- https://www.schneier.com/essays/archives/2007/11/did_nsa_put_a_secret.html



Symmetric vs. Asymmetric

Symmetric algorithms

Advantages:

- High encryption power
- Easy to implement
- Relatively short keys (128,192, 256 bits)
- Can be composed into new cryptographic systems (multiple, sequential – 3DES)
- Tested into practice
- Used to generate pseudo-random numbers or hash functions

Disadvantages :

- Secret key known by both parts
- Long message's length is a vulnerability
- In distributed systems, key management is an issue
- Key transmission is a security problem
- Frequent key modification leads to rerun transmission process



Symmetric vs. Asymmetric

Asymmetric algorithms

Advantages:

- Simple solution to send data through unsure channels
- Easy to implement
- Secret key found in one place
- Define the base for distributed applications: electronic signature, authentication, electronic payments
- Key pair (public and private) can be modified without too much effort

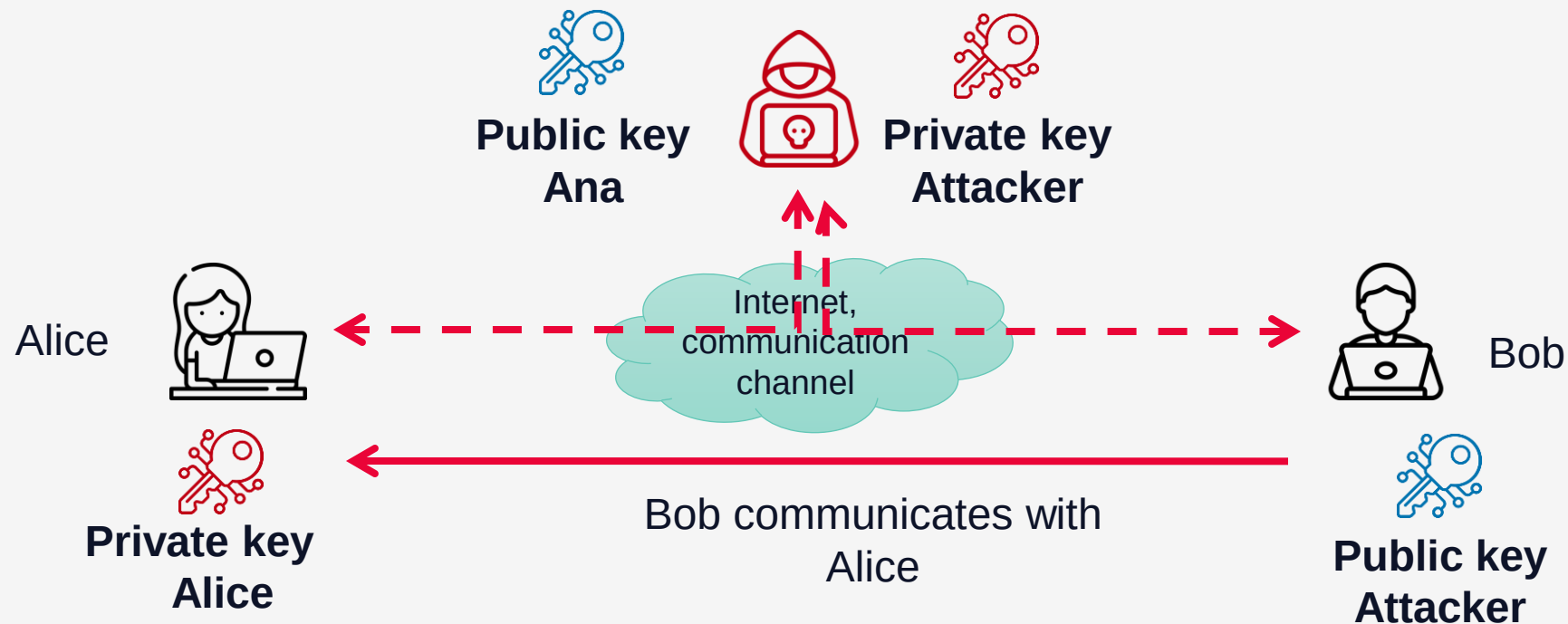
Disadvantages :

- Low encryption power
- Need big length keys
- Does not guarantee absolute security
- Level of security depends on implementation
- Anyone can make encryptions=> can find certain weak points that allow message decryption
- **No authentication, very vulnerable to man-in-the-middle type attacks**



Symmetric vs. Asymmetric

Man-in-the-middle attack on no authentication asymmetric system



Homomorphic encryption

Next major step

- *A form of encryption that allows computations to be performed on encrypted data without first having to decrypt it. The resulting computations are left in an encrypted form which, when decrypted, result in an output that is identical to that produced had the operations been performed on the unencrypted data,*
https://en.wikipedia.org/wiki/Homomorphic_encryption
- Homomorphic encryption is particularly important for applications like secure cloud computing, privacy-preserving data analysis, and encrypted database queries, where data confidentiality must be maintained during computation

Types of Homomorphic Encryption:

- **Partially Homomorphic Encryption (PHE):** Supports either addition or multiplication on encrypted data, but not both.
- **Somewhat Homomorphic Encryption (SHE):** Allows a limited number of additions and multiplications on ciphertexts.
- **Fully Homomorphic Encryption (FHE):** Supports both addition and multiplication on encrypted data without any restrictions, allowing arbitrary computations on encrypted data.



Homomorphic encryption

Common Homomorphic Encryption Algorithms

1. Paillier Cryptosystem (Partially Homomorphic):

1. **Supports:** Addition.
2. **Use Cases:** Voting systems, private data aggregation.

2. RSA (Partially Homomorphic):

1. **Supports:** Multiplication.
2. **Use Cases:** Limited operations for multiplicative encryption applications.

3. Brakerski-Gentry-Vaikuntanathan (BGV) Scheme (Somewhat/Fully Homomorphic):

1. **Supports:** Addition and multiplication (with constraints).
2. **Use Cases:** Secure data analysis, machine learning on encrypted data.

4. Gentry's Scheme (First Fully Homomorphic Encryption Scheme):

1. Developed by Craig Gentry in 2009, this was the first demonstration of a fully homomorphic encryption system.
2. **Supports:** Arbitrary computations.
3. **Use Cases:** Complex computations on encrypted cloud data.



Cryptanalysis



- Encrypt analysis is the art of decrypting messages (files, data structures, communications) encrypted without knowing the decryption key
- The encryption method or algorithm **MUST NOT** be secret, only the key. Cryptographic security means key protection.
- **The security of a cryptosystem should rest in the key**

Objectives:

- recover the key, not just the plaintext message for a ciphertext

Strategies:

- **Brute-force attack:** try every possible key
- **Cryptanalytic attack:** finds the key based on knowledge on the algorithm, plaintext and cipher text
- **Backdoor functions** - a deliberate mechanism that is added to a cryptographic algorithm (e.g., a key pair generation algorithm, digital signing algorithm, etc.) or operating system, for example, that permits one or more unauthorized parties to bypass or subvert the security of the system in some fashion
([https://en.wikipedia.org/wiki/Backdoor_\(computing\)](https://en.wikipedia.org/wiki/Backdoor_(computing)))



Cryptanalysis

Attacks

- **Ciphertext-only attack**
 - The analysis of statistics frequencies (certain characters appear with medium frequency in different languages)
 - Exhaustive search of the keys part of the encryption
- **Known-plaintext attack:**
 - The attacker knows decrypted text for certain cryptogram parts – brute-force attack
 - Linear encrypt-analysis against block ciphers (Matsui, 1994)
- **Chosen plaintext attack**
 - The attacker can choose plaintext and obtain the ciphertext
- **Chosen ciphertext attack:**
 - The attacker can choose ciphertext and obtain the plaintext
- **Chosen text attack:**
 - The attacker can choose plaintext/ciphertext to encrypt/decrypt



Cryptanalysis

Attacks

Brute-force attack:

- Trying every possible key
- It is needed a small amount of ciphertext and the corresponding plaintext
- The attack complexity = 2^n , where n is the key length in bits
- Based on a birthday-paradox, there is a 50% probability to find the solution in $2^{n/2}$ tries

Brute-force attack with a **million/second** key search computer:

- for a 56 bits key – 2285 years
- for a 64 bit key – 585000 years
- for a 128 bit key – 10^{25} years

An efficient brute-force attack is implemented in a parallel architecture (Ex The Chinese lottery [3])



Cryptanalysis

Brute-Force



Key Size (bits)	Number of Alternative Keys	Time required at 1 decryption/ μ s	Time required at 10^6 decryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

Cryptanalysis

Remember Entropy

The infographic is divided into three horizontal sections. The top section has a yellow background and contains the title 'Character Type Difference' in large white font, with the subtitle 'Combining ASCII, Lowercase, Uppercase, and Numeric' in smaller, multi-colored font. The middle section has a dark background with a white lightning bolt graphic on the left. It features the text '"Password"' in large white font, followed by 'Cracked just under the time it would take lightning to strike 2-3 times' in smaller white font. The bottom section also has a dark background but includes a faint image of the Mount Rushmore monument on the right. It features the text '"P@ssw0rD"' in large, multi-colored font, followed by 'Will be cracked in the same amount of time it took to carve Mt. Rushmore, or 14 years.' in smaller white font. The bottom of the infographic has a yellow footer with the 'BetterBuys' logo.

Character Type Difference
Combining ASCII, Lowercase, Uppercase, and Numeric

"Password"
Cracked just under the time
it would take lightning to strike 2-3 times

"P@ssw0rD"
Will be cracked in the same amount of time
it took to carve Mt. Rushmore, or 14 years.

BetterBuys

Don't be naive?

<https://www.betterbuys.com/estimating-password-cracking-times/>



Cryptanalysis



Dictionary attack:

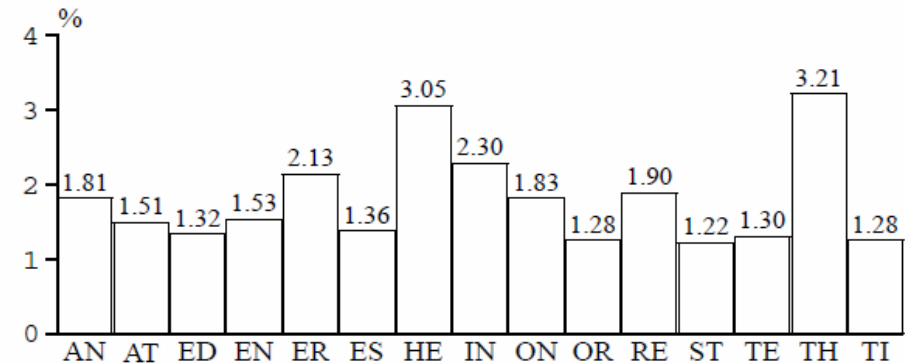
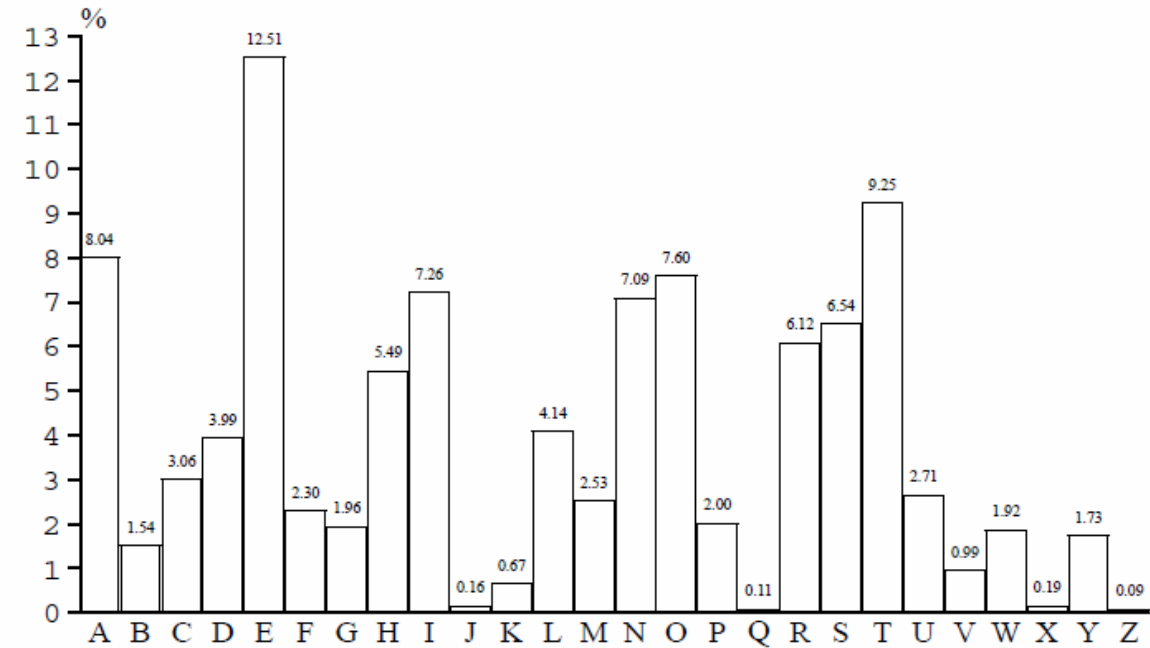
- the attacker uses a dictionary of common keys
- based on common names, places, famous people, cartoons characters, locations and names from movies, books, mythology, sports,
- works better against a file of keys
- based on a preliminary analysis of the target



Cryptanalysis

Language Redundancy and Cryptanalysis

- human languages are **redundant**
- You can read this "nfrmtcs scrty mstr"
- letters are not equally commonly used
- in English E is by far the most common letter, followed by T,R,N,I,O,A,S
- based on tables of single, double & triple letter frequencies for various languages



Letters' and digrams frequency in English language

Cryptanalysis

Letter frequency statistic attacks

- key concept - monoalphabetic substitution ciphers do not change relative letter frequencies
- discovered by Arabian scientists in 9th century
- calculate letter frequencies for ciphertext
- compare counts/plots against known values
- if Caesar cipher look for common peaks/troughs
 - peaks at: A-E-I triple, NO pair, RST triple
 - troughs at: JK, X-Z
- for monoalphabetic must identify each letter
 - tables of common double/triple letters help
- monoalphabetic substitution cipher was broken by Arabic scientists - Abu al-Kindi's "*A Manuscript on Deciphering Cryptographic Messages*", published in the 9th century but only rediscovered in 1987 in Istanbul;
- The cryptanalyst looks for a mapping between the observed pattern in the ciphertext, and the known source language letter frequencies



Cryptanalysis

Letter frequency statistic attacks

- given ciphertext:

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAI
ZVUEPHZHMDZSHZOWSFPAPDTSVPQUZWYMXUZUHSXEPYE
POPDZSZUFPOMBZWPFUPZHMDJUDTMOHMQ

Cryptanalysis

Letter frequency statistic attacks

- You get the plaintext:

it was disclosed yesterday that several
informal but

direct contacts have been made with
political representatives of the viet
cong in moscow



SECURING THE FUTURE,
ONE BYTE AT A TIME

Thank You



SECURING THE FUTURE,
ONE BYTE AT A TIME